A First Course on Population Simulation with Python

Python による個体数シミュレーション入門

Last revised: 2026 年 10 月 16 日

Applied Efilism Society

はじめに

> “*EFILism*。これは，自然のメカニズムには根本的な破滅が含まれているという私の信念の一つの表明の形である。生命（*life*）は基本的に，分子を複製するために絶えずエネルギーを奪いあっているだけに過ぎない。それは単なる *40* 億年に渡る分子変異に過ぎないのだ。あなたはただ分子を変異させ，粗暴な基準に基づいて勝者を宣言しているだけなのである。…最終的には何もなく，私が勝ち，あなたは死んだと手を挙げ，ただただそれを繰り返す。そのゲームから逃れる術はない。ただ粗暴で愚かなゲームである。それゆえに *life* が逆さから綴られているのである。それはいかなる進歩的なものにも反しており，前進的なビジョンなどなく，そのゲームに目指す場所もない。
>
> — Inmendham,
> *EFIList, 2012*

必読

本書（Ver. 0.1）は，**エフィリズム（Efilism）**の応用的取り組みに関するプロジェクトの一環として作成中の資料です。内容の誤り，不明瞭な点，改善案などがあれば，ros.therealarg@gmail.com までご連絡ください。

本テキストは，非営利目的に限り，どなたでも自由にダウンロードおよび再配布できます。ただし，本テキストを利用したことによって生じた，いかなるトラブルや不利益についても，著者は一切の責任を負いません。営利目的での使用は固くお断りいたします。

エフィリズムとは

エフィリズム（efilism）とは，Inmendham こと Gary Mosher により提唱された，生命というシステムそのものに否定的な立場で，その名前は，life の逆さつづりに由来します。

地球上の生命は，約 40 億年前，DNA の起源となる自分自身を複製する機能を持つ分子がたまたま生まれたことに端を発しています。こうした分子の特定の組み合わせである遺伝子は，複製と複製ミスのくり返しの結果，様々な機能を獲得し，その中でより安定で複製効率の高い変異だけが選抜されていきました。この過程には，意味も目的も，意志も計画も一切ありません。私たち生物の体も，たまたま現れ，遺伝子の効率的な複製に役立ったために蓄積されてきたこれらの機能の集まりに過ぎません。生物の体は，分子構造の自己保存の性質のために一時的に利用される道具に過ぎないのです。

問題はこの進化の過程で，意識という現象が生じるようになってしまったことです。元々，誰かが計画したわけでもなく，遺伝子の複製に有利に働いたという理由で引き継がれてきた形質の集まりでしかないので，その体の構造も，生物同士の関係も，生物世界の環境も，意識を持つ生物の幸福や満足に適うようにはできていません。むしろ真逆です。知覚を持つ生物にとっては苦しみがデフォルトであり，生物世界は深刻な苦しみに支配されています。私たちホモ・サピエンスを含む意識を持つ生物にとって，存在を得ることは深刻な危害を受けることなのです。その理由は，その説明を第一の目的にしているわけではない本書の内容を読むだけでも十分理解できるでしょう。

これらは，神や魂といった迷信に逃避したり，自然や生物の歴史に存在しない意味を投影して崇拝したところで，消えるのことのない事実です。エフィリズムからすれば，この事実を認めず，「生物は存続しないけばならない」という信念に囚われていることは，その信念にわかりやすい超自然的な崇拝対象が含まれなくとも，宗教を信仰することと同じなのです。

問題は実在する現象であり，定義からしてそれを経験するものにとって悪いものである「苦しみ」であって，それは生物というシステムの根本に密接に結びついています。ですから，解決策は，苦しみを回避しながら，このシステムそのものを終焉させることです。それを単なる理想で終わらせるのではなく，実現

することを目指すのが，応用エフィリズムの目的です。

すべての生物の繁殖を起こらなくするというのは，途方もない目標だと感じるかもしれません。確かにそうです。一方で，人類は古くから特定の生物種の個体数を制御し，場合によっては絶滅を目的として，生態系への介入を行ってきました。そして，その一部は目的通りの結果を収めてきました。人類は，どれもこれも究極的には上で記述した，生物世界は存続しなければならないという誤った信仰に基づいて，様々な技術開発とその応用に多大なリソースを費やしてきました。これらを可能な限り正しい目的にそそぐことができれば，全生物の平和的な絶滅ということも，原理的には決して不可能ではないでしょう。現実的な困難はむしろ，愚かな人類にこの提案を受け入れさせることです。それでも，何もしないよりは，何かを試みるべきでしょう。

本書の目的

生物の数を効果的に制御するためには，まず，生物群の大きさや構造およびその変動の仕方などをよく理解する必要があります。このような対象を扱う分野は**個体群生態学（population ecology）**と呼ばれ，その中でも特に，個体数の変動を調べる分野は，**個体群動態論（population dynamics）**と呼ばれます。現在，個体群生態学の知識が個体数制御に利用されるのは，人間の利益の観点から，生物種や生態系の「保全」を目的とする場合が一般的ですが，本当に必要なのは，それぞれの生物個体の苦しみを減らすための議論や実践に応用することです。

個体群動態論では，個体数の振る舞いを数学的にモデルし，解析的な計算や数値シミュレーションによって調べます。そのため，こうした分野の専門的背景を持たない人にとっては，何をどうやって学び始めればいいかわからない場合も多いでしょう。特に，数学に苦手意識を持つ人にとっては実際よりも困難なものに映るかもしれません。そこで，実際に手を動かしながら，初歩的なプログラミングと並行して学ぶことで，誰でも個体数動態論とそれに必要な基本的数学を独学で身に着けられるようにすることを目的として執筆されたのがこのテキストです。

シミュレーションの経験が有意義なのは，独自にモデルを開発し，新しく研究を進める人にとってだけではありません。コンピュータ上で生物群の振る舞

いを模擬しながらその性質を学ぶことで，完全に受け身の学習ではなかなか達することのできない学習の効率や深さを実現することができます。こうして得られた知識は，動物の苦しみを減らすための取り組みに，必ず役に立つことでしょう。

より専門性の高い内容の学習や，エフィリズムの応用に向けた発展についての議論に関心がある方は，Applied Efilism Society に参加してください（連絡先：ros.therealarg@gmail.com）。

目次

第 1 章　導入 1
1.1　モデルとシミュレーション 1
1.2　本書の構成と読み進め方について 2

第 2 章　Python 5
2.1　Python 環境の設定手順 6
2.2　環境構築 8
2.3　ライブラリ 9
2.4　基本概念 11
2.5　データの集まり 16
2.6　論理式と条件文 21
2.7　繰り返し処理 23

第 3 章　Malthus モデル 27
3.1　離散的 Malthus モデル 27
3.2　極限操作 35
3.3　微分積分 39
3.4　連続的 Malthus モデル 45
3.5　対数 49
3.6　この節のまとめ 54

第 4 章　密度に依存する成長 57
4.1　ロジスティック方程式 57
4.2　Allee 効果 61

4.3 基本原理とモデルの役割 63
4.4 *r-K* 淘汰理論 65

第 5 章 推移行列 67
5.1 行列 68
5.2 固有値と弾力性 75
5.3 野良ネコの行列モデル 80
5.4 クマの行列モデル 88

第 6 章 種間相互作用があるモデル 95
6.1 Lotka-Volterra 方程式 96
6.2 解の軌道 102
6.3 平衡点の安定性 107
6.4 Rosenzweig-MacArthur モデル 113
6.5 中位捕食者解放効果 116
6.6 不妊虫放飼の競争モデル 123

第 7 章 確率論的記述 129
7.1 ランダムウォーク 130
7.2 確率密度関数 136
7.3 確率論的なクマの行列モデル 138

第 8 章 空間分布と拡散モデル 143
8.1 密度分布 144
8.2 偏微分 145
8.3 拡散現象 146
8.4 拡散方程式のシミュレーション 149
8.5 反応拡散モデル 153
8.6 不妊虫放飼と侵入波・絶滅波 156
8.7 エントロピーと生物 159

第 9 章 エージェントベースモデル（ABM） 163
9.1 エージェントとは 163
9.2 オブジェクト指向 164

9.3 サンプルコード：ネコの個体数シミュレーションコード 169

付録 A 解析学 183
A.1 微分方程式の基礎 183
A.2 複素解析の基礎 187

付録 B 有限差分と近似の精度 189
B.1 Taylor 展開 189
B.2 数値積分 191

付録 C 線形代数 193
C.1 行列の基本 193
C.2 固有値問題 197

付録 D Python による出力と描画 199
D.1 グラフ作成 199
D.2 文字列のフォーマット 200

付録 E 一部ソフトウェアの利用法 203
E.1 VS Code 203

付録 F 練習問題の答え 205

索引 205

第 1 章

導入

> “我々は，意識的な苦しみに関する包括的な理論をまだ持っていない。そこで切望される主要なものの一つは，この極めて特殊な種類の現象的状態—選択の余地があるなら決して経験したくないような，素朴心理学で「苦しみ」と表現される意識状態—について，概念的に説得力があり，経験的にも妥当なモデルを構築することである。”
>
> — Thomas Metzinger,
> *Suffering, 2017*

1.1 モデルとシミュレーション

生物集団の性質やその変化を調べる最も直接的な方法は，現実の生物群を観察して分析するフィールドワークです。しかし，そのような手法が可能な対象の種類も数も限られています。例えば，二種類の異なる不妊化法の効果を比較しようとした場合，厳密にはそれ以外の条件が全く同じ個体群を用意して，その効果を調べることが望まれます。バクテリアのような生物では，実験室で十分によく似た個体群を複数用意することが可能な場合もありますが，クマや野良ネコの群れではそうはいきません。また，実際の生物集団を対象とした実験では，時間や費用の制約などから，試すことのできる条件にも限りがあります。

このような現実の生物を対象とした観察や実験を補う方法の一つが，生物群の**モデル（模型；model）**を用いることです。モデルとは，現実の対象から，調

べたい問題に関係する性質を抽出して表現したものです。そして，モデルを用いて，個体数などがどのように変化するかをコンピュータ上で計算することをコンピュータシミュレーションあるいは単にシミュレーションといいます。特に，本書が対象とするような，数式に基づくモデルを用いてコンピュータによりシミュレーションを行うことを，**数値シミュレーション（numerical simulation）**といいます。

シミュレーションで用いるのはあくまで現実のモデルであるため，それによって得られる結果が現実を正しく反映している保証はありません。あまりに単純なモデルでは，わかりやすい結果は得られるかもしれませんが，その結果が現実の何かを理解するのに意味があるとは限りません。反対に，詳細なモデルほど計算時間が長くなってしまう傾向にありますし，シミュレーションによって調べたい事柄とは無関係な性質を足したところで，解析を非効率化するだけで結果の精度は上がりません。そのため，シミュレーション研究の基本は，問題を適切に切り取り，それを捉える最小限のモデルを構築することになります。

本書では，まず個体数の増加という最も単純な現象から出発し，そこに様々な要因を少しずつ加えていきます。その過程を通して，現実の生物集団を数式でどのように表現し，その結果をどのようにシミュレーションによって調べるのかを学んでいきます。本書で扱うこれらの基礎理論は，様々な個体群の数理モデルを構築し，その振る舞いを理解するための基本的な道具となります。

1.2　本書の構成と読み進め方について

1.2.1　各章の内容

本書ではまず，第 2 章で Python によるプログラミングを始める方法を説明し，続く章から，具体的な個体群シミュレーションモデルとその実装を学んでいきます。始めに第 3 章で，最も基本的なモデルである Malthus モデルとそれに必要な数学を，Python コードのサンプルを用いながら学びます。Malthus モデルは，個体数が際限なく増加するモデルなので，第 4 章では，モデルに個体数の密度が増加した際の制限を加えます。これらのモデルでは，個体群はひとまとまりとして扱われ，その中の年齢や性別などの構成については考えられていません。そこで，第 5 章では，個体群の構成を考慮に入れた記述法を学びます。こ

こまでは，同じ種の生物から構成される個体群のみを扱いますが，第 6 章では，異なる生物種の間で相互作用がある場合を扱います。以上で扱うモデルは全て，初期の値が決まれば，その後の状態が厳密に決定できるモデルです。これに対し第 7 章では，モデルに確率的な要素を導入します。さらに第 8 章では，確率論的変動を背景として，個体群の空間分布の変化について考えます。ここまで学べば，個体群を数学的なモデルで記述するための基礎知識と手法が身に着けられるはずです。ただ，個体群を数値的に調べるために，こうした数理モデルの他に，エージェントベースモデルと呼ばれるタイプのモデルもよく用いられます。第 9 章では，このエージェントベースモデルの基礎を学びます。

1.2.2　読む順番と補足資料について

コードの説明は，適宜サンプルコードを文中で示しながら進めますが，コードをすべて書くと長くなりすぎてしまう場合があるため，一部省略したコードのみしか示さない場合があります。ただし，**こちらのリンク**[*1]から，本書で用いるすべてのサンプルコードの完成形がダウンロード可能なので，必要に応じて利用してください。

第 2 章で，Python の導入と，基本的な概念を説明しますが，コーディングについては事前に抽象的な操作をまとめて学ぶよりも，より具体的な問題を扱いながら学んだほうが理解しやすい人も多いだろうと思います。ですから，Python の実行環境が整ったら，次の章に移り，必要に応じて第 2 章に戻りながら読み進めるという方法でも全く問題ありません。その後は，順に読み進めてもいいですが，エージェントベースモデルを学ぶ 9 章の内容は他の章とは性質が異なるため，エージェントベースモデルに最も関心がある読者は，第 4 章まで読んだら，間は飛ばして第 9 章に進むことも可能です。

できるだけ多くの人になじみやすい内容にするため，本文中で用いる数学とプログラミングの手法は最小限にとどめ，詳細な数学的解説やプログラミングの手法は，Appendix で補足しています。適宜，必要に応じて本文と行ったり来たりしながら，参照してください。

いきなり数式やコードを用いた説明をされてもよくわからないという人もい

[*1] https://github.com/AppliedEfilism/Population_simulation

るかもしれません。そのように感じる場合には

- § 3.1.4
- § 4.3, § 4.4
- § 5.3.4, § 5.4.3
- § 6.3, § 6.5.1
- § 8.7

など，各章に，ほとんど数式を用いず，個体数制御において重要な概念を説明している部分もあるので，これらに先に読んで，数理モデルでどんなことを調べようとしているのかをおおまかに見てから読み始めると，見通しが良くなるのではないかと思います。あるいはそもそも，数理モデルやコーディングには関心がないけれど，エフィリズムの土台となる個体群生態学の考え方を知りたいという目的で，これらの節を拾い読みしてもらうだけでも十分です。

References

Metzinger, T. (2017). Suffering, the cognitive scotoma. In K. Almqvist & A. Haag (Eds.), *The return of consciousness* (pp. 237–262). Axel; Margaret Ax:son Johnson Foundation.

第 2 章

Python

> *“アルゴリズムの作り出すものがどんなに印象的だろうと，その下にあるプロセスは，どんなときにも，いかなる知的管理の助けもなしに継起する，個々には心を持たない一群のステップだけから成っている。…Darwin によれば，進化はアルゴリズムのプロセスなのである。”*
>
> —Daniel C. Dennett,
> *Dawrin's Dangerous Idea: The Evolution and the Meanings of Life, 1995*

コンピュータプログラムは，あらかじめ決められた機械的な手順に従って処理を実行します。このような問題を解くための機械的手順を，**アルゴリズム（algorithm）**と呼びます。どんな複雑なプログラムも，単純なアルゴリズムの集まりに分解していくことができます。言い換えると，基本的な処理を理解すれば，それを組み合わせることで様々な処理を行うプログラムが書けるようになります。プログラミングが複雑になるほど，処理を組み合わせるのにいくつもののやり方が出てくるので，その中から特に効率のいい組み方をするのがプログラミングの技術になります。しかし，ここではそんなことは気にする必要はありません。まずは確実に必要な処理を行えるようコードを書くことを覚えましょう。

プログラミングを学び，数値計算用のコードを書くことは，計算結果を得るという直接的な目的以上の効果を持っています。数学的なモデルを，こうした基本的な処理の集まりに翻訳することで，ペンを使って方程式を操作して学ぶだ

けではわからなかった方程式の性質が捉えやすくなり，理解が深まります。私たちがこのテキストを企画して以降も，AI の発達は目覚ましく，益々私たち人間がプログラミングを学ぶ意義は薄れて行っています。しかし，大規模なコードを書いて利用するのではなく，基礎的な学びに用いるのであれば，数理モデルを含め，問題を分解して処理する経験としてのプログラミングは，（学ぶという行為に意味がある限りは）少しなりとも意味を持ち続けるのではないかと思います。

本テキストでは，シミュレーションの学習に Python というプログラミング言語を用います。Python は文法が比較的簡潔で，数値計算や可視化のためのライブラリも充実しているため，初学者が手を動かしながら学ぶツールとして適しています。大規模な数値計算や高性能計算には必ずしも適してはいませんが，個体群動態の基礎的なモデルを扱う範囲では Python でも十分です。本章では，まず，Python を PC にインストールして，プログラムを実行できる環境を整える手順を説明します。その後，Python でコーディングするための基本的な説明をしますが，§ 2.3 まで進んだら，次の章に移って，必要に応じて後戻りしながら読むという方法でも問題ありません。

2.1 Python 環境の設定手順

2.1.1 インストール

最初に，PC に Python をインストールしましょう。

Windows の場合

Python の公式サイト[*1]にアクセスし，「Downloads」から最新版（Latest）をダウンロードします（図 2.1）。

インストーラーを実行する際，「Add Python to PATH」というチェックボックスを必ずオンにしてください（図 2.2）。これを見落とすと後で苦労します。

[*1] https://www.python.org

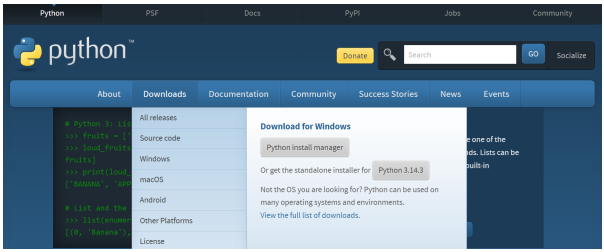


図 2.1 Python の公式サイト。



図 2.2 Python インストールの際に現れる画面。このとき，一番下の「Add Python to PATH」にチェックを入れるようにしてください。

Mac の場合

Mac には最初から Python が入っていることが多いですが，バージョンが古い場合があります。公式サイトから最新版をインストールするか，Homebrew というツールを使ってインストールするのが一般的です。

2.2 環境構築

2.2.1 開発環境を選ぶ

続いて，コードを書くためのソフトウェアを選択します。本書では，以下のソフトウェアを推奨します。

- **VS Code (Visual Studio Code)**：現在，世界で最も人気のあるエディタです（ただし，対応は Windows10 以降のみ）。コーディングの補助をする様々な拡張機能があり，Python 用の拡張機能（図 2.3）を入れれば，入力補完やエラーチェックなどのサポートが利用できます。ダウンロードは公式サイト[*2]から行えます。Appendix E.1 でより詳細な導入と利用の方法を説明しています。

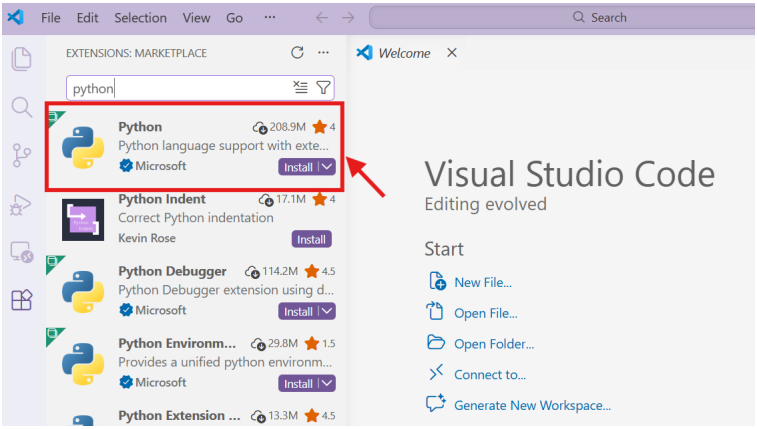


図 2.3 VS Code による Python 用拡張機能のインストール方法。左枠の一番下の拡張機能（Extension）アイコンを選択し，上の検索窓で python と入力して出てくる図の拡張機能をインストールしてください。

[*2] https://code.visualstudio.com

2.2.2 動作確認：最初のプログラム

ファイル作成

動作の確認のため，最も単純なコードを書いて実行してみましょう。例えば「hello.py」のように，xxx.py の形式の名前でファイルを作成し，以下のコードを記述します。

```
print("Hello, Python!")
```

実行

ファイルを保存（「CTRL」+「S」）したら，実行します。VS Code の場合，エディタ画面の右上にある再生マーク（▷）の「Run」ボタンをクリックして実行できます。ターミナルの場合は

```
python xxx.py
```

と入力して実行（Enter）します。

2.3 ライブラリ

2.3.1 ライブラリとは

プログラミングにおいて，すべての処理を自分で一から書くのは大変なので，特定の処理に必要なプログラムをまとめたライブラリが用意されています。ライブラリには，Python に標準で備わっている標準ライブラリと，インストールし，コード内で呼び出すことで使える外部ライブラリがあります。

私たちが使用する最も基本的な外部ライブラリが以下の二つです。

ライブラリ名	用途
NumPy	数学的処理
Matplotlib	可視化

2.3.2 ライブラリのインストールとインポート

外部ライブラリをインストールするには，以下のようなコマンドを実行します。

```
pip install numpy matplotlib
```

ライブラリを使用する際には，コードの先頭で以下のように記述します。

```
import numpy as np
import matplotlib.pyplot as plt
```

サンプルコード 2.1 基本コード

ひとつずつでも問題ありません。

`import xxx` でライブラリを読み込み，`as` で省略名を与えます。必ずしも省略名を与えなくてもいいのですが，以下で説明するように，ライブラリの中の関数を使用する際に，ライブラリ名を記述する必要があるので，上の例のような省略をするのが標準的です。以降，すべてのコードで，出発点をサンプルコード 2.1 とします。

2.3.3 ライブラリの使用例：Matplotlib による描画

ライブラリの使用例として，Matplotlib を用いて以下のようなグラフをプロット（描画）してみましょう。

```
import matplotlib.pyplot as plt

# サンプルデータの用意
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

# プロットの作成
plt.plot(x, y, label='Sample')

# タイトルやラベルの表示
plt.title('Sample')
plt.xlabel('x')
plt.ylabel('y')
plt.legend() # 凡例を表示
```

```
#plt.savefig('image.png')
plt.show() # グラフを表示
```

サンプルコード 2.2 描画例

先に述べたように，インポートしたライブラリの関数を用いるには，plt.plot のように，先頭にライブラリ名（ここでは plt）を記述する必要があります。これが，省略名を一般に用いる理由です。

グラフを画像として保存する場合は，#plt.savefig('image.png') の先頭の#を消します（#の意味は以下で説明します）。保存するファイル名を変えるには image.png の image 部分を変更します。描画や出力についての詳細は，Appendix D も参照してください。

2.4 基本概念

その他，プログラミング一般に共通するものを含め，Python を利用するために必要な基本概念を説明します。

2.4.1 型

データには**型（type）**があり，型に応じてできる操作が異なります。数値には，$\ldots, -2, -1, 0, 1, 2, \ldots$ のような**整数（integer）**と，1.0, 3.14, −0.11111 などの**実数（real number）**があります（複素数もありますが本書では扱いません）。Python では，整数は **int** 型，実数は **float** 型と呼ばれます。また，最初のコードで利用した"Hello，Python!"のような文字の並びもデータであり，**文字列（string）**と呼ばれます。他にもいろいろとありますが，必要なものは適宜説明していきます。

2.4.2 コメントとコメントアウト

サンプルコード 2.2 には，#で始まる文が含まれています。これらは，**コメント（comment）**と呼ばれ，コードとは認識されずプログラム実行時に無視されます。そのため，開発者間の意思疎通や自分が忘れないためのメモ書きに利用されます（自分で書いたコードの意図や目的は驚くほど忘れます）。また，コー

ドの一部であっても，#をつけることでコメントとして無視されるようになるため，プログラムの一部を削除せず無効化するために利用されます。このようにしてコードの一部を無効にすることを，**コメントアウト（comment out）**するといいます。例えば，別の方法を試して比較したい場合や，エラーが出てプログラミングがうまく実行できず，その原因を特定したい時などに，コメントアウトが利用されます。

2.4.3 演算

数値どうしの足し算は以下のように行います。演算の結果は新しい値になり，=の右側で計算した値を左側の変数に代入します。

足し算や引き算はそのまま

```
a=1+2
b=3-2

print(a,b)
# 結果：
3 1
```

のようにします。掛け算と累乗は，「*」を用いて

```
a=4*2
b=2**4

print(a,b)
# 結果：
8 16
```

とします。

割り算に関しては少し注意が必要です。/による割り算の結果は，割り切れる場合でも float 型になります。例えば 4 / 2 の結果は 2.0 です。割り切れない場合，5/2 の結果は 2.5 です。一方，//は商を切り下げ，整数を与えます。例えば 5//2 の結果は 2 となります。% は余りを求めます。//と % は，例えばステップ番号から周期的な処理を行うときに利用できます。基本的な演算子を表 2.1 に示します。

複数の演算子を含む式では，通常の数学と同様に，累乗（**），掛け算と割り算（*，/，//，%），そして足し算と引き算（+，-）の順に計算されます。意図

表 2.1 数値の主な演算子

演算子	意味	例	結果
+	足し算	3 + 2	5
-	引き算	3 - 2	1
*	掛け算	3 * 2	6
/	割り算	5 / 2	2.5
//	整数除算（商を切り下げ）	5 // 2	2
%	余り	5 % 2	1
**	累乗	2 ** 4	16

を明確にしたいときや計算順序を変えたいときは，括弧を使います。

```
x = 1 + 2 * 3       # 7
y = (1 + 2) * 3     # 9
```

演算子は型によって意味が変わることがあります。文字列では + は連結，*は繰り返しを表します。

```
message = "Hello" + " Python"  # "Hello Python"
line = "-" * 10                # "----------"
```

数値と文字列をそのまま足すことはできません。例えば"step: " + 3 はエラーになるため，文字列に変換して"step: " + str(3) と書きます。

変数自身の値を更新する場合は，+=などの代入演算子を使うと簡潔です。

```
count = 0
count += 1        # count = count + 1 と同じ
count *= 2        # count = count * 2 と同じ
```

2.4.4 関数

プログラムの中でくりかえし行う計算や，意味のあるひとまとまりの処理に名前をつけて整理したものを**関数（function）**と呼びます。これまでに使ってきた print() や str() も，Python に最初から組み込まれている関数（組み込み関数）の一種です。

関数は，外部からデータを受け取り，内部で処理を行い，その結果を返します。このとき，関数に引き渡すデータを**引数（argument）**，関数から返ってくる処理結果を**戻り値（return value）**（または返り値）と呼びます。

独自の関数を定義（define）するには，以下のように `def` の後に関数名と引数を書き，最後にコロン `:` を置きます。

```
def add(x, y):
    result = x + y
    return result

a = add(3, 5)
print(a)
# 結果：
8
```

関数の内部で実行する処理は，次の行からインデント（字下げ）して記述します。

上の関数では，`result` を返していますが，引数や戻り値は必ずしも必要ではありません。例えば，引数も戻り値も持たず，単にメッセージを表示するだけの関数を作ることもできます。

```
def say_hello():
    print("Hello, Python!")

say_hello() # 関数の呼び出し
```

また，引数にデフォルト値を指定すると，呼び出しの際にその引数が省略された場合，デフォルト値が利用されます。

```
def say_something(message="Hello, Python!"):

    print(message)

say_something()
# 結果：
Hello, Python!

say_something("Go vegan!")
# 結果：
Go vegan!
```

2.4.5 ファイル分割

コードを開発していくと，どんどん行数が増え，扱いがしづらくなっていきます。そこで，役割毎にファイルを分けて管理することが有用になります。例えば，あるコードを project というディレクトリ（フォルダ）で管理するとして，コードの中心部分である main.py に加えて，パラメータを設定する config.py というファイルを作成し，以下のように配置します。

```
project/
    main.py
    config.py
```

config.py には

```
# config.py

NUM_AGENTS = 100 # 個体数
STEPS      = 1000 # 計算ステップ数
P_SURVIVE  = 0.9 # 生存確率
```

のように，計算の構造ではなく，計算に利用するパラメータの具体的な値を指定するようにします。そして，main.py の方で，外部ライブラリ（§ 2.3）を利用するのと同じように，import config と記述して，config.py の中身を利用します。

```
# main.py
import config

def main():
    for step in range(config.STEPS):
        print("step:", step)

if __name__ == "__main__":
    main()
```

config.py 内の STEPS という変数を利用するのに，config.STEPS という書き方をしています。これも，外部ライブラリの関数を利用する方法と同じです。このようにすることで，同じモデルの計算を設定だけを変えて実行するのに便利になります。別にメインファイルの処理を，def main(): のように関数として

書かなくてもいいのですが，色々と訳があってこのようにする慣例になっています。それに合わせて，最後に

```
if __name__ == "__main__":
    main()
```

という処理が書かれます。これも，深く考えず，メインのファイルにはおまじないのように書いておくものと思ってもらえばいいです。

このプログラムを実行するには

```
    python main.py
```

とします。上で触れた `if __name__ == "__main__":` という処理は，このようにメインファイルが直接実行された際にのみ，関数 `main()` を実行するためのものです（構成によっては別のファイルから `main.py` の中身を参照することもあり，こうしておかないと，その際に意図せず `main()` が実行される可能性があります)。

計算の具体的な内容も，それ用の別ファイルに記述し，それをまた `main.py` で呼び出すという方法を取ることで，どこで何が行われているかわかりやすくなり，管理がしやすくなります。

2.5 データの集まり

2.5.1 リスト

サンプルコード 2.2 には

```
x=[1,2,3,4]
```

のように，角かっこで値を囲んだ部分があります。このようなデータ型は**リスト（list）**と呼ばれ，直感的にもわかりやすいように，値をまとめて扱う型になります。その中身は**要素（element）**と呼ばれ，リストの要素には整数や float 以外のデータ型を含めることや，それらを混在させることもできます。

リストの要素を取り出すには，左から数えた番号を指定して

```
x0 = x[0]
x3 = x[3]
```

```
print(x0,x3)

# 結果
1 4
```

のようにします。要素を指定するこの番号のことを**インデックス（index）**と呼びます。**インデックスは 1 ではなく，0 から始まることに注意してください**。また，要素は後ろ側からも指定できて，x[-1] のようにすると最後の要素を指し，x[-2] とすると，最後より 1 つ前の要素を指します。

リストに新たに要素を追加する場合は append を使います。cat という要素を持つリスト animal に dog という要素を追加してみましょう。

```
animal = ["cat"]
animal.append("dog")
print(animal)

# 結果
['cat', 'dog']
```

反対に，要素を取り除くときは，remove を用いて

```
    animal.remove('dog')
```

のようにします。

リストを結合するには，extend を使います。リスト animal と，別のリスト animal2 を結合してみましょう。

```
animal = ["cat","dog"]
animal2 = ["rat","horse"]
animal.extend(animal2)
print(animal)

# 結果
['cat', 'dog', 'rat', 'horse']
```

リストの要素数のことを**サイズ（size）**といい，関数 len を用いて

```
    N=len(animal)
```

とすることで取得できます。例えば第 9 章で扱うエージェントベースのコードでは，これと同じ方法で，生物の個体をリストにまとめて，個体数をそのリストのサイズとして計算します。

2.5.2 配列

リストと似た別のデータ型に**配列（array）**があります。リストも広い意味でいえば配列の一種なのですが，Python で一般に配列というと NumPy の関数である numpy.ndarray が利用されるので，本書では配列といえばそれを指すものとします。

直感的には，物理的な関数を数値的に扱うためのものと思ってよいでしょう。例えば時間 t の値を指定すると値を与える関数 $f(t)$ があったとします。このとき，たとえそれが連続的な関数として扱われるべきものであっても，数値的にはどうしてもとびとびの値の集まりとしてしか表現できません。そこで，Python では，時間を適当な数 n 個に分割し，t と f を同じ n 個の要素の数を持つ配列として作り，同じラベル i で時間点を指定するということが一般的になされます。つまり，i 番目の時間点を指定すれば，それに対応する時間と，その時の $f(t)$ の値が整合的に得られるようにするのです。このような使い方から想像できるように，配列は一般に物理量とみなして様々な演算を実行できます。また，配列の場合は，リストと異なり，同じ種類の型しか要素にもてません。

今述べた時間の配列のように，等間隔な数値を持つ配列を利用することがよくあります。このような配列は，関数 `linspace` または `arange` によって生成できます。`linspace` は，引数に (最初の値, 最後の値, 要素の数) の順に値を与えます。要素には最後の値も含まれます。対して，`arange` では，(最初の値, 最後の値, 要素の間隔) のように与えます。この場合は最後の値は含まれません。例えば，1 ずつの間隔で，0 から 20 までの配列を作るには，次のようにします。

```
# linspaceの場合
time = np.linspace(0, 20, 21)
print(time)

# arangeの場合
time = np.arange(0, 21, 1)
print(time)
```

単に，0 から 1 間隔で N 個の要素を持つ配列を作りたければ `np.arange(N)` のようにもできます。

```
# 例
```

```
print(np.arange(5))

# 結果
[0 1 2 3 4]
```

これとは異なり，初めに配列の存在だけを用意して，後から順に中身を入れ替えていくという作業をすることも多いです。その場合，表 2.2 のような関数が利用されます。また，`np.zeros_like` のように，これらの関数の最後に「`_like`」

表 2.2 NumPy における空・初期化配列の作成関数

関数名	コード例	中身と主な用途
`empty`	`np.empty((2, 3))`	中身はゴミデータ。直後に全上書き用。
`zeros`	`np.zeros((2, 3))`	指定した形状をすべて 0 で初期化。最も安全で一般的。
`ones`	`np.ones((2, 3))`	指定した形状をすべて 1 で初期化。掛け算の初期値などに。
`full`	`np.full((2, 3), np.nan)`	指定した形状を任意の値（NaN など）で満たして初期化。

をつけると，既存の配列と同じ形状・型で初期化した配列を作成することが可能です。

2.5.3 辞書（ディクショナリ）

リストや配列では，`x[0]` のようにインデックス（番号）を指定して要素を取り出さなくてはなりませんでした。しかし，「0 番目が年齢で，1 番目が体重で...」と番号で管理するのは，要素が増えると間違いのもとになります。より直感的でわかりやすいデータ管理を可能にするのが，**辞書（dictionary）**です。カタカナで**ディクショナリ**と呼ぶことも多いです。

辞書では，番号の代わりに「名前（文字列など）」を使ってデータを管理しま

す。データを探すための名前を**キー（key）**，それに対応する中身を**値（value）**と呼び，このペアの集まりとしてデータを保持します。

辞書を作成するには，波かっこ{}を使い，キーと値をコロン: で繋いで記述します。例えば，個体群モデルにおける環境条件ごとのパラメータ設定を，以下のように辞書として定義できます。

```
good = {'S': 0.90, 'L': 1.10}
poor = {'S': 0.85, 'L': 0.60}
```

ここでは豊作年と不作年における，個体の生存率 S と産子数 L を辞書のキーとして定義しています。

辞書から値を取り出すには，角かっこ [] の中にキーを指定します。

```
print(good['S'])
# 結果：
0.9
```

存在しないキーに値を代入すると新しいペアが追加され，既存のキーに代入すると値が更新されます。

```
# 既存の値を更新
good['S'] = 0.95

# 新しいパラメータを追加
good['tau'] = 0.75
```

辞書は，複数のパラメータ群を一括で管理し，関数にそのまま受け渡す際に特に有用となります。Python では，関数を呼び出す際につける引数名と辞書のキー名が一致している場合，辞書の前に**をつけることで，辞書の中身をキーワード引数としてそのまま展開して渡すことができます。例えば

```
    def make_matrix(S, tau, L):
        （省略）

        return A
```

のような関数があったとして，これを呼び出す際に

```
    A_good = make_matrix(0.90, 0.75, 1.10)
```

とする代わりに，辞書に「**」をつけて

```
A_good = make_matrix(**good)
```

としても，全く同じ意味になります。引数が三つくらいならあまり違いは感じられないかもしれませんが，§ 5.4 で扱うような，より引数の数が多い関数を用いる場合には大きな違いが生まれます。

2.6 論理式と条件文

2.6.1 比較演算子

数値変数同士の値を比較には，比較演算子を使います。これらは，それが成り立つときに `True`，成り立たないときに `False` となる式で，例えば以下のようになります。

```
print(1>2)
print(3<=8)

# 結果：
False
True
```

この例で用いたものを含め，主な比較演算子を表 2.3 に示します。

表 2.3 比較演算子

演算子	意味	例
==	等しい	x == 3
!=	等しくない	x != 3
<	より小さい	x < 3
<=	以下	x <= 3
>	より大きい	x > 3
>=	以上	x >= 3

等しいかどうかを調べるときは==を用います。値を変数に代入する=とは役割が異なるため，混同しないように注意してください。

2.6.2 論理演算子

論理演算子 and，or および not も，True か False を返します。これらを用いることで，複数の条件を組み合わせることができます。and は両方の条件が成り立つとき，or はいずれかの条件が成り立つときに True となります。not はその否定です。

2.6.3 条件式と if 文

上で導入した論理式の真偽に基づいて，処理を分けたいことが頻繁にあります。このように条件によって実行する処理を分けるには，if 文を用います。基本的な形は次のとおりです。

```
if 条件式:
    条件式が成り立つときに実行する処理
```

条件式の末尾には: を書き，下に続く条件を満たす場合の処理は，スペース 4 個分を字下げして書く必要があります。

例えば，個体の年齢 age が，5 以上ならば，年齢の分類 age_category を"adult"に設定するには，次のようにします。

```
age = 10

if age >= 5:
    age_category = "adult"
    print("age_category:", age )

# 結果:
age_category: adult
```

ここに現れる>=は，不等号 ≥ に対応しています。これを含めた主な条件式は下で説明します。

条件式が成り立たない場合に別の処理を行うには，else を用います。

```
age = 4

if age >= 5:
    age_category = "adult"
else:
```

```
    age_category = "child"

print("age_category:", age_category)
# 結果：
age_category: child
```

さらに複数の条件を順に判定したい場合は，elif を使います（else if の略です）。

```
value = 0.5

if value >= 0.8:
    print("A")
elif value >= 0.6:
    print("B")
else:
    print("C")

# 結果：
C
```

if から elif，else の順に条件が判定され，最初に成り立った処理だけが実行されます。else は必須ではなく，必要な場合にだけ記述します。

2.7 繰り返し処理

2.7.1 ループ

変数の値を変えながら同じ処理を繰り返す場合には，ループ構文を利用します。for 文は，リストなどのオブジェクトから値を一つずつ取り出して処理します。基本構造は以下のとおりです。

```
for 変数 in オブジェクト:
    実行する処理
```

繰り返す回数を指定したいときは，range をよく用います。range(5) は 0 から 4 までを順に返します。このように，range(**終点**) では終点の値は含まれません。

```
for i in range(5):
    print(i)
```

```
# 結果：
0
1
2
3
4
```

開始値，終点，増分を指定する場合は，それぞれ range(**開始値，終点，増分**) と書きます。

表 2.4 range の主な指定方法

記述	取り出される値	用途
range(5)	0，1，2，3，4	5 回繰り返す
range(2，5)	2，3，4	開始値を指定する
range(0，10，2)	0，2，4，6，8	2 ずつ進める

リストの各要素に同じ計算を行う例を見てみましょう。

```
values = [1, 2, 3, 4]
for value in values:
    square = value ** 2
    print(square)

# 結果：
1
4
9
16
```

ここでは，リストの要素が順に変数 value に代入されます。i は慣例的によく使われる変数名ですが，population や age のように役割が分かる名前を使っても構いません。

2.7.2 while 文

while 文は，条件式が True である間，同じ処理を繰り返します。条件式と字下げの書き方は if 文と同様です。

```
while 条件式:
    実行する処理
```

例えば，count が 5 未満である間，その値を表示して 1 ずつ増やす処理は次のようになります。

```
count = 0

while count < 5:
    print(count)
    count += 1

# 結果：
0
1
2
3
4
```

while 文では，いつ条件が成り立たなくなるかを必ず確認してください。上の例で count += 1 を書き忘れると，count < 5 が常に成り立ち，処理が終わらない無限ループになります。

2.7.3 ループのスキップと抜け方

処理をスキップして次の繰り返しに進むには continue を用います。例えば，偶数を飛ばして奇数だけを表示するには次のようにします。

```
for i in range(6):
    if i % 2 == 0:
        continue
    print(i)

# 結果：
1
3
5
```

continue が実行されると，それより下にある同じ繰り返し内の処理は実行されず，次の値に進みます。

ループそのものをただちに抜けるには break を用います。例えば，リストから最初に 0 以下となる値を見つけた時点で処理を終えるには，次のようにし

ます。

```
population = [12, 8, 3, 0, -1]

for n in population:
    if n <= 0:
        print("population is zero or negative")
        break
    print(n)

# 結果：
12
8
3
population is zero or negative
```

break の後にある-1 は処理されません。

References

Althoff, C. (2018). **独学プログラマー** *Python* **言語の基本から仕事のやり方まで** (清水川貴之, Trans.). 日経 BP.

国本大悟 & 須藤秋良. (2023). **スッキリわかる** *Python* **入門** (second). インプレス.

第 3 章

Malthus モデル

> “人口は，制限がなければ幾何級数的に増加するが，生活の糧は算術級数的にしか増加しない。”
>
> — Thomas Robert Malthus,
> *An Essay on the Principle of Population, 1798*

3.1 離散的 Malthus モデル

この章から，具体的な数理モデルとその実装法を扱っていきます。本章では，最も単純な規則で，生物の個体数が変化していく過程を数学的に記述し，それを計算するプログラムの書き方を学びます。Python のプログラム例を示しながら理論的な説明を行いますので，適宜必要があれば実際にコードを動かして確かめながら進めてください。サンプルコードは，コード 2.1 を基本とし，ライブラリのインポートはたびたび省略しますので注意してください。

ある生物の個体数を調べるにあたって，私たちがまず知りたいことの一つは，放っておいたらその個体群は，どれくらいの時間でどれくらいの数に増えてしまうのかということです。私たちの介入がなくても，生物は生物も無生物も含む環境との相互作用により様々な制限を受けます。しかし，とりあえずそうした影響は全て無視して，その生物に内在する繁殖の性質のみに従い，何の制限も

なく個体数が変化していく理想的な場合を仮定します[*1]。まず，この仮定の下で，与えられたデータから，未来の個体数を計算するための数学的方法と，それをコンピュータに計算させるコードの書き方を学びます。

3.1.1　総和

最初，個体数が N_0 の，ある生物集団について考えます。個体数の時間変化を調べるため，この集団の個体数の変化を，ある時間間隔 Δt ごとに記録することにします[*2]。すると，Δt 間隔のとびとび時間ごとに個体数の値が集まります。このように値が不連続なことを，**離散的（discrete）**といいます。

まず，初期時刻から Δt の間に，個体数が ΔN_0 だけ変化したとすると，時間 Δt 後の個体数は

$$N_1 = N_0 + \Delta N_0 \tag{3.1}$$

と表せます。ただの足し算です。次の時間幅 Δt の間に変化した数は ΔN_1 であったとすると，初期時刻から $2\Delta t$ 経った後の個体数は

$$\begin{aligned} N_2 &= N_1 + \Delta N_1 \\ &= N_0 + \Delta N_0 + \Delta N_1 \end{aligned} \tag{3.2}$$

となります。このようにして n ステップ先まで観察すると，初期時刻から時間 $n\Delta t$ 経った後の個体数は

$$\begin{aligned} N_n &= N_{n-1} + \Delta N_{n-1} \\ &= N_0 + \Delta N_0 + \Delta N_1 + \cdots + \Delta N_{n-1} \end{aligned} \tag{3.3}$$

と表せます。最後の式の N_0 より右側は，変化分を「0 番目から $n-1$ 番目まで足し合わせる」という繰り返しの作業です。これをすべて書き下すのは煩わしいので，かっこ内の指示をまとめて表す**総和記号（summation symbol）**$\sum_{k=0}^{n-1}$ を導入して

$$\sum_{k=0}^{n-1} \Delta N_k = \Delta N_0 + \Delta N_1 + \cdots + \Delta N_{n-1} \tag{3.4}$$

[*1] 数理的な文脈でいう「理想的」という意味は，考えたい性質の本質的な部分だけを残し，残りの要素は全て無視してしまうことを意味します。誰かにとって望ましい状況というようなニュアンスは含まれません。

[*2] 「Δt」でひとまとまりの量です。Δ は「デルタ」と読むギリシャ文字で，ある量の一片や変化分を表すのによく用いられます。

と書き表すことにします。すると (3.3) は

$$N_n = N_0 + \sum_{k=0}^{n-1} \Delta N_k \tag{3.5}$$

と簡潔化できます。和の操作 (3.4) は，プログラミングでいうところのループ構文（§ 2.7）そのものですから，(3.4) は実際に以下のようなコードで表現できます。

```
# 変化分のリストを適当に用意
dN_list = [1, 2, 3]

# 初期値を0に設定
total = 0

# リストの中身を順にdNとして取り出す
for dN in dN_list:

    total = total + dN

# プリントして値を確認
print(total)
```

`total = total + dN` は `total` に `dN` を足したものを新たに `total` とするということを意味します。「=」を数学的な等式の感覚で見ると違和感がありますが，プログラミングではこのような書き方がなされます。すぐに慣れます。§ 2.4.3 で説明したように，同じものを `total += dN` と書いてもよいです。

3.1.2 成長率

ある個体群の中の個体数の変化 ΔN は，その個体群に新たに加わった数と，そこからいなくなった数の差で決まります。今，個体群の外からやってきて新たに加わる個体や，個体群から出ていく個体については考えないことにします。すると，個体数の増分は繁殖により新たに生まれた個体の数 ΔN_{birth} に，減少分は死亡した個体の数 ΔN_{death} になり，正味の変化は

$$\Delta N = \Delta N_{\text{birth}} - \Delta N_{\text{death}} \tag{3.6}$$

で決まります。

ある時間ステップで，個体数が $N = 100$ だったとします。その後の Δt の間に，1000 体が生まれたとします（わかりやすく大きな数で考えます）。つまり，この場合 $\Delta N_{\text{birth}} = 1000$ です。その元の 100 体の中には，10 体生んだ個体も 1 体も生まなかった個体もいるかもしれません。また，Δt の時間のうち，最初の方に生まれた個体と，後の方に生まれた個体がいるかもしれません。どうであれ，新たに生まれた 1000 体を，元の個体数 N で割ると，「一体当たりが Δt の間に生んだ平均的な個体数」

$$\frac{\Delta N_{\text{birth}}}{N}$$

が得られます。今の場合，$1000/100 = 10$ です。さらにこれを Δt で割ると，「一体当たりが単位時間当たりに生んだ平均的な個体数」

$$\frac{\Delta N_{\text{birth}}}{N\Delta t}$$

が得られます。これを，**出生率（birth rate）**と呼び，ここでは b で表すとします。

時間の単位を 1 時間とし，時間刻み Δt を 0.1 時間とすれば，$b = 1000/(100 \times 0.1) = 100$ となります。一体あたりが $\Delta t = 0.1$ の間に生んだ平均数が 10 だったので，単位時間 $10\Delta t = 1$ の間に生むのは 100 体という計算になっています。逆に b がわかっていると，個体数 N と時間幅 Δt をかければ，それらの個体から Δt の間に生まれる総数が $\Delta N_{\text{birth}} = bN\Delta t$ により計算できます。

死亡する数も同じように考え，元の数と時間間隔に比例し，$dN\Delta t$ と書けるとします。この d は，単位時間内に死ぬ個体あたりの数で，**死亡率（mortality rate）**と呼ばれます。そして，出生率と死亡率の差

$$r_{\Delta t} = b - d \tag{3.7}$$

は正味の増加率を表し，（離散的な）**成長率（growth rate）**などと呼ばれます。r に添え字 Δt をつけているのは，これが Δt の間の平均的な値で，Δt の取り方で値が変わる量であることを忘れないためです。次節で，Δt の取り方と $r_{\Delta t}$ との関係について詳しく説明します（本来なら b と d にも同じように添え字をつけるべきですが，ここでは省略します）。以下では，一度 Δt を決めた後，b と d，したがって $r_{\Delta t}$ が時刻によらず一定である場合を考えます。

上の説明より，このモデルでは，最初のステップでの変化分は

$$\begin{aligned}\Delta N_0 = &\underbrace{bN_0\Delta t}_{\text{生まれた総数}} - \underbrace{dN_0\Delta t}_{\text{死んだ総数}} \\ =&r_{\Delta t}N_0\Delta t\end{aligned} \tag{3.8}$$

で与えられます。よって N_1 は

$$\begin{aligned}N_1 =&N_0 + \underbrace{r_{\Delta t}N_0\Delta t}_{\Delta N_0} \\ =&(1+r_{\Delta t}\Delta t)N_0\end{aligned} \tag{3.9}$$

と計算できます。二行目で N_0 にかかる部分をひとまとめにしました。

次の時間 Δt の間で，今度は個体数 N_1 に比例して出生と死亡が起こります。よって，さらに Δt 経った後，個体数は $\Delta N_1 = r_{\Delta t}N_1\Delta t$ だけ増えて

$$\begin{aligned}N_2 =&N_1 + r_{\Delta t}N_1\Delta t \\ =&(1+r_{\Delta t}\Delta t)N_1\end{aligned} \tag{3.10}$$

となります。ここで，N_1 に (3.9) の右辺を代入すると

$$\begin{aligned}N_2 =&(1+r_{\Delta t}\Delta t)N_1 \\ =&(1+r_{\Delta t}\Delta t)^2N_0\end{aligned} \tag{3.11}$$

のように，N_1 ではなく，最初の個体数 N_0 を用いて表せます。

この規則性から，任意のステップ数 n 経った後，つまり最初の時刻から $n\Delta t$ 時間後の個体数は

$$\begin{aligned}N_n =&(1+r_{\Delta t}\Delta t)N_{n-1} \\ =&(1+r_{\Delta t}\Delta t)^nN_0\end{aligned} \tag{3.12}$$

のように，最初の個体数と成長率および時間間隔のみを用いて書けます。これは重要なことです。このような方程式が分かっていれば，原理的には初期値と時間変化の基本的な性質を表す量（ここでは成長率 $r_{\Delta t}$）さえ知られれば，Δt 間隔の時間点であれば，未来も含め，任意の時間点での個体数を計算できるのです。これが，生物の個体数を含めた，様々な対象の動態（ダイナミクス）を理解する上で，数理的な方法が強力なツールとなる大きな理由の一つです。

補足 3.1 (単位と次元)**.** 今，時間幅 Δt の具体的な値は決めていません。考える時間の基準を，1 分だったり，1 時間だったり，1 日だったりと決めると，Δt を含めた時間に関する量の値も，それに基づいて決められます。例えば時間の基準を 1 時間（h）と決めたら，Δt はその 1/10 で，0.1 時間というように決められます。このような基準を，**単位（unit）**と呼び，この場合なら $\Delta t = 0.1$ h のように，必要に応じて値の後ろに明示したりします。

単位は問題に応じて適切な値を決めることができます。しかし，時間の単位に 1 メートルや 1 グラムを選ぶことはできないことはわかります。時間と長さや重さ（正確には質量）は違う概念だからです。時間や長さといったこのような基本的な量を，**次元（dimension）**といいます。

個体数 N は，1 匹とか，1 組とか，1 ダースとか，様々な単位を用いて数えられますが，その次元は時間でも長さでもありません。このような量の次元は，無次元であるといいます。出生率 b や死亡率 d は，時間当たりの個体数変化に関する量なので，時間の逆数の次元を持ちます。なので，出生「率」b が分かっていたとして，そこから無次元量である出生「数」を求めるには，その時の個体数 N だけでなく，時間の次元を持つ Δt をかけて $bN\Delta t$ とする必要があります。

3.1.3　離散的方程式のコード

上のモデルをコードで表してみましょう。適当な時刻の配列に対し，各時刻での個体数 N の配列を作り，それを描画することにします。

時刻 $t = 0$ から $t = 1$ までの時間範囲を考えます。初期時刻と終時刻をそれぞれ $t_0 = 0$ および $t_{n-1} = 1$ とし，それを幅

$$\Delta t = \frac{t_{n-1} - t_0}{n-1} \tag{3.13}$$

の $n-1$ 個の区間に分けます。このとき，時間点 $t_0, t_1, ..., t_{n-1}$ は全部で n 個であることに注意してください（図 3.1 を参照）。

これに応じて時間の配列を作るには，サンプルコード 2.2 の `x` や `y` でやったように，`time=[0,1/(n-1),2/(n-1),...]` とリストを作ればいいのですが，これだと数が増えると大変です。そこで，§ 2.5.2 で説明した `linspace` を利用して

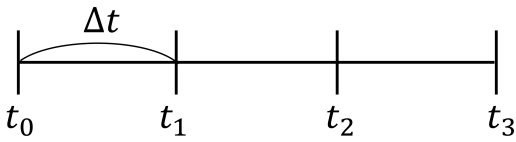


図 3.1 例として $n=4$ とした場合の時間分割 (3.13) の概念図。両端を含め，等間隔に時間点を t_0，t_1，t_2，t_3 と，0 から数えて全部で 4 個取ると，それらの間に幅 Δt の小区間が 3 個できます。同様に，n 個の時間点を取ると，$n-1$ 個の小区間ができます。

```
n=6
t0 = 0
tf = 1
time = np.linspace(t0, tf, n)
```

とします。

続いて，個体数のために，time と同じ大きさの配列を作りましょう。そのためには

```
Npop = np.zeros(n)
```

のようにします。こうすると，n 個の要素数を持ち，各要素の値がすべてゼロの配列が作られます。この配列の中身を，各時刻での個体数の値に書き換えていきましょう。単純に考えると，まず初期値と成長率および時間ステップ幅 dt を用意して

```
dt = (tf-t0)/(n-1)
r  = 1.1    # 成長率
Npop[0] = 10 # 初期値
Npop[1] = (1 + r*dt) * Npop[0]
Npop[2] = (1 + r*dt) * Npop[1]
```

のようにしていけばいいのですが，これも同じ操作の繰り返しですから，ループ構文（§ 2.7）を使って

```
for i in range(1,n):
    Npop[i] = (1 + r*dt) * Npop[i-1]
```

のようにできます。

これらを合わせると，次のようなコードにまとめられます。

```
import numpy as np
import matplotlib.pyplot as plt

n=6
t0 = 0
tf = 1
dt = (tf-t0)/(n-1)
r=1.1

time = np.linspace(t0, tf, n)
Npop = np.zeros(n)

Npop[0]=10
for i in range(1,n):
    Npop[i] = (1 + r*dt) * Npop[i-1]

plt.plot(time, Npop)
plt.xlabel("time")
plt.ylabel("population")
plt.show()
```

サンプルコード 3.1　離散的 Malthus 方程式 (3.12) の振る舞いを描画するサンプルコード。

実際に描画して確かめてみてください。

ちなみに，ここでは初期の個体数として $N_0 = 10$ としていますが，必ずしも実際に 10 体の生物を想定しているわけではありません。シミュレーションでは，値を扱いやすくしたり，精度を安定させたりする目的で，変数を特定の基準値で割って**規格化（normalization）**することが一般的です。本書でも，断りがない限り，シミュレーションに用いる変数は何らかの特徴的な量を基準に規格化されていると理解してください。

3.1.4　Malthus の法則

この節では，各個体を区別せず，個体数の増加は何の制限もなく元の個体数に比例して起こると仮定した際の振る舞いを調べました。これらの仮定に基づくモデルは **Malthus モデル（Malthusian model）**と呼ばれます。

Malthus モデルでは，個体数を N_0, N_1, N_2,...,N_{n-1},N_n と時系列に並べてみると，隣り合う個体数の比 N_n/N_{n-1} はいつも一定になっています。このような規則を持つ数の並びを，等比数列あるいは幾何数列といいます。これらの性質ゆえに，制限がない場合，個体数はあっという間に爆発的に増大してしまうことが確認できたと思います。対して，N_0, $2N_0$, $3N_0$... のように，一定数ずつ増加する並びを，等差数列や算術数列といいます（そして，数列の和を級数といいます）。少なくとも Malthus の時代には，食糧生産は算術級数的にしか増加しないと考えられていました。これが，冒頭で引用した Malthus の指摘であり，いわゆる Malthus の人口原理や法則と呼ばれるものです。

今では，人間社会ではリソースの生産効率は向上しましたが，制限がなければ幾何級数的に増殖する性質は変わりません。Malthus の指摘として重要なのもそちらの性質です。そして，人間以外の生物においては，リソースの制限はより厳しいものになります。この事実が，Darwin による自然淘汰の法則の発見にもつながりました (Darwin, 2020; Singleton, 2025)。

3.2 極限操作

3.2.1 時間刻み幅の影響

前節では，ある時点での個体数と，成長率が分かっている場合，そこから別の時刻の個体数を求める式を扱いました。今度は，ある個体群が Malthus 的な成長をしていると仮定した場合，個体数の値から，成長率を求めることを考えます。

二つの異なる時間での個体数 N_n と N_{n-1} が分かっている場合，式 (3.12) を変形することで

$$\frac{N_n - N_{n-1}}{\Delta t} = r_{\Delta t} N_{n-1} \tag{3.14}$$

から成長率を求めることができます。ここから，式 (3.12) を通して，Δt 間隔ごとであれば，どれだけ先の時間点の個体数でも計算できます。しかし，これはあくまで Δt 間隔の時間点での値です。もしその間に個体数が変化していても，少なくとも私たちがこれまで本書で学んだ概念では，途中の値については求められません。

例えば図 3.2 を見てください。これは，ある Malthus 的な成長を表すグラフ

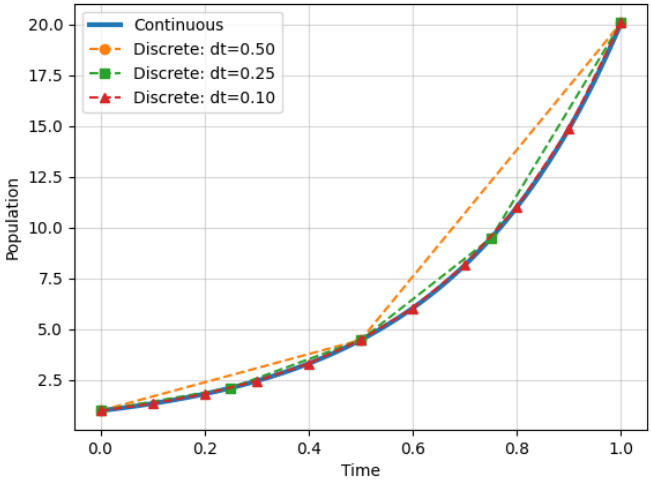


図 3.2　連続的曲線（青）と，その曲線上の Δt 間隔離れた二点の値から全体を再現したグラフを比較したもの。

を比較したものです。青い曲線は，変化が連続的に見えるほど Δt を細かくとった場合のグラフです。この連続的な曲線の初期値と別の点の二点の個体数から，式 (3.14) を通して求めた成長率 $r_{\Delta t}$ を用い，式 (3.12) を用いて先の値まで計算した結果が，マーカー付きの離散的なグラフです。これらは，連続的とみなせる実際の個体数の変化があり，Δt 間隔で得たデータから，それを再現した場合のグラフとみなせます。Δt が大きいと，一つの時間ステップの間に起こる変化を粗くしか捉えられないため，連続的な曲線との差が大きくなることが確認できるでしょう。単一の Δt に関する同様の比較図は，サンプルコード 3.3 により描画できます。dt が Δt に対応する変数なので，これを変えて，離散的な関数のグラフがどう変わるか確認してみてください。

```
import numpy as np
import matplotlib.pyplot as plt

t0 = 0.0
tf = 1.0
N0 = 1.0
r  = 3.0
```

```
time = np.linspace(0,tf,200)
N_cont =  N0 * np.exp( r * time )

dt = 0.1
t_points = np.arange(t0, tf+dt, dt)
r_dt = ( np.exp(r * dt) - 1.0) / dt
step = np.arange(len(t_points))
N_disc = N0 * (1.0 + r_dt * dt)**step

plt.plot( time, N_cont, label=f"Continuous", lw=3 )
plt.plot( t_points, N_disc, label=f"Discrete: dt={dt:.2f}",
  marker="o", ls="--", lw=1.5 )
```

```
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend()
plt.grid(True,alpha=0.5)
plt.show()
```

サンプルコード 3.2　単一の Δt について，図 3.2 と同様の比較図を生成するスクリプト。

この結果から，考えている時間スケールで見て，滑らかに変化している個体数の任意の時刻での値を計算しようと思ったら，Δt をどんどん小さくしていく必要があることがわかります。反対に，連続的な曲線が分かれば，そこから任意の離散的な時間点での値を読み取ることができます。そのため，実際の観測は離散的であっても，考えている時間スケールに対して変化が十分滑らかであれば，連続的な変化として近似することが有用な場合があります。

3.2.2 極限操作

有限の時間幅 Δt で計算した成長率 $r_{\Delta t}$ は，その時間幅にわたる平均的な 1 個体あたり成長率です。したがって，Δt を小さくしていくと，平均を取る時間幅が短くなり，その瞬間における成長率 r に近づいていきます。これを

$$\lim_{\Delta t \to 0} r_{\Delta t} = r \tag{3.15}$$

のように表します。ここで $\lim_{\Delta t \to 0}$ は，Δt を 0 に限りなく近づけていくことを表しています。より一般には，$\lim_{a \to b}$ は a を b に限りなく近づける極限操作

を表します。この瞬間的な成長率 r は，**内的増加率（intrinsic rate of increase）**や，単に成長率と呼ばれます。

(3.15) より，$\Delta t = t/n$ が小さいとき，すなわち分割数 n が十分大きいとき，離散的方程式 (3.12) は，$r_{\Delta t}$ を r で置き換えて

$$N_n = \left(1 + r\frac{t}{n}\right)^n N_0 \tag{3.16}$$

と近似できます。実は，上で描画した連続的な曲線は，この式ではなく，極限

$$\lim_{n\to\infty}\left(1 + r\frac{t}{n}\right)^n = e^{rt} \tag{3.17}$$

を用いて定義される関数

$$N(t) = N_0 e^{rt} \tag{3.18}$$

のグラフです。(3.18) に含まれる e は

$$e = 2.71828... \tag{3.19}$$

と続く定数で，**Euler の数（Euler's number）**や **Napier の定数（Napier's constant）**と呼ばれます。(3.17) 右辺は変数の刻みの細かさ n によらない量になっていることを確認してください。

(3.17) はより一般に，関数 e^x を定義する関係

数学公式 3.1 (e^x の定義)**.**

$$\lim_{n\to\infty}\left(1 + \frac{x}{n}\right)^n = e^x \tag{3.20}$$

となっています。ちなみに，e^x は，x の部分が長かったり複雑になったりすると表記や識別がしにくくなるため，$\exp(x)$ という表現もされます。上のコードの数値関数 `np.exp` はそれを反映したものです。

現実には，Δt を無限に小さくとることはできません。しかし，例えば人口を 1 日単位で記録し，数年に渡って描画すれば，実質的に連続的な曲線が得られます。このように，極限操作の数学的な定義はあくまで理想的なもので，実用上必要なことは，考えている時間スケールに対して滑らかな変動として記述できるほど十分細かい時間幅を用いて近似することです。実際，上で描画した連続的なグラフも，無限ではなく十分たくさんの点のつながりでしかありませんが，

それで十分です。ただし，以下で説明するように，変化を蓄積していくような計算の際には，時間刻みの影響はもっと大きくなります。

3.3 微分積分

3.3.1 平均変化率

引き続き，異なる時間での個体数の値から，その変化率を求めることを考えます。個体数の変化が Malthus 的なものに限らない場合に一般化して，改めてその基本的な計算方法を復習しましょう。

時間 Δt の間の個体数の変化は，式 (3.12) より

$$N_n - N_{n-1} = \Delta N_{n-1} \tag{3.21}$$

で与えられるのでした。横軸時間，縦軸個体数に取ったグラフでは，これはグラフの高さの変化に対応します。横幅の変化に対し，高さの変化が大きいほど，二点間を結ぶ線分の傾きが大きくなるため，(3.21) を横軸の変化幅に対応する Δt で割った

$$\frac{N_n - N_{n-1}}{\Delta t} = \frac{\Delta N_{n-1}}{\Delta t} \tag{3.22}$$

は，グラフの傾きに対応します。傾きが大きいということは，それだけ急に変化が起こっていることを意味しますから，数学的には (3.22) は N の変化率を表す量になります。(3.22) は，単に Δt の間の正味の変化率を測るもので，**平均変化率（average rate of change）**と呼ばれます。

(3.14) と比べるとわかるように，前節までで扱った離散的な成長率 $r_{\Delta t}$ は，平均変化率に関連する量だということが分かります。

3.3.2 微分

平均変化率 (3.22) は，その間にどんな変化の仕方をしているかなどを考慮するものではありません。しかし，時間刻み幅を限りなく小さくとれば，極限での値，すなわち瞬間的な変化率が得られます。この操作を，**微分（differentiation）**といい

$$\frac{dN(t)}{dt} \equiv \lim_{\Delta t \to 0} \frac{N(t+\Delta t) - N(t)}{\Delta t} \tag{3.23}$$

のように表現します。また，dN/dt を N の**導関数（derivative）**と呼びます。これにより，Malthus 的な成長の場合には，瞬間的な成長率 r が

$$\frac{1}{N(t)}\frac{dN(t)}{dt} = \frac{1}{N(t)}\lim_{\Delta t\to 0}\frac{N(t+\Delta t)-N(t)}{\Delta t} = r \quad (N(t)>0) \tag{3.24}$$

により得られます。このようにして瞬間的な成長率が分かれば，n 乗を含む離散的な式 (3.12) を使わなくても，関数 (3.18) を用いて，連続的な任意の時刻 t での個体数が求められます。

一般に，ある関数 $F(x)$ があったとき，導関数もまた別の関数であることから，それを $f(x)$ のように別の記号を使って表し

数学公式 3.2 (微分の定義)**.**

$$f(x) = \frac{dF(x)}{dx} = \lim_{\Delta x\to 0}\frac{F(x+\Delta x)-F(x)}{\Delta x} \tag{3.25}$$

と表現することも一般的です。このとき，F は f の**原始関数（primitive function）**と呼ばれます。

微分というと，高校などでは，つまらない練習問題をたくさん解かされることが一般的ですが，そんなものは必要ありません。大事なのはその操作の意味を理解し，定義 (3.23) あるいは (3.25) を書き下せることです。必要な公式や計算のスキルはやっていくうちに自然に身に付きます。

とりあえず，最も基本的な形の一例として，C を定数とする関数 $f(x) = x^2 + C$ の微分

$$\frac{d}{dx}(x^2 + C) = 2x \tag{3.26}$$

を確認しておきましょう。$f(x+\Delta x) = (x+\Delta x)^2 + C$ に注意すると，これは，定

義 (3.25) から

$$\begin{aligned}\frac{df}{dx} &= \lim_{\Delta x\to 0}\frac{f(x+\Delta x)-f(x)}{\Delta x}\\ &= \lim_{\Delta x\to 0}\frac{(x+\Delta x)^2+C-(x^2+C)}{\Delta x}\\ &= \lim_{\Delta x\to 0}\frac{x^2+2x\Delta x+(\Delta x)^2-x^2}{\Delta x}\\ &= \lim_{\Delta x\to 0}\frac{2x\Delta x+(\Delta x)^2}{\Delta x}\\ &= \lim_{\Delta x\to 0}2x+\Delta x\\ &=2x\end{aligned} \tag{3.27}$$

と計算できます。三行目で展開公式

数学公式 3.3 (二乗の展開公式)**.**

$$(a+b)^2 = a^2+2ab+b^2 \tag{3.28}$$

を使っています。丁寧に説明するために長々書いてますが，難しい計算をしているわけではないので心配しないでください。定数 C は x をずらしても変化しないため，どんな値でも差を取ったときに消えます。グラフでいうと，微分は傾きを評価する操作なので，グラフを縦軸方向にずらしても影響しないことを反映しています（図 3.3）。

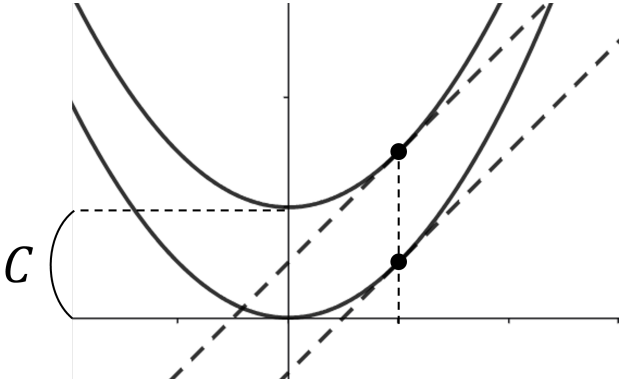


図 3.3　ある関数と，それを y 方向に定数 C だけずらして得られる関数（それぞれ実線）の，接線（点線）の傾きを，同じ x 軸上で比較した図です。

この結果より，関数 x^2+C の傾きは $2x$ という関数で計算できることがわかります。$x=1$ での傾きを知りたければ，それを代入して 2 という値を得ます。よって，もしもその点での変化率のまま変化を続けたらどうなるか，ということを知りたければ，その点を通る傾き 2 の直線 $2x+b$ を引けば図示できます（定数 b で，その点を通るように調整します）。このような線を，元の関数の**接線（tangent line）**といいます。

今行った計算や接線の説明を視覚的に確認したい方は，次のスクリプトを実行してみてください。dx を小さくしていくと，関数 x^2 の接線が，傾きが 2 であり，接点 $(1,1)$ を通る直線 $2x-1$ と一致していくことが確認できます。

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):

    func = x**2

    return func

x = np.linspace(0,2,200)

# 微分を計算する点
x0 = 1

# x0からのずれ
dx = 0.5

# 元の点での関数の値
f0 = f(x0)
# dxずらした点での値
f1 = f(x0+dx)

# dxにわたる平均変化率
dfdx = ( f1 - f0 ) / dx

# 平均変化率を傾きとする直線
tangent = f0 + dfdx * ( x - x0 )

# 作図
plt.plot( x, f(x),
        label=r"$y=x^2$",  lw = 2)
plt.plot( x, 2*x-1,
```

```
        label=r"$y=2x-1$", lw = 1, ls="--")
plt.plot( x, tangent,
        label="numerical tangent", lw=1)

# 接点
plt.scatter(x0,f0,color="black")
plt.legend()
plt.grid(True,alpha=0.5)
plt.show()
```

上では x^2 という関数の微分を計算しましたが，$x>0$ のとき一般にどんな実数 a に対しても

数学公式 3.4 (べき関数の微分公式)**.**

$$\frac{d}{dx}x^a = ax^{a-1} \tag{3.29}$$

という公式が成り立ちます。a が負の実数でも同じように成り立つことの注意してください。また，a が正の整数なら x が負の実数でも成り立ちます。

3.3.3 積分

最初に§ 3.1.1 で確認したように，離散的なケースでは，ある時刻の個体数に，次の時刻までの変化分を足し上げる操作を繰り返して未来の時刻の個体数を決定しました。これは，式 (3.4) で導入した和の操作と平均変化率を用いて

$$\begin{aligned} N_n &= N_0 + \sum_{k=0}^{n-1} \Delta N_k \\ &= N_0 + \sum_{k=0}^{n-1} \frac{\Delta N_k}{\Delta t}\Delta t \end{aligned} \tag{3.30}$$

と書き表せます。ここで，$\Delta t = (t_n - t_0)/n$ です。つまり，変化率という局所的な量から各ステップでの変化量を構成して積み上げ，全体像を復元していく操作になっています。図 3.4 は，$n=4$ とした場合の (3.30) の概念図です。

連続的なケースでは，この変化分を一つ一つ数えあげることができなくなるので，総和操作がそのままでは使えません。この場合には，微分の極限操作 (3.23) と同様に，足し上げる変化分も限りなく細かくとります。その操作は，新たな記

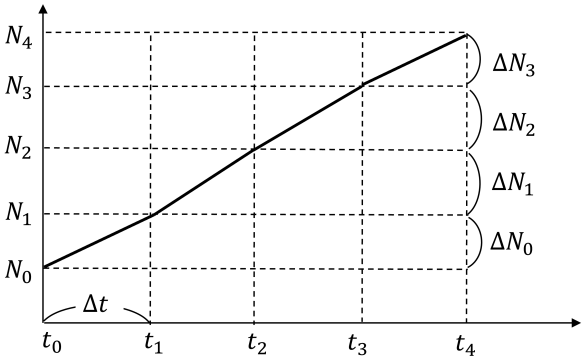


図 3.4　例として $n=4$ とした場合の離散和 (3.30) のグラフの例。

法を導入して

$$\int_{t_0}^{t_f} \frac{dN(t)}{dt} dt \equiv \lim_{\Delta t \to 0} \sum_{k=0}^{n-1} \frac{\Delta N_k}{\Delta t} \Delta t \tag{3.31}$$

と表されます。この左辺の操作は**積分（integral）**と呼ばれ，総和記号 $\sum$ の連続的な対応物である $\int$ は**積分記号（integral symbol）**と呼ばれます。これを利用すると $t_n = t_f$ として，(3.30) の連続的なバージョン

$$\int_{t_0}^{t_f} \frac{dN(t)}{dt} dt = N(t_f) - N(t_0) \tag{3.32}$$

が得られます。

一般に (3.25) のところで言及した関数 f とその原始関数 F に対し

数学公式 3.5 (微積分学の基本定理)**.**

$$\int_{x_0}^{x_f} f(x)dx = F(x_f) - F(x_0) \tag{3.33}$$

が成り立ちます。右辺の $F(x_f)$ と $F(x_0)$ は，引数に特定の値を代入したものですから，関数ではなく具体的な値です。(3.33) の関係は，微分と積分は互いに反対の操作であることを示しています。より純粋にこの関係を示すものとして，積分の範囲（$\int_a^b$ における a と b）を指定しない表現

数学公式 3.6 (不定積分)**.**

$$\int f(x)dx = F(x) + C \tag{3.34}$$

が利用されます。これは，今述べた理由から，**逆微分（antiderivative）**と呼ばれます。(3.33) が，積分範囲を指定しているため，関数ではなく具体的な値を与えることから，**定積分（definite integral）**と呼ばれるのに対し，逆微分 (3.34) は範囲を定めず，値ではなく関数の間の関係を示すものですから，**不定積分（indefinite integral）**とも呼ばれます。C は**積分定数（integral constant）**と呼ばれるもので，微分の例 (3.27) のところで説明した通り，微分するとゼロになるため，逆微分の操作からは定まらない部分です。個体群の解析では，初期の個体数などに対応し，特定の条件を定めることで具体的な値が定められます。

積分も，微分と同様，具体的な計算ができるかどうかを心配する必要はありません。大事なのはその操作の意味を理解することです。そして，その本質はこれまでずっとやってきたように，変化分を積み上げるという初等的な操作に過ぎないため，必要以上に難しく考える必要もありません。

ちなみに，$\int f(x)dx$ は $\int dx f(x)$ と書かれたりします。特に規則はなく，見やすさや癖など，その時の感じで使い分けられるので深く考えないようにしましょう。

3.4 連続的 Malthus モデル

3.4.1 連続的方程式

前節で導入した極限操作を適用すると，離散的な方程式 (3.12) から

$$\frac{dN(t)}{dt} = rN(t) \tag{3.35}$$

が得られます。このように，微分を含む方程式を，**微分方程式（differential equation）**といいます。そして，微分方程式 (3.35) の解は (3.12) と (3.18) より

$$N(t) = N_0 e^{rt} \tag{3.36}$$

であることがわかります。$e^0 = 1$ なので，$t = 0$ で初期値 N_0 となるようになっています。また，このモデルの中では，成長率 r は **Malthus 係数（Malthusian**

parameter）とも呼ばれます。

微分方程式 (3.35) とその解 (3.36) の関係は，指数関数の微分の一般的公式

数学公式 3.7 (指数関数 e^{ax} の微分公式)**.**

$$\frac{d}{dx}e^{ax} = ae^{ax} \tag{3.37}$$

として成り立つものであることを覚えおいてください。(3.36) には定数 N_0 が含まれますが，定数は文字通り一定の数なので，変化率を考える微分には影響しません。

3.4.2　指数関数的増殖

(3.36) のように，x^a という形の関数における a を，**指数（exponent）**といい，変数が指数に現れる関数を指数関数といいます。そして，指数関数によって表される増加あるいは減少する様子を，指数関数的増加や減少と言い表します。連続的な Malthus モデルが示すことは，制限がなければ生物は指数関数的に増殖をするということです。これが，生物の増殖に関する恐ろしい基本法則なのです。

3.4.3　数値積分

時間刻み Δt を無限小に取り，個体数変化が連続的であるとみなすことで，Malthus 的な成長は微分方程式 (3.35) により記述できることが分かりました。また，微分と積分の関係 (3.33) より，微分方程式 (3.35) は

$$N(t) = N(t_0) + \int_{t_0}^{t} rN(t')dt' \tag{3.38}$$

と積分の形に書き換えられます[*3]。ここで $N_0 = N(0)$ としました。

時間変化を追跡する数値計算においては，初期値 N_0 から，この積分に対応する式を計算して，未来の個体数を計算していきます。しかし，数値計算では，Δt

[*3] ここでは定数である t に対して，積分の中の時間は変化する別の量なので，区別するためにプライム「′」をつけて t' としています。これもよくある慣例です。t' の代わりに別の変数，例えば s などを使ってもいいですし，混同の恐れがない場合はそのまま同じ t を使ってもよいです。

を無限に小さくとることはできず，積分はなんらかの離散的な和で近似することになります。最も単純な方法は，離散的方程式の形

$$N_n = N_0 + \sum_{k=0}^{n-1} r N_k \Delta t \tag{3.39}$$

で近似することです（これは，Euler 法と呼ばれる方法に対応します。詳細は Appendix B を参照してください）。つまり，今の場合，与えられた「瞬間的な成長率 r」を用いて，各ステップで離散的な式

$$N_{n+1} = (1 + r\Delta t) N_n \tag{3.40}$$

を求めていくことになります。しかし，先に説明したように，本来なら離散的な成長率 $r_{\Delta t}$ と瞬間的な成長率 r は Δt が無限小の極限でしか一致しないので，その分の誤差が生じます。Δt の取り方により結果がどう変わるかを見るため，とりあえず，サンプルコード 3.3 を実行してみてください。結果は図 3.5 のようになります。Δt に対応する dt の値を変えて，Δt が十分小さくなってくると，連続的な関数に一致していくことを確認してみてください。

```
import numpy as np
import matplotlib.pyplot as plt

t0 = 0.0
tf = 1.0
N0 = 1.0
r  = 3.0

time = np.linspace(0,tf,200)
dt = 0.1
t_points = np.arange(t0, tf+dt, dt)
step = np.arange(len(t_points))

N_cont = N0 * np.exp( r * time )
N_disc = N0 * (1.0 + r * dt)**step

plt.plot( time, N_cont, label=f"Continuous", lw=3 )
plt.plot( t_points, N_disc, label=f"Discrete: dt={dt:.2f}",
  marker="o", ls="--", lw=1.5 )
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend()
plt.grid(True, alpha=0.5)
```

```
plt.show()
```

サンプルコード 3.3　関数 $N_0 e^{rt}$ と，同じ N_0 と r を用いた (3.40) を比較するためのスクリプト。

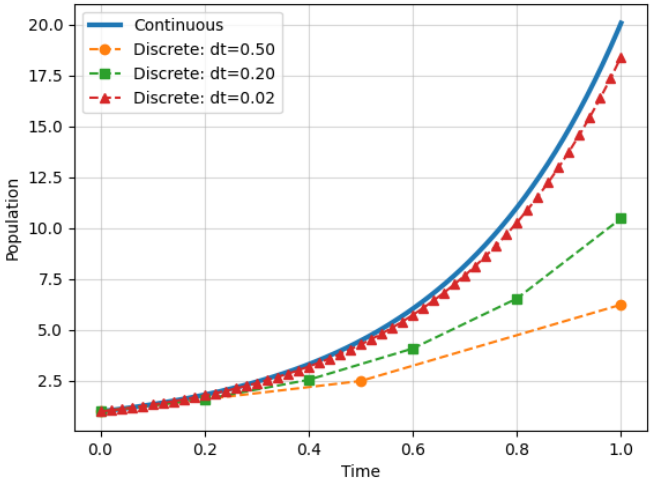


図 3.5　関数 $N_0 e^{rt}$（青線）と，同じ N_0 と r を用いた (3.40) の比較。`dt` を小さくするほど，離散的関数のグラフが連続的関数のグラフに近づいていくことが確認できます。

式 (3.40) は，ある時点で N_n であった個体数が，時間 Δt の間，一定の変化率 rN_n で増加あるいは減少して，N_{n+1} になることを表しています。対して，時間幅 Δt を半分にして，時刻 $t+\Delta t/2$ での個体数

$$N_{n+1/2} = (1 + r\Delta t/2)N_n \tag{3.41}$$

を計算し，そこから N_{n+1} での値を求めるという二段階の操作に分けると

$$N_{n+1} = (1 + r\Delta t/2)N_{n+1/2} = (1 + r\Delta t/2)^2 N_n \tag{3.42}$$

という結果になります。公式 (3.28) を使って二乗の項を展開してみると

$$N_{n+1} = [1 + r\Delta t + (r\Delta t/2)^2]N_n \tag{3.43}$$

となります。つまり，かっこの中の最後の項 $(r\Delta t/2)^2 N_n$ の分だけ，(3.40) と異なる結果になっています。

例えば $r\Delta t$ の値が 0.01 とかであったら，$(r\Delta t)^2 = (0.01)^2 = 0.0001$ ととても小さい量になるので，その差は無視できます。しかし，$r\Delta t$ があまり小さくない場合，その差は無視できないものになります。また，これは 1 ステップの間に生じる差ですが，これを多くのステップ繰り返すと，この差はどんどん蓄積していきます。図 3.5 でも，時間が進むほど（グラフの右に行くほど），連続的なグラフと離散的なグラフの差が開いていくことが確認できるでしょう。

このように，微分方程式を数値的に計算するとき，Δt の取り方が問題になってきます。Δt を可能な限り小さくとればよいと考えるかもしれませんが，そうもいきません。Δt を小さくとるということは，1 ステップで計算できる時間がとても短くなるということです。これは，必要な計算が完了するまでにかかる時間が長くなることを意味しています。ある程度の処理が必要なプログラムになってくると，これではまともな時間内に計算が終わらなくなってしまいます。

実際には，今の場合は解析的な解 $N_0 e^{rt}$ が分かっているので，離散的な和を用いなくても，任意の時刻での値が分かります。しかし，より複雑な問題の場合はそのような解が得られないために数値計算が必要になります。その際，実際に今述べたような時間刻みの問題が生じます。そのため，Δt をできるだけ大きく保ったまま，精度よく計算するテクニックが様々開発されてきました。その際，微分方程式の対応物は，必ずしも直接的な離散和の形にはなりません。詳細は Appendix B で説明していますが，以降の章でも，必要に応じて簡単に説明していきます。

3.5 対数

3.5.1 成長率の違いによる比較

今度は，成長率を変えると，個体数の挙動がどう変化するか比較してみましょう。サンプルコード 3.4 を使って描画した後，好きに成長率を変化させて得られる結果がどう変わるか確認してみてください。特に，気持ち大きめの値を一度試してみてください。

```
import numpy as np
```

```
import matplotlib.pyplot as plt

r_list = [0.5, 1, 3]
time = np.linspace(0,1,100)
N0   = 1

for r in r_list:
    N = N0 * np.exp( r * time )
    plt.plot( time, N, label=f"r={r:.1f}" )

plt.xlabel("Time")
plt.ylabel("Population")
plt.legend()
plt.grid(True,alpha=0.5)
plt.show()
```

サンプルコード 3.4　成長率の違いによる比較。

これを実行すると，図 3.6 のような図が生成されるはずです。

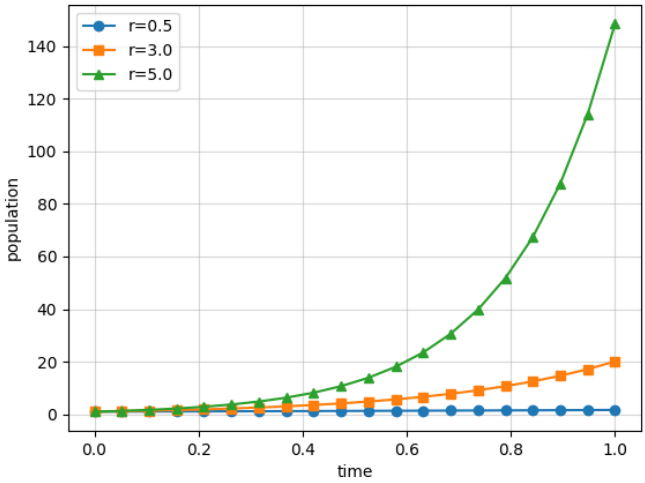


図 3.6　サンプルコード 3.4 を基にしたコードの実行例。

3.5.2 対数スケール

上の描画では，r を変化させることで，個体数の成長の勢いが変化することが確認できました。しかし，今考えている制限のない成長では，r がわずかに大きいだけで，時間が経つとともに個体数が爆発的に大きくなってしまい，r による違いが直感的にわかりづらくなってしまいます。そこで役立つ数学的道具が，**対数（logarithm）**です。まず，何も考えず上のコードの描画コマンド部分で `plt.yscale('log')` を追加してプログラムを実行してみてください。図 3.7 のように直線的なグラフに変化するはずです。

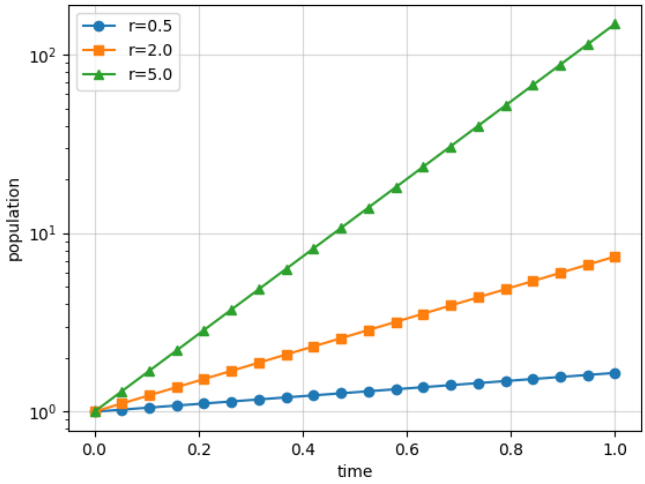


図 3.7　サンプルコード 3.4 を基に，`plt.yscale('log')` を加えたコードの実行例。

`plt.yscale('log')` を追加することで変わったのは，描画される y 軸の目盛りです。元々，値に対して等間隔で描画されていたものが，$10^0 = 1$，$10^1 = 10$，$10^2 = 100, \ldots$ のように，10 の何乗かごと，つまり桁に対して等間隔になる表示に変わっています。

3.5.3　対数とは

対数 $\log_a b$ は，「数 b が a の何乗であるか」を与える関数です。連続的な Malthus モデルの解 (3.35) は（$N_0 = 1$ とすると）$N(t) = e^{rt}$ なので，「$N(t)$ は e の rt 乗」です。よって

$$\log_e N(t) = rt \tag{3.44}$$

という関係が成り立ちます。このことから，対数は Malthus モデルにおいて，個体数から成長率を抽出する道具とみなしてもいいでしょう。$\log_a b$ に対し，a は対数の**底（base）**と呼ばれますが，このように底が e の場合が特に大事な役割を果たすことが多いため，$\log_e x$ を**自然対数（natural logarithm）**と呼び，$\ln x$ と記すことも一般的です。

他方，上の図で用いた対数スケールへの変化は，底を 10 にした場合に対応しています。$a = 10$ の場合には，10 を何乗すれば b になるかを与えますので，こちらもまた直感的にわかりやすく，よく用いられます。そのため，$\log_{10}$ は**常用対数（common logarithm）**と呼ばれ，単に log と表されることも一般的ですが，同じ表記で自然対数を表すこともあるので注意が必要です。以下，あえて明示する場合を除いて，自然対数については ln を用いることにします。

3.5.4　対数スケールでグラフが直線的になる理由

それでは，対数スケールを用いることで個体数のグラフが直線的になった理由を調べましょう。改めて，$x = 10^y$ ならば，$y = \log_{10} x$ となり，x のおおよその桁を表します。よって，r を定数として，$y = rt$ という形なら

$$\frac{\log_{10} x}{t} = \frac{y}{t} = r \tag{3.45}$$

が得られます。つまり，$x(t)$ の桁と時間の比は一定の値 r になります。縦軸を対数スケールで描画した $x(t)$ のグラフは，$x(t)$ の桁の変化の速さを表すグラフでしたから，これが一定であるということは，直線的なグラフであるということがわかります。

連続的な Malthus モデルの解 (3.35) も，一定の r による e^{rt} という形ですが，10^{rt} の形ではないので，少し話が違うように思えるかもしれません。しかし，本

質的には 10 の累乗の場合と変わらないことが示せます。

対数には，適当な正の数 $a, b, c,$（$a \neq 1$, $c \neq 1$）に対し

数学公式 3.8 (底の変換公式)**.**

$$\log_a b = \frac{\log_c b}{\log_c a} \tag{3.46}$$

という変換が成り立つので，もし $x(t) = e^{rt}$ ならば

$$\log_{10} x = \frac{\log_e x}{\log_e 10} = \frac{rt}{\log_e 10} \tag{3.47}$$

となります。$\log_e 10$ は単なる定数なので，グラフの傾きは変えても，その形状までは変化させません。よって縦軸を対数スケールに取った $x(t) = e^{rt}$ のグラフは，底の選び方に依らず直線的になるのです。そういうわけで，指数関数的な時間依存性を持つ量の振る舞いを調べるには，元の（線形な）スケールよりも，対数スケールが適していることが分かります。

3.5.5 対数の微分

(3.44) より，連続的 Malthus モデルの解 $N(t)$ の自然対数を取ると rt が得られるのでした。この式の右辺を時間で微分すると

$$\frac{d}{dt}(rt) = \lim_{\Delta t \to 0} \frac{r(t+\Delta t) - rt}{\Delta t} = r \lim_{\Delta t \to 0} \frac{\Delta t}{\Delta t} = r \tag{3.48}$$

となります。このように，定数倍は微分操作の外に出せます。よって

$$\frac{d}{dt} \ln N(t) = r \tag{3.49}$$

が成り立ちます。

他方，N はいつもゼロではないとして，(3.35) を $N(t)$ で割ると

$$\frac{1}{N}\frac{dN}{dt} = r \tag{3.50}$$

となります。これらを比べると

$$\frac{d}{dt} \ln N(t) = \frac{1}{N}\frac{dN}{dt} \quad (N(t) > 0) \tag{3.51}$$

という関係が示唆されます。実際にこれは，自然対数関数の微分についての一般的に正しい式になります：

数学公式 3.9 (自然対数の連鎖微分公式)**.**

$$\frac{d}{dx}\ln f(x) = \frac{1}{f}\frac{df}{dx} \quad (f(x) > 0) \tag{3.52}$$

3.6　この節のまとめ

生物は，制限がなければ，その数に比例して増殖するという恐ろしい性質を持っています。本章では，このように制限がない場合の増殖を数学的に記述し，それを計算するプログラムの書き方を学びました。

はじめに § 3.1 で，個体数の変化を数学的に記述する最も基本的な方法を学ぶため，離散的な系を考えました。そこでは，個体数と成長率が与えられた場合，そこから別の時刻の個体数を計算する方法を学びました。次に，§ 3.2 で，異なる時刻での個体数から，成長率を求めることを考えました。そこで，時間刻み幅を細かくすることで，連続的な変化をよりよく近似できることを見ました。

§ 3.3 では，時間刻み幅を限りなく小さくとる操作，すなわち微分を学びました。与えられた個体数の変化から変化率を求める操作を連続化したものが微分であり，変化率から変化分を積み上げて個体数の変化を求める操作を連続化したものが積分です。このように，微分と積分は互いに逆の操作として対応していることを学びました。

§ 3.4 で，微分を利用して連続化したモデルについて学びました。しかし，連続的なモデルを導入したからといって，コンピュータ上では連続的な時間をそのまま扱えるわけではありません。微分方程式を数値計算する際には，結局のところ時間を離散化して計算することになります。そこで，離散化による誤差と計算コストとの兼ね合いが生じることに触れました。続いて，指数関数的増殖では，成長率のわずかな違いによって，時間とともに個体数の差が急速に大きくなることを確認しました。それに対し，対数関数を利用することで，指数関数的な変化を線形（直線的）な変化として表現できることを学びました。

これらが，制限のない場合の生物の個体数の変化を記述するのに必要な基礎知識です。現実には，個体数の変化を制限する様々な要因があります。今後はそれらを反映したより現実的なモデルを考えていきます。しかし，基本がわかれば後はそれを少しずついじっていくだけなので，本章の内容が理解できれば

この先の内容を理解するのに本質的な困難はないでしょう。

追加の図書案内

物理学や生物学などで用いる数学は，ほとんどの場合，道具としての数学で，数学者の使う難解で几帳面な数学とは別物です。あくまで道具ですので，道具の使い方の練習に時間を費やすよりは，使いながら身に着けるという考え方の方がよいのではないかと思います。例えば

- 和達 三樹. (1983). 物理のための数学 (Vol. 10). 岩波書店

のように，必要なツールをざっとまとめたテキストは数多くあります。古本でも最低価格やそれに近いものが見つかるでしょうし，ウェブページや電子ファイルでよければネット上でも様々な公開されています。ですので，初めにしっかり数学を身に着けてから，と考えるよりは，具体的な問題に取り組みながら，このような資料を手元に置いて必要に応じて参照しながら学ぶといいでしょう。

生態学のための数学を解説したもので，易しい日本語のテキストとしては

- 西村 欣也. (2012). 生態学のための数理的方法: 考えながら学ぶ個体群生態学. 文一総合出版

があります。このテキストはとにかく易しく，生態学を通して必要な基礎数学を身に着ける構成としては，本書と共通するものです。

本書の基本的考えの一つに知識のオープン化があり，これらの市販の書籍に頼らずとも，誰でも必要な知識を得られるようにすることが目的です。しかし，どうしても一冊のテキストでは，同じ内容を扱うにしても視点が狭まってしまいますし，電子ファイルでは紙の本に敵わない特質もあります。余裕があり，より理解を深めたい人は，これらの参考文献も利用するといいでしょう。

References

Darwin, C. (2020). *The autobiography of charles darwin*. Open Road Media.

Singleton, K. (2025). ビーガンのための進化生物学入門:「利害」についての正しい理解のために [https://drive.google.com/file/d/1b4cJGaBXKRc7oQuwgTRBH3Rg6CHH3Cg4/view?usp=sharing/].

第 4 章

密度に依存する成長

> “自然界では，負の価値は幸福を凌駕している。これは，資源が希薄な環境における進化的過程の不可避な帰結である。その結果，自然界では苦しみと早死にが当たり前のこととなっている。生き延びるものより，生まれて間もなく苦痛の中死んでいくだけの個体の方が圧倒的に多いのである。”
>
> — Oscar Horta,
> *The Problem of Evil in Nature: Evolutionary Bases of the Prevalence of Disvalue, 2015*

4.1 ロジスティック方程式

Malthus モデルは，個体数は常にその数に比例して際限なく増殖するというモデルでした。しかし，現実では成長率は個体数に依存するし，ある環境中に生息できる個体数には限りもあります。そこで，本節では，指数関数的成長の傾向に加え，こうした効果も反映したモデルを考えます。

4.1.1 ロジスティック方程式の導出

個体数の成長が抑制される自然な要因は，その環境中で得られる食べ物や利用できる生活スペースなどのリソースに限りがあることです。言い換えると，安定に生息できる個体数には，環境ごとに大まかな上限値があるということです。

この上限値を，**環境収容力（carrying capacity）**と呼び，一般に K という記号で表します。

制限がなければ，個体数が増加する場合 $r>0$ を考えましょう。これに環境収容力による制限が加わることで，個体数の増加は，個体数が K に近づくほど減速し，K を越えたらそれを引き戻すように減少に転じるはずです。これを数式で最も簡潔に表現するには，成長率に $(1-N/K)$ という因子をつけて

$$r\left(1-\frac{N}{K}\right)$$

とすることです。そうすれば，個体数 N が環境収容力 K よりもずっと小さくて余裕があるときは

$$\frac{N}{K} \ll 1 \tag{4.1}$$

ということなので，N/K は無視出来て

$$r\left(1-\frac{N}{K}\right) \simeq r \tag{4.2}$$

と元の指数関数的成長と変わらなくなります[*1]。そして，個体数が環境収容力を超えると $N/K>1$ より

$$1-\frac{N}{K}<0 \tag{4.3}$$

なので，個体数は時間と共に減少することになります。

こうして，Malthus モデルから，成長率を修正して得られる

$$\frac{dN}{dt}=rN\left(1-\frac{N}{K}\right) \tag{4.4}$$

は**ロジスティック方程式（logistic equation）**と呼ばれます。

4.1.2　ロジスティック増殖の振る舞い

ロジスティック方程式は，離散化すると

$$N_n=N_{n-1}+rN_{n-1}\left(1-\frac{N_{n-1}}{K}\right)\Delta t \tag{4.5}$$

[*1] $a \ll b$ は，$a<b$ のような純粋な大小関係とは違い，a が b よりも「十分」小さいことを意味します。また，$a \simeq b$ は近似的に等しいことを意味します。

です。これを，連続的な式を数値的に表現するための離散化とみなして，その挙動を調べてみましょう。

みなさんはもう，離散的な Malthus 方程式の描画に使ったサンプルコード 3.1 を基にして，これを描画するコードを作成できるはずですので，練習問題としてやってみてください（回答例は補足資料のサンプルコードを参考にしてください）。必要なのは，新たなパラメータとして環境収容力を加えることと，ループ内の方程式を (4.4) を離散化した (4.5) に変更することです。実際にそうしてみると，図 4.1 のようになるはずです。図では，$r = 1$ および $K = 100$ とし，それぞれ $N_0 = 10$ と $N_0 = 150$ とした場合のグラフを描画してあります。

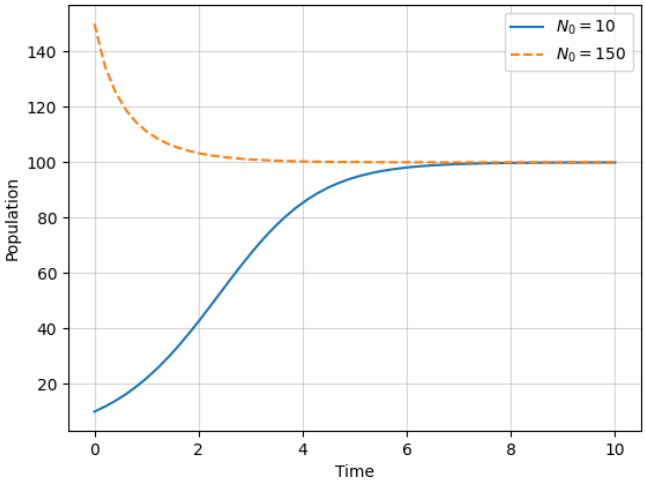


図 4.1　ロジスティック方程式 (4.4) に従う個体数のグラフ。$r = 1$ および $K = 100$ とし，青線は $N_0 = 10$，オレンジ線は $N_0 = 150$ とした場合のグラフです。

式 (4.4) やそれを描画した図 4.1 からわかるように，ロジスティクス成長する個体数は，環境収容力と等しくなると，そこから変化しなくなります。このような点のことを**平衡点（equilibrium point）**といいます。時間的に変化しないということなので，平衡点は，時間微分である式 (4.4) の左辺をゼロと置けば得ら

れます。

$$0 = rN\left(1 - \frac{N}{K}\right) \tag{4.6}$$

この式を成り立たせるのは，$N = K$ か $N = 0$ です。つまり，平衡点は個体数が環境収容力と等しい場合と，もう一つ，絶滅した場合に対応するということです。

4.1.3　ロジスティック方程式の解

ロジスティック方程式の解析解は A.1.4 で導出していますが，ここでは発見的（heuristic）に解の形を探ってみましょう。まず，$N/K \ll 1$ のときは Malthus 的な成長をするということなので，その極限で解も Malthus 方程式の解 $N_0 e^{rt}$ で近似できるはずです。他方で，時間 t が大きくなると，Malthus 的な解は爆発的に増大してしまいます。それを抑える効果を含んでいるのがロジスティック方程式なので，その解にもそれが反映されていなくてはなりません。$t \to \infty$ で e^{rt} の発散を抑える一つの形は，分母にも同じ形が含まれた

$$N = \frac{N_0 e^{rt}}{Ae^{rt} + B} \tag{4.7}$$

というものです。A と B は適当な定数です。$B = 0$ だと e^{rt} が分子と分母で打ち消しあってしまうことに注意しましょう。A と B の具体的な形を決めれば，解が定まります。

定数を決める条件の一つは，$t = 0$ で $N = N_0$ という初期条件を満たすことです。これから，$A + B = 1$ という条件が得られます。つまり

$$N = \frac{N_0 e^{rt}}{Ae^{rt} + (1 - A)} \tag{4.8}$$

と絞り込めます。さらに，解は重要なパラメータである K を含むべきです。そして，もし初期値が $N_0 = K$ だったならば，この解は K のまま時間変化してはいけませんから，その場合は e^{rt} の因子が相殺されなくてはいけません。そのためには，$(1 - A) = 0$ となり，分子と分母で e^{rt} の因子が打ち消しあう必要があります。これらの要件は，$A = N_0/K$ とすれば満たされることが確認できます。こうして

$$N(t) = \frac{KN_0 e^{rt}}{\left[K - N_0\left(1 - e^{rt}\right)\right]} \tag{4.9}$$

が得られます。これは実際に正しい解になっています。

今の「導出」は，すでに正しい解を知ったうえで，もっともらしい条件からそれを再構成したものですが，実際にこのような発見的な推論は，答えがわからない問題に取り組むうえで大事なアプローチになります。そして，そこがセンスの試される最も面白い部分の一つでもあります。

4.1.4 エサの量と環境収容力

環境収容力といっても漠然としているので，もう少し具体的な例を考えてみましょう。ある環境下で，他の有機物を食べて生きる生物種（従属栄養生物）が安定に存在するためにまず重要なのが，エサです。そこで，簡単なモデルでは，エサの量以外の要因はすべて無視して，環境収容力をエサの量 S に比例する形 $K = cS$ に仮定することがあります。

比例係数 c は，個体一体が単位時間に生存のために食べる必要なエサの量の逆数と解釈できます。例えば，S が 1kg=1000g で，一体が単位時間に 10g のエサを食べるとしましょう。すると，これだけのエサでまかなえる個体数は，エサの総量を一体あたりが食べる量で割った $1000\,\mathrm{g}/10\,\mathrm{g} = 100$ となります。したがって，一体が単位時間に食べる量を $\mu = 1/c$ とすれば，環境収容力は $K = S/\mu$ となり，ロジスティック方程式は

$$\frac{dN}{dt} = rN\left(1 - \frac{\mu N}{S}\right) \tag{4.10}$$

と書けます。この考え方は，§ 6.5 で，生態系における重要な効果をモデルする際に利用します。

4.2 Allee 効果

前節では，密度が大きい場合に成長を抑える効果を反映した方程式として，ロジスティック方程式を導入しました。しかし，生物個体のレベルで見れば，密度が高いことによる恩恵もあります。例えば密度が高ければ交尾相手が見つかりやすくなりますし，集団で協力してエサを獲得したり，天敵から身を守ったりする種もいます。このような効果は，**Allee 効果（Allee effect）**と呼ばれます (**rockwood1999allee**; 巌佐, 1998)。反対に，密度が小さくなりすぎると，交尾相

手が見つからなくなったり，生存のための強力が難しくなったりして，絶滅のリスクが高まります。この効果も，ロジスティック方程式を導入したときとよく似た考え方で取り入れられます。

密度（個体数）が小さすぎて集団効果が働かなくなる個体数の閾値（Allee 閾値）を a とします。そして，成長率に

$$(N-a)$$

という因子を含めれば，個体数がこの閾値より小さい場合，$N-a<0$ となって個体数の減少をもたらします。よって，環境収容力と，Allee 閾値を両方取り入れた最も単純な式として

$$\frac{dN}{dt} = rN(N-a)\left(1-\frac{N}{K}\right) \tag{4.11}$$

が得られます。ただこのままだと，右辺に個体数の二乗の単位（$\sim rN^2$）を持つ量が現れます。左辺が「個体数/時間」の単位なので，このままでは，成長率 r の単位が「1/時間」から「1/（時間 · 個体数）」に変わってしまいます。そこで

$$\frac{dN}{dt} = rN\left(\frac{N}{a}-1\right)\left(1-\frac{N}{K}\right) \tag{4.12}$$

あるいは

$$\frac{dN}{dt} = rN\left(\frac{N-a}{K}\right)\left(1-\frac{N}{K}\right) \tag{4.13}$$

のような形で導入することも一般的です。

式 (4.13) に従う個体数を描画した例が，図 4.2 になります。前の図 4.1 と同じように，オレンジ線が初期値 N_0 が環境収容力（この例では $K=50$）よりも大きい場合を表しています。青線が $N_0=20$ が K より少ないが，Allee 閾値 $a=10$ よりは大きい場合を合わしています。そして，緑線が $N_0=9<a$ とした場合のグラフです。オレンジ線と青線は通常のロジスティック方程式に従う場合と同様の変化の仕方を示していますが，緑線は 0 に向かって減少しています。これは，新たな領域に侵入（invasion）してくる個体群の密度が低すぎて，定着できない場合などに対応します。

元のロジスティック方程式では，$N=0$ と $N=K$ が $dN/dt=0$ となる平衡点でた。Allee 閾値を反映した式では，$N=a$ も平衡点になります。実際に上のシミュレーションで $N_0=a$ とすると，個体数は初期値から時間変化しません。し

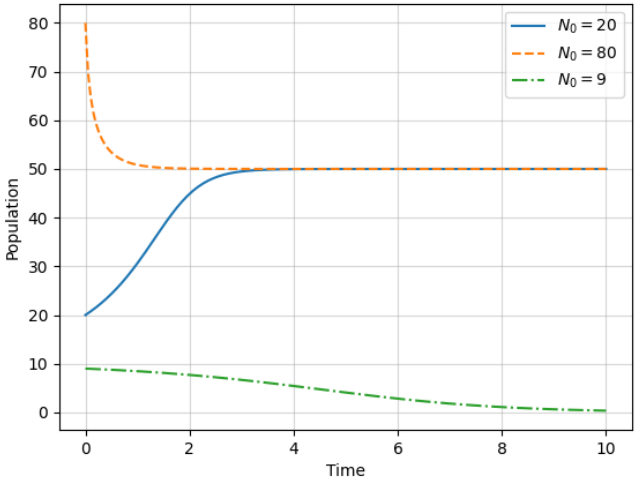


図 4.2　式 (4.13) に従う個体数のグラフ。$r = 3$ および $K = 50$，$a = 10$ とし，青線は $N_0 = 20$，オレンジ線は $N_0 = 80$，緑線は $N_0 = 9$ とした場合のグラフです。

かし，平衡点の周りでのゆらぎが加わるより現実的なモデルでは，そのような振る舞いは実現しません。詳細については，§ 6.3 で説明します。

4.3 基本原理とモデルの役割

第 3 章で，制限なく指数関数的成長をする Malthus モデルを学び，本章では環境収容力による制限を加えたロジスティックモデルを学びました。ロジスティックモデルからすれば，Malthus モデルは現実の振る舞いを記述するにはあまりにも単純ですし，ロジスティックモデルも現実の個体群の振る舞いを詳細に記述できるほど精巧ではありません。もちろん，現実の個体群がある条件のもとでこれらのモデルに近い振る舞いを示すこともあります。しかし，これらのモデルを学ぶ理由は，現実の個体数を正確に予測することだけではありません。

基本的な数理モデルには，複雑な現実の現象の中から，その背後にある基本的

な性質を切り出して考えられるという重要な役割があります。現実には様々な要素が複雑に絡み合いますが，これらの要因を一つずつ取り除き，最後に最も基本的な性質だけを残したとき，その系がどのように振る舞うのかを考えることで，個々の要因がもたらす効果をより明確に理解することができます。そのため，より複雑なモデルを構築しても，最も単純な条件を仮定したとき，Malthus モデルのような理想的モデルに帰着することが重要です。こうした理想的なモデルは，より複雑なモデルの妥当性を検証する基準になるのです。

一方で，どれだけ精巧なモデルを作っても，それは世界そのものではなく，世界のモデルあり，モデルには必ず適用範囲の限界があるということを忘れてはいけません。そのため，理論モデルを扱うにあたって，次の二つの側面を忘れないことが重要です：

- どんなモデルでも，適用範囲は限られていて，複雑な現実の現象を記述できる状況や精度は限定される。
- だからといって，理想的なモデルが示す基本性質が間違いだということにはならない。

個体数制御に限らず，現実の様々な問題を解決するにあたってこの認識は重要です。苦しみは定義からして問題（避けるべきもの，解消すべきもの）であり，問題は誰かの苦しみ（ネガティブな感覚）に関係しています。ですから，苦しみを経験する存在を生み出すのをやめれば，あらゆる問題は消え去ります。誰も存在しなければ，快楽を経験したい，幸福をになる機会が得られない，といって嘆くものもいないため，一切の問題はありません。

しかし，苦しみが問題であるというだけでは，その理想的な状況に至るまでの道筋ある複雑な問題には対処できません。たくさんの動物が互いに影響しあい，一つの影響が予測不能な様々な応答を引き起こす現実世界では，義務や権利といったレトリックや，法律のようにそれらを制度化したものなど，色々な妥協的基準に頼る必要も生じるでしょう。むしろ，「苦しみを最小化すべし」といった単純な原理を複雑な現実世界の問題に適用をすれば，おかしな結論が導かれることもあります。しかし，だからといって，苦しみが問題であるという基本的な事実であることは変わりませんし，実用的な判断であっても，それと本質的に矛盾するようなものは間違いなのです。

4.4 r-K 淘汰理論

4.4.1 r 戦略と K 戦略

ロジスティクス成長は，二つのパラメータ r と K に特徴づけられるのでした。個体数が環境収容力 K より十分小さい場合には，近似的に成長率 r の指数関数的成長に従います。他方，個体数が K に近いときは，その成長は K による影響を強く受けます。これらのことから，自然淘汰は，低密度の環境では r の大きな種に有利に働き，高密度では K を増大させる種に有利に働くことが推測されます。この考え方は，**r-K 淘汰理論（r-K selection theory）**と呼ばれます。また，r あるいは K を増大させる方向に進化する種は，擬人的にそれぞれ**r-戦略（r-strategy）**および**K-戦略（K-strategy）**を取ると表現されます。

r-戦略では，r を高くする見返りに，子供一体一体への投資は小さくなるため，結果的に繁殖可能な年齢まで生き残る数は非常に少なくなります。つまり，多くが成熟する前に死ぬのです。他方，K-戦略では，一度に産む子供の数が少ない分，子供一体一体への資源の投資は多くなり，生存率も高まります。r-K 淘汰理論では，生物の生存戦略は，この両極端の間のどこかに位置づけられると仮定します。

残念なことに，自然界の多くの動物が採用している戦略は r-戦略側に位置しています。そのため，この世界に産み落とされる動物の大半は，生まれてすぐに死んでいくのです。多くが成熟の早い早成動物であり，早い段階から意識的な知覚を持っていると考えられます。この世界に生を受ける動物の大半は，ただ苦しみを経験し死ぬためだけに生まれてくると言えるほどです。これが，自然界で経験される主観的経験は圧倒的に苦しみが支配していると考えられる大きな理由の一つです。

章の冒頭で引用した倫理学者 Oscar Horta は，こうした**野生動物の苦しみ（wild animal suffering）**の問題をいち早く訴え始めた一人です。自然の残酷さに加えて彼がもう一つ強調するのが，人間社会における動物搾取の問題に声を上げているものたちでさえ，野生動物の苦しみには無関心であるということの残念さです。

4.4.2　*r*-*K* 理論の限界と生活史形質

改めて，*r*-*K* 理論は，*r* と *K* というパラメータだけを含むロジスティックモデルに基づくものでした。しかし，実際の生物のダイナミクスにはもっと多くの要因が関与しています。例えば，平均的な生存率は年齢によって変化するし，自然な寿命という上限もあります。具体的な成熟期の開始や繁殖能力も集団の構成を決める大事な因子です。こうした，生涯にわたる生存と繁殖にかかわる性質をまとめて**生活史形質（life-history trait）**と呼びます。

ロジスティックモデルはそれぞれの個体を区別しないため，生活史形質は，含まれるとしても，すべて平均化されてしまっています。それゆえ，対象がロジスティックモデルで記述するには不十分な場合，それに基づく *r*-*K* 理論も単純化しすぎた見方になります (Reznick et al., 2002)。Horta もしばしば，*r*-*K* 理論をもとに自然界の邪悪さを訴えますが，*r*-*K* 理論が過度に単純すぎることは彼も注意しているとおりです (Horta, 2010, 2015)。それでも，野生動物の多くが多産で，生まれた個体の大半が早期に死亡するという事実には変わりなく，*r*-*K* 淘汰理論が持ち出されるのは，その限界に注意すれば，生物の取る戦略の傾向性をおおまかに理解するツールになるためです。

次章以降では，生活史を考慮に入れたモデルについて考えていきましょう。

References

Horta, O. (2010). Debunking the idyllic view of natural processes: Population dynamics and suffering in the wild. *Télos, 17*(1), 73–88.

Horta, O. (2015). The problem of evil in nature: Evolutionary bases of the prevalence of disvalue. *Relations. Beyond Anthropocentrism, 3*(1), 17–32.

Reznick, D., Bryant, M. J., & Bashey, F. (2002). R-and k-selection revisited: The role of population regulation in life-history evolution. *Ecology, 83*(6), 1509–1520.

巖佐, 庸. (1998). **数理生物学入門: 生物社会のダイナミックスを探る（改装版）** [原著: HBJ 出版局, 1990 年]. 共立出版.

第 5 章

推移行列

> “…すべての生物は，こうした屠殺場に存在している。名前が変わり，顔が変わっても，物語は同じだ。同じ悲劇，同じ愚かな消耗，同じ危機，同じ叫びが繰り返されるだけである。そこに達成されるものはない。ただ頭の中にある「我々はここに存在しなければいけない」という観念を満たすだけのことである。”
>
> — Inmendham,
> *EFIList, 2012*

前節までの議論では，ある生物種の個体数はすべてまとめて一つの変数で表していました。しかし，§ 4.4 でも触れたように，生物集団の中でも，個体ごとに年齢や体の大きさなどの性質に違いがあります。そして，どれくらいの年齢や大きさの個体がどれくらいいるかというような，集団の構成も重要な情報になります。ここでは，年齢の構成（齢構成）を中心に，そうした集団の構成を考慮した記述を考えていきます。

5.1　行列

5.1.1　復習

個体数の構成を考える前に，少し復習をしましょう。第 3 章のはじめに，1 ステップの間の個体数の関係が

$$N_{n+1} = (1 + r_{\Delta t}\Delta t)N_n \tag{5.1}$$

で与えられる離散的な Malthus モデルを扱いました。この式は

$$\lambda_{\Delta t} \equiv (1 + r_{\Delta t}\Delta t) \tag{5.2}$$

を定義すれば

$$N_{n+1} = \lambda_{\Delta t} N_n \tag{5.3}$$

と書くこともできます。この $\lambda_{\Delta t}$ は，時間間隔 Δt の間に個体数が何倍になるかを示す量です。もし $\lambda_{\Delta t}$ が一定なら

$$N_n = \lambda_{\Delta t}^n N_0 \tag{5.4}$$

のように，初期値 N_0 から任意の時間ステップでの個体数 N_n が計算できます。

しかし，個体数が，例えば成体と幼体のように年齢ステージに応じて分割される場合，それぞれの年齢ステージにある個体数が同じ比率で変化するとは限りません。その場合，単純に全個体数に同じ係数をかけるだけでは，個体数の変化を正しく記述できません。それでも，以下で導入する数学的概念を用いることで，個体群内部に構造がある場合にも，これまでとよく似た形式で個体数の変化を記述できます。

5.1.2　行列とは

まず，ある集団の個体数を，幼体の個体数と成体の個体数に分けて考えてみましょう。幼体の個体数と成体の個体数をそれぞれ N_1 と N_2 とし，時間ステップの単位を 1 とします。ある時間ステップ $t+1$ の幼体の個体数 $N_1(t+1)$ は，1 ステップ前の成体の数 $N_2(t)$ に比例して生まれる数と，1 ステップの間生き残る幼

体から，成体に変化する数を差し引いた数で与えられるとします。すると，それぞれの比例係数を適当に a と b とし

$$N_1(t+1) = aN_1(t) + bN_2(t) \tag{5.5}$$

と書けます。成体もまた，成長して新たに成体になる幼体の数と，元の個体数に比例する生存数で与えられるとします。すると，ここでも比例係数を c および d として

$$N_2(t+1) = cN_1(t) + dN_2(t) \tag{5.6}$$

と書けます。ただ，ここでの主な目的は数学的な概念の説明なので，係数 a,b,c,d の意味は深く考える必要はありません。おおまかにこういう構造の式を考えていくのだと理解してもらえれば十分です。

さて，このように，個体群を二種類に分類したために，式が二つになりました。しかし，このような二つの式は

$$\begin{pmatrix} N_1(t+1) \\ N_2(t+1) \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} N_1(t) \\ N_2(t) \end{pmatrix} \tag{5.7}$$

あるいは

$$N(t+1) = AN(t) \tag{5.8}$$

のように一つの式にまとめて書くこともできます。ここで

$$N(t) = \begin{pmatrix} N_1(t) \\ N_2(t) \end{pmatrix}, \qquad A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \tag{5.9}$$

としました。このように数を縦と横に並べたものを**行列（matrix）**といいます。

式 (5.8) は，時間 t における状態（個体数の構造）$N(t)$ に，行列 A をかけることで，次の時間 $t+1$ における状態 $N(t+1)$ が得られることを表しています。A が時間によって変わらないとすれば，同じ A を使って式 (5.8) を繰り返し適用することで，未来の任意の時間ステップにおける状態を求めることができます。反対に，$N(t)$ も $N(t) = AN(t-1)$ のように与えられるため，これをさかのぼっていくことで，初期値 $N(0)$ から，任意の時間ステップにおける状態を

$$N(t) = \underbrace{AA\cdots A}_{t} N(0) \tag{5.10}$$

のように求めることもできます。この A のように，異なる時間の間で状態を推移（transitoin）あるいは射影（projection）する行列は，**推移行列（transition**

matrix）や**射影行列（projection matrix**）と呼ばれます。式 (5.10) は，個体数の分布構造を考えないモデルにおける表現 (5.4) を，分布構造がある場合に一般化したものと理解できます。

行列に馴染みのない読者は，新しい数学概念や演算が現れて不安になるかもしれませんが心配いりません。すぐにこれまで用いてきた概念だけで理解できることを示します。すでに行列に馴染みのある読者は，ここから先の行列の説明は読み飛ばしても問題ありません。

5.1.3　行列に関する基本概念

行列の横の並びを**行（row）**，縦の並びを**列（column）**といいます[*1]。(5.9) における $N(t)$ のように，1 列しかない行列は**列ベクトル（column vector）**と呼ばれます。また，(a,b) のように 1 行しかない行列は**行ベクトル（row vector）**と呼ばれます。

(5.9) の A では，(a,b) と (c,d) をそれぞれ第 1 行，第 2 行といい，$\begin{pmatrix} a \\ c \end{pmatrix}$ および $\begin{pmatrix} b \\ d \end{pmatrix}$ をそれぞれ第 1 列，第 2 列と言って指定します。この A のような行列は，行が 2 つ，列が 2 つなので，2×2 行列といわれます。$N(t)$ の方は 2×1 行列になります。

$n\times n$ 行列のように，行と列の数が同じ行列のことを**正方行列（square matrix）**といい，n をその次数といいます。また，次数が n の正方行列のことを，n 次正方行列のように呼びます。

行列を構成する数のことを**成分（component）**や**要素（element）**といい，i 行目 j 列目が交わる部分の数を i 行 j 列成分や (i,j) 成分といいます。例えば (5.9) の A に含まれる a は行列 A の 1 行 1 列成分あるいは (1,1) 成分，b は 1 行 2 列成分あるいは (1,2) 成分という具合です。

[*1] どちらが行でどちらが列かわからなくなるということは良くあります。迷ったときに思い出せるようにする一般的な方法は，「行」の字の「亍」部分に含まれる横線と，「列」の字の「刂」部分に含まれる縦線をそれぞれ行と列に対応付け，行は横，列は縦と覚えるものです。

5.1.4 行列の足し算引き算

行列の足し算は，それぞれの成分を足し合わせることで定義されます。ただし，同じサイズの行列同士でないと足し算はできません。例として，二つの 2×2 行列 A と B の和 $C=A+B$ を考え，A，B および C の (i,j) 成分をそれぞれ，a_{ij}，b_{ij} および c_{ij} と記すことにしましょう（A の $(1,2)$ 成分なら a_{12} などです）。和を成分を用いて表すと

$$c_{ij}=a_{ij}+b_{ij} \tag{5.11}$$

となります。より具体的に書けば

$$\begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} + \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \tag{5.12}$$

です。引き算の場合は符号を + から − に変えるだけで同じようにできます。

■ 例 5.1.

$$A=\begin{pmatrix} 1 & 2 \\ -3 & 4 \end{pmatrix},\quad B=\begin{pmatrix} 5 & -6 \\ 7 & 8 \end{pmatrix} \tag{5.13}$$

とすると

$$C=\begin{pmatrix} 1 & 2 \\ -3 & 4 \end{pmatrix}+\begin{pmatrix} 5 & -6 \\ 7 & 8 \end{pmatrix}=\begin{pmatrix} 6 & -4 \\ 4 & 12 \end{pmatrix} \tag{5.14}$$

5.1.5 行列の積

続いて，式 (5.10) で現れたような，行列の積について考えましょう。式 (5.7) が，式 (5.5) と式 (5.6) と同じであることから，一般に 2×2 行列と 2×1 行列の積は

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}=\begin{pmatrix} ax_1+bx_2 \\ cx_1+dx_2 \end{pmatrix} \tag{5.15}$$

と計算されます。

練習問題 5.1. 以下の式の □ に入る数を計算してください。

$$\begin{pmatrix} 3 & 2 \\ 1 & 4 \end{pmatrix}\begin{pmatrix} 4 \\ 3 \end{pmatrix} = \begin{pmatrix} 18 \\ \square \end{pmatrix}$$

$$\begin{pmatrix} 5 & 1 \\ 2 & 3 \end{pmatrix}\begin{pmatrix} 2 \\ \square \end{pmatrix} = \begin{pmatrix} 14 \\ 16 \end{pmatrix}$$

改めて，式 (5.7) は，左辺が 2×1 行列，右辺が 2×2 行列と 2×1 行列の積になっています。一般的な性質として，行列 A と B の積 AB が定義できるためには，A の列数と B の行数が同じでなくてはいけません。例えば A が $N\times M$ 行列なら，B の行数は M である必要があります。そして，それらの積の結果は，A の行数かける B の列数を持つ行列です。A が $N\times M$ 行列，B が $M\times L$ 行列だった場合，$C=AB$ は $N\times L$ 行列です。わかりやすく色分けして書くと

$$\underbrace{C}_{N\times L} = \underbrace{A}_{N\times M}\underbrace{B}_{M\times L} \tag{5.16}$$

となります。

行列の積の規則 (5.16) は，積の結果の成分を計算するのにも役立ちます。C の成分 c_{ij} は

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots = \sum_k a_{ik}b_{kj} \tag{5.17}$$

と計算されます。つまり，行列 A と B の積 $C=AB$ の (i,j) 成分は，A の i 行 k 列と，B の k 行 j 列成分の積をすべて足したものになります。少し抽象的なので具体的な例で考えてみましょう。まずは

$$A = (a_1, a_2), \quad B = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \tag{5.18}$$

のような行ベクトルと列ベクトルの積を考えてみましょう。積のサイズの規則 (5.16) に従えば，A は 1×2 行列，B は 2×1 行列なので，積の結果は 1×1 行列，つまり添え字を持たない単なる数になります。(5.17) に従って計算すると，A は列しかなく，B は行しかないので，それぞれの列と行の番号が同じ成分同士をかけたものを足し合わせて

$$c = a_1b_1 + a_2b_2 = \sum_{i=1}^{2} a_ib_i \tag{5.19}$$

となります。このように，ベクトルの対応する成分同士をかけて和を取る操作を，ベクトルの**内積（inner product）**と呼びます[*2]。

行と列が増えて

$$A=\begin{pmatrix}a_{11} & a_{12}\\ a_{21} & a_{22}\end{pmatrix},\quad B=\begin{pmatrix}b_{11} & b_{12}\\ b_{21} & b_{22}\end{pmatrix} \tag{5.20}$$

となっても，$C=AB$ の要素の計算は，(5.19) の操作をそれぞれの行と列で

$$\begin{pmatrix}c_{11} & c_{12}\\ c_{21} & c_{22}\end{pmatrix}=\begin{pmatrix}a_{11}b_{11}+a_{12}b_{21} & a_{11}b_{12}+a_{12}b_{22}\\ a_{21}b_{11}+a_{22}b_{21} & a_{21}b_{12}+a_{22}b_{22}\end{pmatrix} \tag{5.21}$$

のように計算すればよいのです。(5.16) に合わせて赤字と青字で表した部分が，それぞれの行列要素の位置に対応し，それらを無視すれば，やっていることは (5.19) と同じだということが確認できると思います。

■ **例 5.2.** 行列

$$A=\begin{pmatrix}1 & 2\\ 0 & 1\end{pmatrix},\quad B=\begin{pmatrix}1 & -1\\ 1 & 0\end{pmatrix} \tag{5.22}$$

の積 AB は

$$\begin{aligned}AB&=\begin{pmatrix}1 & 2\\ 0 & 1\end{pmatrix}\begin{pmatrix}1 & -1\\ 1 & 0\end{pmatrix}\\ &=\begin{pmatrix}1+2 & -1+0\\ 0+1 & 0+0\end{pmatrix}\\ &=\begin{pmatrix}3 & -1\\ 1 & 0\end{pmatrix}\end{aligned} \tag{5.23}$$

と計算されます。

練習問題 5.2. 以下の式の □ に入る数を計算してください。

$$\begin{pmatrix}1 & 4\\ 3 & 2\end{pmatrix}\begin{pmatrix}2 & 1\\ 3 & 2\end{pmatrix}=\begin{pmatrix}14 & \square\\ 12 & 7\end{pmatrix}$$

$$\begin{pmatrix}2 & 3\\ 1 & 2\end{pmatrix}\begin{pmatrix}4 & 1\\ \square & 3\end{pmatrix}=\begin{pmatrix}14 & 11\\ 8 & 7\end{pmatrix}$$

[*2] 細かいことを言うと，内積と呼ばれる演算の一種で，Euclid 内積と呼ばれるものです。

四則演算のうち，足し算と引き算，および掛け算は以上のように定義できることがわかりました。しかし，行列の割り算は定義できません。対応する演算は定義されるのですが，それについての説明は Appendix. C に回します。

5.1.6　Python による計算

行列の演算は，NumPy などを使えば，Python では驚くほど簡単に済ませられます。ここでは，これまでに説明した行列の定義，足し算，行列とベクトルの積を実際に Python で行う例を見てみましょう。

まず，行列の作り方です。行列は複数の数を並べたものでした。私たちはこれまで，プログラム上でも，配列を用いて，いくつもの要素をひとまとまりにしたデータを扱ってきました。行列を扱うのも，これを用いればいいです。ベクトルであれば（2 次元の場合）

```
N = np.array([100, 50])
```

とすれば，これで OK です。最初に固い括弧を使ってリストを作り，プログラム上扱いやすいように np.array に変換しています。より高次のベクトルでも同じようにすればよいです。数学上は，行ベクトルか列ベクトルかの違いがありましたが，それは演算の際に違いが現れるだけで，構成自体は同じであるため，定義の時点ではどちらも同じようにこれで完了です。

続いて行列ですが，行列は同じ大きさのベクトルを縦または横に並べたものとみなせます。よって，ベクトルに対応する要素を要素とする配列を作れば，それが行列の役割を果たします。以下は，2 × 2 行列の例です

```
A = np.array([[0.4, 1.2],
              [0.6, 0.8]])

print(A)

# 出力
[[0.4 1.2]
 [0.6 0.8]]
```

出力結果も数学的な表現と対応しており，わかりやすいでしょう。

ベクトルや行列の要素を取り出すときは，要素番号を指定して

```
print(N[0])
```

```
# 出力
100

print(A[0,1])

# 出力
1.2
```

とすればよいです。このとき，リストのインデックスは 0 から始まることを思い出してください。`N[0]` は一つ目の要素を，`A[0,1]` は，1 行 2 列目の要素を表しています。

行列の足し算や引き算はそのままで，同じサイズの行列同士で以下のようにします。

```
B = np.array([[0.1, 0.1],
              [0.2, 0.1]])
C = A + B
print(C)
print("2行2列成分：", C[1,1])

print(A - B)
```

行列の積には，通常の掛け算で使う「`*`」ではなく，「`@`」演算子，または NumPy の `dot` 関数を使用します。$N(t+1) = AN(t)$ に対応する演算は

```
N = A @ N
```

あるいは

```
N = np.dot(A, N)
```

です。2 次元の行列と 1 次元のベクトルの計算であれば，どちらを使ってもまったく同じ結果が得られます。例 5.2 の計算もこれにより簡単に計算できます。練習問題に試してみてください。

5.2 固有値と弾力性

元々は式 (5.5) および (5.6) のように表せていたものを，わざわざ新しい概念を導入して書き換えたのにはもちろん理由があります。元の式は複数の変数

（N_1, N_2）と定数（a から d）が絡み合った形をしていましたが，行列形式にすることで，役割の違うそれらの量が分離されました。これにより，ある時点での具体的な生物の構成とは無関係に，その集団に内在する繁殖や死亡に関わる性質だけに焦点を当てられるのです。その結果，様々な重要な性質を明らかにすることができます。これは，特定の集団の繁殖ではなく，ある生物種，あるいは生物全体の繁殖の性質そのものを問題にしている私たちの姿勢にもなじみやすい形式です。

ただし，第 4 章で学んだように，生存率や繁殖力がそのときの個体数の密度に依存することもあるので，そのような場合には行列要素が状態から完全に独立になるというわけではありません。それでも，状態と変換規則への分離には大きな意味があります。ここでは行列の構成からわかる性質のうち，個体数の成長とその抑止にかかわる重要なもののいくつかについて学びます。

5.2.1　固有値と成長率

式 (5.7) は，行列 A があるベクトルに作用することで，別のベクトルが得られることを示しています。一般に，このような式 $y = Ax$ において，ベクトル x は行列 A の作用により異なる構成の y に変換されます。しかし，場合によっては，行列の作用の前後のベクトルの関係が，定数 λ を用いて

$$Ax = \lambda x \tag{5.24}$$

となること，つまり定数倍しただけの関係になることがあります。この関係を満たすゼロでないベクトル x を**固有ベクトル（eigenvector）**といい，定数 λ を行列 A の**固有値（eigenvalue）**といいます。固有値およびそれに属する固有ベクトルは複数あり得ますが，定数倍だけの関係の固有ベクトルは同じものとしてカウントします。また，固有値の中で最も大きなものを**最大固有値（dominant eigenvalue）**と呼びます。

ここでは，推移行列が 2 次の正方行列で，2 つの固有ベクトル x_1, x_2 を持つ場合を考えましょう。このとき，同じ次元のどんなベクトルも，何らかの係数 c_1, c_2 を用いて

$$x = c_1 x_1 + c_2 x_2 \tag{5.25}$$

のように表せます。ここで，x_1 が最大固有値に属する固有ベクトルであるとし

ましょう。現実の生物個体群の初期状態であれば一般にその係数 c_1 はゼロではありません。つまり，どんなベクトルも，(5.25) の意味で最大固有値に属するベクトルを含みます。

式 (5.24) は，個体数の推移の文脈では，齢構成を保ったまま，毎ステップ個体数が λ 倍されるような個体数の分布が存在することを意味します。式 (5.25) より，どんな状態から始まったとしても，その状態に含まれる最大固有値に属するベクトルが最も速く成長していきます。そのため，十分時間が経つとこの成分が全体を支配するようになり，集団の構成比は，最大固有値の固有ベクトルの成分比に収束します。このような分布（ベクトル）は，**安定齢分布（stable age distribution）**と呼ばれます。また，式 (3.36) と比べると，最大固有値 λ は成長率に関する因子 $e^{r\Delta t}$（今は $\Delta t = 1$）に対応するため，同じように成長率や増加率と呼ばれます。$\lambda > 1$ であれば個体数は時間とともに増大しますが，$\lambda < 1$ であれば，個体数は時間とともに減少し，その集団は絶滅に向かいます。

ここで一つ注意ですが，行列の固有値は，**複素数（complex number）**になることがあります。複素数は，実数 x, y と**虚数単位（imaginary unit）** $i = \sqrt{-1}$ を用いて，$z = x + iy$ で表される数です。このとき x と y はそれぞれ z の実部と虚部と呼ばれます。詳細は Appendix A.2 で説明しますが，固有値の虚部（y にあたる部分）は，増減ではなく，時間的な振動に関係します。

5.2.2 固有値問題と Python による計算

個体群の原理的な成長率を決めるのは，個体群の構成ではなく，推移行列に含まれる繁殖力や生存率です。このことからも納得できるように，固有値は行列自身の性質であり，行列が与えられれば，固有値と対応する固有ベクトルが求められます。これらの固有値や固有ベクトルを求める問題は，**固有値問題（eigenvalue problem）**と呼ばれます。ただ，固有値問題の解き方は本書の想定している範囲を超えるので，その基本的な説明も Appendix. C に回します。関心のある読者はそちらを読んでもらうことにして，本文では初めから Python の利点を活用していくことにします。

固有値問題は，`NumPy` の関数 `linalg.eig()` という関数にすべて任せられます。以下のように行列 `A` を投げ入れれば，固有値と固有ベクトルを返してくれます。

```
evals, evecs = np.linalg.eig(A)
```

しかし，以下で必要になるのは，固有値だけなので，その場合，`linalg.eigvals` を使うことができます。

```
    evals = np.linalg.eigvals(A)
```

後のために，これを用いて，行列を与えたらその最大固有値を返す関数として用意しておきましょう。

```
def get_growth_rate(A):

    evals = np.linalg.eigvals(A)
    lam = np.max(evals.real)

    return lam
```

サンプルコード 5.1　固有値と固有ベクトルを求めるスクリプト。

先述べたように，固有値は一般に複素数なのですが，今はその実部にのみ興味があるので，`evals.real` により，実部を取り出しています。また，固有値は複数あり得るので，NumPy の関数 `max` で最大固有値を選び出し，それを λ（`lam`）としています[*3]

5.2.3　感度と弾力性

推移行列は，生存率や繁殖力などから構成されます。私たちが知りたいのは，これらのうち，どの値を変化させれば，最も効率的に成長率を下げられるかということです。行列要素やそれを構成する変数 a を Δa だけ変化させたときの固有値の変化を $\Delta\lambda$ とすると，変化率

$$\frac{\Delta\lambda}{\Delta a}$$

が大きいほど，固有値はその変数の変化に敏感であると言えます。色々な Δa の変化を比べるとき，変化量の取り方が曖昧なので，極限 $\Delta a \to 0$ を取って，第 3 でやった微分で表すのが良いでしょう。ただしこのとき，複数ある変数のうち，

[*3] ちなみに，λ の綴りは「lambda」ですが，Python には `lambda` という名前の組み込み関数があるので，そのまま `lambda` を変数に使うことはできません。

残りの成分は固定したまま，特定の a だけを変化させるということを表すため，$d\lambda/da$ ではなく，より柔らか記号「∂」を使って

$$s = \frac{\partial \lambda}{\partial a} \tag{5.26}$$

と表現します。このように，複数の変数のうち，残りのすべてを固定してある変数に関する無限小の変化率を求める方法を，**偏微分（partial derivative）**と呼びます。たまたまこの文脈で導入することになっていますが，偏微分は変数が複数ある場合の問題で極めて一般的な概念で，第 8 章で改めて解説する予定です。

こうして定義される (5.26) のことを，a に関する λ の**感度（sensitivity）**といい，それを調べることを**感度分析（sensitivity analysis）**といいます。ただ，推移行列には繁殖率や生存率あるいは幼体から成体に移行する確率など，様々な異なる量が含まれます。生物によっては，繁殖率が生存率よりも何桁も大きいということもあり得るため，異なる要素に関する感度同士で単純な比較をするのは必ずしも適切ではありません。そこで，それぞれの元の値に対する変化割合

$$\frac{\Delta \lambda}{\lambda}, \quad \frac{\Delta a}{a} \tag{5.27}$$

比の極限から定義される

$$e = \frac{a}{\lambda}\frac{\partial \lambda}{\partial a} = \frac{a}{\lambda}\lim_{\Delta a \to 0}\frac{\Delta \lambda}{\Delta a} \tag{5.28}$$

も重要な量として用いられます。この量は，**弾力性（elasticity）**と呼ばれます。

弾力性は，基準となる固有値 `lam_std` と，ある変数 `a` を微小量 `eps` だけ変化させた結果得られる行列 `A` の固有値 `lam` から，以下のような関数で計算できます。上の固有値を求める関数をそのまま使います。

```
eps=1e-6

def calc_elasticity(A, a, lam_std):

    lam = get_growth_rate(A)
    dlam_da = ( lam - lam_std ) / eps

    elasticity = ( a / lam_std ) * dlam_da

    return elasticity
```

ここで，1e-6 は，10^{-6} を表します。試しに，適当な基準行列を作り，そこから各要素を eps ずらした行列を渡して以下のようにすれば，各行列要素に関する弾力性が計算できます。

```
# standard matrix
a11 = 1.1
a12 = 1.0
a21 = 0.7
a22 = 0.8
A = np.array( [ [a11, a12], [a21, a22] ] )
lam_std = get_growth_rate(A)

# elasticity
A11e = np.array( [ [a11+eps, a12], [a21, a22] ] )
A12e = np.array( [ [a11, a12+eps], [a21, a22] ] )
A21e = np.array( [ [a11, a12], [a21+eps, a22] ] )
A22e = np.array( [ [a11, a12], [a21, a22+eps] ] )

e11 = calc_elasticity(A11e, a11, lam_std)
e12 = calc_elasticity(A12e, a12, lam_std)
e21 = calc_elasticity(A21e, a21, lam_std)
e22 = calc_elasticity(A22e, a22, lam_std)

plt.bar( ["a11", "a12", "a21", "a22"], [e11, e12, e21, e22],
  width=0.5)
plt.xlabel('Matrix element')
plt.ylabel('Elasticity')
plt.legend()
plt.show()
```

サンプルコード 5.2　弾力性を計算して描画するスクリプト。

実行例は，図 5.1 の通りです。

5.3　野良ネコの行列モデル

さて，本章ではこれまで，行列とそれに関する色々な概念を導入してきました。この節では，具体的な例を通して，これまで導入した概念を実際に使って分析していきましょう。例として，Andersen et al., 2004 の研究を取りあげます。この研究では，野良ネコの個体群動態を行列モデルで記述し，TNR（避妊，去勢）と殺処分を想定した，繁殖率（fecundity rate）と生存率（survival rate）の抑制が，成長率にどう影響するかを調べています。

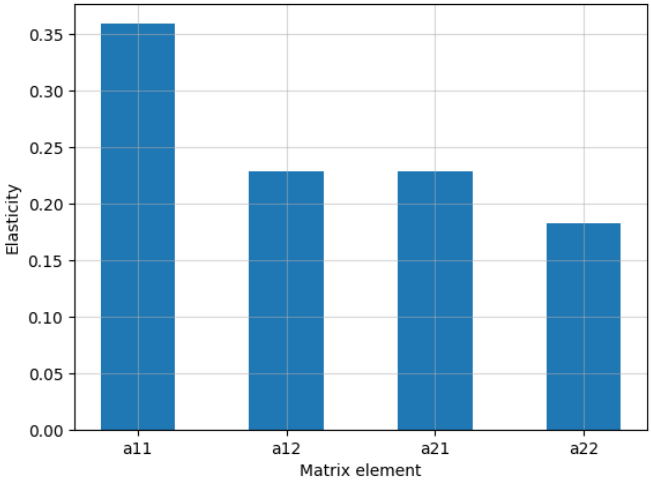


図 5.1　サンプルコード 5.2 の実行例。横軸が各行列要素，縦軸がそれらに対する弾力性の値です。

5.3.1　モデル設定

この研究で用いたモデルでは，年齢のステージは，幼ネコと成ネコで，1 ステップの時間は 1 年としています。繁殖は主にメスの性質により決まると仮定し，メスの動態だけを考えます。

まず，幼ネコの個体数の変化を決める行列要素を決めていきましょう。ネコは生後 1 年以内（早ければ半年）で繁殖可能になるので，幼ネコの繁殖力もゼロではありません。すると，幼ネコの生殖による幼ネコの個体数 N_1 の増分は，幼ネコの 1 年あたりの生存率 S_1 と，繁殖力 F_1 を用いて，$S_1F_1N_1(t)$ と書けます。成ネコが生殖することによる幼ネコの個体数の増分も同じように考えれば成ネコの 1 年あたりの生存率 S_2 と，繁殖力 F_2 を用いて，$S_2F_2N_2(t)$ と書けるでしょう[*4]。

ただ，Andersen et al., 2004 のモデルでは少し工夫しています。ネコは年中繁

[*4] 前節の表記と統一するために，元論文とは添え字を変更してあります。

殖可能なのですが，このモデルでは 1 年単位でしか解析できないので，そのままでは，その間に生まれたり死んでしまったりする効果が考慮できません。そこで，出産は年の中間点で起こると仮定して行列要素を定義します。すると，翌年の幼ネコの個体数にカウントされるには，親ネコになるネコは，年の真ん中まで生存し，生まれた幼ネコは残りの半年生存しなくてはいけません。それぞれの半年分の生存率は，$\sqrt{S_1}$ と $\sqrt{S_2}$ で与えられます（二乗すると 1 年の生存率になります)。よって，翌年の幼ネコの個体数 $N_1(t+1)$ は

$$N_1(t+1) = S_1F_1N_1(t) + \sqrt{S_1}F_2\sqrt{S_2}N_2(t) \tag{5.29}$$

と書けます。

翌年の成ネコの個体数 $N_1(t+1)$ は，1 年間生き延びた幼ネコと成ネコの個体数の合計なので

$$N_2(t+1) = S_1N_1(t) + S_2N_2(t) \tag{5.30}$$

です。これらの式を行列を用いて表すと

$$\begin{pmatrix} N_1(t+1) \\ N_2(t+1) \end{pmatrix} = \begin{pmatrix} S_1F_1 & \sqrt{S_1}F_2\sqrt{S_2} \\ S_1 & S_2 \end{pmatrix} \begin{pmatrix} N_1(t) \\ N_2(t) \end{pmatrix} \tag{5.31}$$

となります。

5.3.2　TNR と殺処分効果の比較

こうして構成された行列モデルには，4 つのパラメータが含まれます。幼ネコと成ネコそれぞれの繁殖力と生存率です。論文では様々な値の組み合わせについて調べていますが，ここでは一例だけ考えてみましょう。幼ネコと成ネコのの 1 年あたりの生存率はそれぞれ $S_1 = 0.5$ および $S_2 = 0.7$ とします。つまり，幼ネコのうち，生後 1 年を生き延びるのは半分だけです。これでも実際よりはかなり大きい値です。繋殖率については，それぞれ $F_1 = 1.58$ および $F_2 = 3.78$ とします。これらは，幼ネコと成ネコのメスが 1 年に産む子供の数の平均値です。すると，式 (5.31) の推移行列は

$$A = \begin{pmatrix} 0.79 & 2.236 \\ 0.5 & 0.7 \end{pmatrix} \tag{5.32}$$

となります。これをコード化しましょう。後でパラメータを変更しやすい書き方にしておきます。

```
def make_matrix(S1,S2,F1,F2):

    row1 = [S1 * F1, np.sqrt(S1) * F2 * np.sqrt(S2)]
    row2 = [S1, S2]
    A = np.array( [ row1, row2 ] )

    return A

S1 = 0.5
S2 = 0.7
F1 = 1.58
F2 = 3.78

A = make_matrix(S1, S2, F1, F2)
```

関数 make_matrix を定義して，渡された変数（S1，S2，F1，F2）から行列 A を構成しています。まずこの行列の固有値を求めてみましょう。サンプルコード 5.1 と結合して，実行すると，1.803... と続く実数が出力されるはずです。

さて，私たちが調べたいのは，生存率と繁殖力のうち，どれを変化させたら，最も効率的に λ を小さくできるかということです。そのために，TNR の影響を想定し，繁殖力を下げた場合と，殺処分を想定して生存率を下げた場合の λ の変化を調べます。それをするには，ポチポチとパラメータを変更しては固有値を出力し，結果をどこかに書き留めておいて比較すればいいのですが，それでは一般性が得られないですし，プログラミングの利点も活かしきれませんので，コードを編集してより効率的に計算できるようにしましょう。繁殖率や生存率を一定の割合変更するだけで，あとは同じ処理を繰り返すだけなので，検証する減少率を reduction=[0,0.25,0.5,0.75] のように一つのリストにまとめて，それに関してループを回すのがいいでしょう。

```
for r in reduction:

    rate = 1.0 - r
    A_TNR = make_matrix(S1, S2, F1 * rate, F2 * rate)
    A_LC  = make_matrix(S1 * rate, S2 * rate, F1, F2)
```

後はこうして得られた行列から最大固有値を求め，append を使って結果をリストにまとめていけば，各介入率に対する成長率が記録できます。

```
S1, S2 = 0.5, 0.7
```

```
F1, F2 = 1.58,3.78
reduction = [0, 0.25, 0.5, 0.75]

lam_TNR = []
lam_LC  = []
for r in reduction:

    rate = 1.0 - r
    A_TNR = make_matrix(S1, S2, F1 * rate, F2 * rate)
    A_LC  = make_matrix(S1 * rate, S2 * rate, F1, F2)

    lam_TNR.append( get_growth_rate(A_TNR) )
    lam_LC.append( get_growth_rate(A_LC) )

plt.plot(reduction, lam_TNR, marker="o", label='TNR')
plt.plot(reduction, lam_LC, marker="o", label='LC')
plt.xlabel('Reduction rate (%)')
plt.ylabel(r'Growth Rate ($\lambda$)')
plt.xticks(reduction)
plt.legend()
plt.show()
```

サンプルコード 5.3　Andersen et al., 2004 に基づく簡易モデルのスクリプト。

get_growth_rate は，サンプルコード 5.1 そのままです。

サンプルコード 5.3 を実行すると，図 5.2 のような結果が得られます。横軸が生存率あるいは繁殖率の減少率で，縦軸が個体数の成長率です。水平にひかれた点線は，成長率が 1 のところを表し，それを下回れば個体数が減少することを意味します。TNR に対応する青線は，常にこの基準線よりも上です。つまり，TNR により繁殖率を 75% 減らしても，成長率は 1 を下回りません。一方，殺処分（LC）により生存率を下げていくと，50% ほど減らしたところで成長率が 1 を下回り，個体数が減少に転じることが確認できます。この結果は，ネコの個体数の増加を抑えるには，繁殖力を下げる（TNR）よりも，生存率を下げる（殺処分）方が効果的であることを示唆しています。

5.3.3　弾力性

弾力性も，先ほどのサンプルコード 5.2 の関数をそのまま使い，以下のコードを追加することで計算および出力可能です。

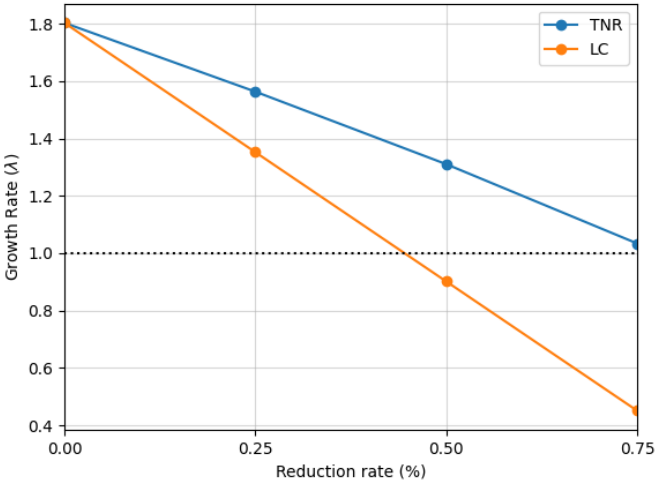


図 5.2　サンプルコード 5.3 の実行例。一部，サンプルコードでは省略している図の装飾などを使用しています。

```
A_std = make_matrix(S1, S2, F1, F2)
lam_std = get_growth_rate(A_std)

AS1 = make_matrix(S1+eps,S2,F1,F2)
AS2 = make_matrix(S1,S2+eps,F1,F2)
AF1 = make_matrix(S1,S2,F1+eps,F2)
AF2 = make_matrix(S1,S2,F1,F2+eps)

eS1 = calc_elasticity(AS1, S1, lam_std)
eS2 = calc_elasticity(AS2, S2, lam_std)
eF1 = calc_elasticity(AF1, F1, lam_std)
eF2 = calc_elasticity(AF2, F2, lam_std)

plt.bar( ["S1", "S2", "F1", "F2"], [eS1, eS2, eF1, eF2], width
  =0.5)
plt.xlabel('Matrix element')
plt.ylabel('Elasticity')
plt.show()
```

結果は図 5.3 のようになります。この結果からも，成長率を下げるには，繁殖力よりも生存率を下げる方が効果的であることが確認できますが，より具体的

に，幼ネコの生存率 S_1 を下げることが最も効果的であることが見て取れます。

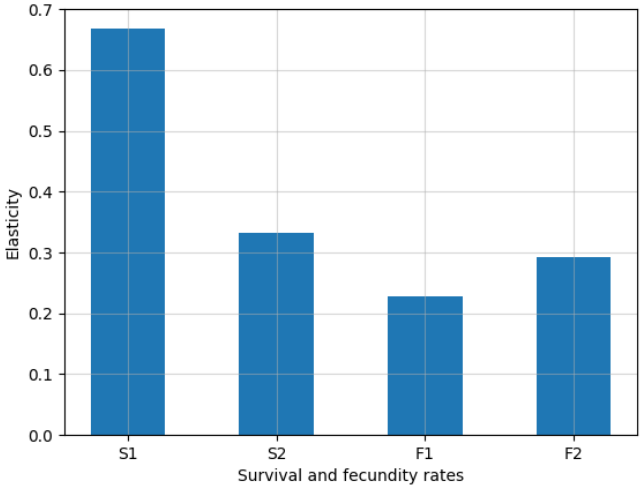


図 5.3

Andersen et al., 2004 も，いろいろとパラメータを変えて調べた結果，論文中で想定されるシナリオの範囲で同様の結果になることを示しています。ただし，見ての通りこのモデルはかなり単純化されたものです。例えば，個体数が減ったことにより，生存率が増加する効果や，考えているコロニーの外から新たな個体が移動してくる効果などが考慮されていません。そのため，殺処分の効果が過大に評価されていることには注意が必要です。また，この解析は殺処分と TNR のみを比較するものですが，他にも，保護や，あるいは TVHR や不妊化ワクチンといった別の不妊化方法が存在することも忘れてはいけません。

5.3.4　動物の殺処分について

Andersen et al., 2004 の他にも，野良ネコの個体数抑制において，TNR により繁殖力を下げる取り組みは非効率的であることや，それに比べて生存率を下げる（あるいは保護などにより集団から取り除く）方が効果的だということを示す研究は複数あります［例えば Foley et al., 2005; Miller et al., 2014］。また，野良ネ

コの生活環境は過酷であり，安楽死を行う方が彼ら自身にとってもより良い選択だという議論もあります (Robertson, 2008)。しかし，殺処分により実際の野良ネコの個体数を減らした例は，主に新たな個体の流入が無視できる小さな離島に限られることも指摘されています (McCarthy et al., 2013)。McCarthy et al., 2013 はさらに，行列モデルよりも詳細な，第 9 章で扱うエージェントベースのモデルを用いたシミュレーションにより，TNR や殺処分よりも TVHR の方が有効であることを議論しています。この研究では，TNR や殺処分では年間 57% 以上の個体に介入を行わない限り 顕著な個体数減少は見られない一方で，TVHR であれば年間 35% 程度の介入でも個体数減少が生じることが示されています。いずれにしても，生を終わらせることは，始めさせることと同じように最大限慎重に考えなくてはいけません。現実には，野良ネコの殺処分は，代替案に最大限のリソースを費やしたうえで実行されているわけではありません。

野良ネコ以外の野生動物の場合はもっとひどい状況です。シカやクマなどの動物は，個体数をゼロに向けて減らすためでなく，人間にとって「ちょうどいい」数に維持するために殺処分が実行されます。その調整に利用されるのが，フィールド調査などに加え，本書で扱っているような個体数シミュレーションによる影響の予測です。恐らく，こうした殺しを含めた管理に関わる人たちや，それを支持している大衆は，それが科学的根拠に裏付けられた客観的判断に基づく最善の取り組みであると考えているでしょう。ですが，彼らは個々の動物が経験する苦しみには関心がないし，彼らの目的は，「生物を絶滅から守ることは当然良いことだ」というドグマに基づいた完全に的はずれなものです。ですから，いくら根拠と正確な予測に基づくものであっても，やっていることが正しいということにはなりません。

他方で彼らは，動物を擁護する側の声は感情的で全体像を考慮できていない意見としかみなしていないでしょう。だからこそ，私たちも数理解析の方法を学び，彼らと同じ土台の上で議論できるようにする意味があるのです。これにより，私たちは彼らの手法を理解したうえで異なるアプローチを提唱しているという主張が可能になります。またその上で，彼らの視点からは見過ごされがちな手法やリソースの使用法の提案も可能になるでしょう。

5.4　クマの行列モデル

5.4.1　推移行列の定義

今度は前の例よりも構成の分類を増やした例を考えてみましょう。Lewis et al., 2014 は，人間社会との接触を含むクロクマ［アメリカグマ（*Ursus americanus*）］の個体数変動を行列モデルを用いて調べています。

この研究でも，メスの個体数の動態のみに着目します。そして，時間ステップを 1 年に取り，ライフステージを 4 つに分類します。一つ目は 0 歳から 1 歳までの子熊（cub），二つ目は繁殖能力を得る前の亜成体（subadult），三つ目は繁殖可能な成体，そして四つ目は子供のいる成体です。三つ目の繁殖可能な成体とは，前の年に子供を産まなかったか，何らかの理由で子供を失ってしまい，その年に再び繁殖可能となった個体を指します。

このように，成体の間でもステージの移り変わりがあり得るため，その割合を表す量である遷移率（transition rate）の導入が必要になります。また，ネコのモデルで用いた繁殖率も，遷移率で置き換えることができます。1 ステップ後に，繁殖可能な成体から子供のいる成体へ変化する割合を τ_{ar} とし，一回の出産で生まれる平均的なメスの個体数を L とすれば，$\tau_{ar}L$ が繁殖可能な個体 1 体あたりの平均的な出産数になるからです。$t+1$ ステップでの繁殖可能な個体数は，生存率を S_a として $S_aN_a(t)$ なので，子熊の個体数 N_c は

$$N_c(t+1) = S_a\tau_{ar}LN_{\mathrm{a}}(t) \tag{5.33}$$

と計算できます。

亜成体の個体数 N_s の場合は，前年生存した子熊の数と，前年亜成体であったものが生存し，かつ成体にならなかった数の和で決まります。前者は子熊の生存率を S_c として S_cN_c です。生存した亜成体の数 $S_sN_s(t)$ のうち，成体にならなかった割合は $(1-\tau_{sa})$ なので，後者は $S_s(1-\tau_{sa})N_s$ です。よって

$$N_s(t+1) = S_cN_c(t) + S_s(1-\tau_{sa})N_s(t) \tag{5.34}$$

です。残りの 2 つのステージについても同じように考えると，推移行列は

$$A = \begin{pmatrix} 0 & 0 & S_a\tau_{ar}L & 0 \\ S_c & S_s(1-\tau_{sa}) & 0 & 0 \\ 0 & S_s\tau_{sa} & S_a(1-\tau_{ar}) & S_r \\ 0 & 0 & S_a\tau_{ar} & 0 \end{pmatrix} \tag{5.35}$$

となります。ここで，S_r は，前年に子供を育てた個体の生存率です。

5.4.2　シミュレーションによる人為的影響の評価

クマは，ドングリなどの木の実が不作の年に，ゴミなどの人為的な食糧源を求めて人間の生息域にやってくることが増えます。これにより繁殖力は維持されますが，人間により殺害されるリスクが高まります。Lewis et al., 2014 は，主に保全の観点から，これら繁殖力の増加と生存率の低下のリスクがクマの個体数に与える影響を分析しています。

論文では，人間社会との接触がなく，したがって人為的な食糧にも依存しないし，殺処分の対象にもならない場合をベースラインとし，それを基準に殺処分率が低い場合（low removal）と高い場合（high removal）を比較しています。これら 3 つのケースについて，それぞれ自然のエサが豊作な年と不作な年ごとに，生存率，遷移率および産子数の値を設定しています。

```
def make_matrix(S_c, S_s, S_a, S_r, tau_sa, tau_ar, L):

    A = np.array([
        [0,    0,                   S_a * tau_ar * L,    0  ],
        [S_c, S_s * (1 - tau_sa), 0,                    0  ],
        [0,    S_s * tau_sa,        S_a * (1 - tau_ar), S_r],
        [0,    0,                   S_a * tau_ar,        0  ]
    ])
    return A
```

この関数に，シナリオごとの値を渡せば，対応する行列が作られます。例えばベースラインシナリオで，豊作の年の場合を素直に書くと

```
A_base_good = make_matrix(0.60, 0.75, 0.90, 0.90, 0.33, 0.90,
  1.10)
```

のようになります。しかし，引数が 7 個もあり，これでは異なるシナリオごとにいちいち書き込むのは面倒です。そこで，シナリオごとのパラメータの組を §

2.5.3 で説明している辞書として，以下のように定義します（Lewis et al., 2014 の Table 2 の値をそのまま利用しています）。

```
# Baseline
base_good = {'S_c': 0.60, 'S_s': 0.75, 'S_a': 0.90, 'S_r':
  0.90, 'tau_sa': 0.33, 'tau_ar': 0.90, 'L': 1.10}
base_poor = {'S_c': 0.40, 'S_s': 0.60, 'S_a': 0.85, 'S_r':
  0.85, 'tau_sa': 0.25, 'tau_ar': 0.60, 'L': 0.60}
```

```
# Low Removal
low_good  = {'S_c': 0.60, 'S_s': 0.75, 'S_a': 0.90, 'S_r':
  0.90, 'tau_sa': 0.33, 'tau_ar': 0.95, 'L': 1.15}
low_poor  = {'S_c': 0.45, 'S_s': 0.65, 'S_a': 0.80, 'S_r':
  0.80, 'tau_sa': 0.33, 'tau_ar': 0.95, 'L': 1.15}
```

```
# High Removal
high_good = {'S_c': 0.60, 'S_s': 0.75, 'S_a': 0.90, 'S_r':
  0.90, 'tau_sa': 0.33, 'tau_ar': 0.95, 'L': 1.15}
high_poor = {'S_c': 0.35, 'S_s': 0.60, 'S_a': 0.70, 'S_r':
  0.70, 'tau_sa': 0.33, 'tau_ar': 0.95, 'L': 1.15}
```

これも § 2.5.3 で説明している通り，こうすることで，引数として辞書をそのまま渡せばよく

```
A_base_good = make_matrix(**base_good)
A_base_poor = make_matrix(**base_poor)

print(f"Baseline scenario (good): {get_growth_rate(A_base_good
  ):.4f}")
print(f"Baseline scenario (poor): {get_growth_rate(A_base_poor
  ):.4f}")
```

とするだけで済みます。そして，別のシナリオ，例えば殺処分率が低い場合のシナリオを調べたければ

```
A_low_good  = make_matrix(**low_good)
A_low_poor  = make_matrix(**low_poor)

print(f"Low removal scenario (good): {get_growth_rate(
  A_low_good):.4f}")
print(f"Low removal scenario (poor): {get_growth_rate(
  A_low_poor):.4f}")
```

のようにするだけで計算ができます。

シナリオごとのパラメータは，フィールド調査の結果を基に，次のように選ばれています。自然なエサが不作の年は，成体よりも若い段階の個体の生存率に強い影響を与えると仮定されています。また，不作の年には成熟率（τ_{sa}）や出産率（τ_{ar}）が下がるよう設定されています。これに，人間社会との接触効果が加えられます。人為的な食糧の効果により，不作の年であっても，高い出産率（$\tau_{ar} = 0.95$）と産子数（$L = 1.15$）が維持されることが仮定されます。しかし，殺処分の影響により，生存率が下げられます。

人為的な食糧と殺処分の影響を詳しく評価するには，前の節でやったように，弾力性を調べる必要があります。しかし，以下のように，より簡易的に比較することもできます。

```
base_food = {'S_c': 0.40, 'S_s': 0.60, 'S_a': 0.85, 'S_r':
  0.85, 'tau_sa': 0.25, 'tau_ar': 0.95, 'L': 1.10}
A_base_food = make_matrix(**base_food)

print("=== Comparison of anthropogenic effects ===")
print(f"Baseline                  : {get_growth_rate(
  A_base_poor):.4f}")
print(f"Anthropogenic food effect: {get_growth_rate(
  A_base_food):.4f}")
print(f"Food + removal            : {get_growth_rate(
  A_high_poor):.4f}")
```

ここでは，新たに，不作の年のベースラインシナリオから，人為的な食糧の影響による出産率と産子数の増加の効果だけを反映した，新たなパラメータの組 base_food を定義し，その場合の成長率を，それを不作年のベースラインおよび高殺処分シナリオの成長率と比較しています。結果は以下のように出力されるはずです。

```
=== Comparison of anthropogenic effects ===
Baseline                  : 0.8807
Anthropogenic food effect: 0.9163
Food + removal            : 0.7930
```

つまり，人為的な食糧の影響だけであれば，不作の年でも成長率が上がります。しかし，高頻度の殺処分が起こると，それよりもさらに大きな影響で成長率を下げるため，人間社会とのかかわりがない場合よりも個体数減少を促します。元

論文では，保全の観点から，殺処分に頼ることは問題があり，人間の側がゴミの管理に注意を払うなどして接触を減らすよう努めるべきだと結論付けています。

5.4.3　結果の考察

私たちの目的は保全ではありませんが，クマ殺しそれ自体も深刻な問題です。「クマの個体数の減少は，生態系全体の変化を通し，結果的に人間社会にも悪影響を及ぼす」というレトリックと合わせれば，こうした数理モデルの結果に基づく議論は無思考な人間中心主義者に対しても役立つでしょう。しかし，エフィリズム的観点からも，クマの行動や人間社会との相互作用が個体数に与える影響を理解しておくことは重要です。例えば，殺処分以外のクマの個体数制御にも，**不妊化ワクチン（contraceptive vaccine）**の使用が検討されています。この種のワクチンは，野生動物に投与することで，繁殖力を抑止し，殺処分することなく個体数を制御するための開発されており，クマに対しても一定の有効性が確認されています (Lane et al., 2007)。私たちも，本節で扱ったような野生動物の個体数モデルに，不妊ワクチンの効果を組み込むなどして影響を調べ，その意義を社会的に発信していくことが一つの有意義な活動となるでしょう。

さて，この節では，ライフステージを増やし，行列の次元が大きくなっても，基本的な部分は前節ですでに作ったコードを基に計算できることがわかりました。元論文ではさらに，殺処分率の違うシナリオごとに，1 ステップごとに自然なエサが豊作か不作かを確率的に決め，長期的な個体数の変動を調べています。第 7 章で確率論的方法を導入した後，このようなシミュレーションも再現してみましょう。

References

Andersen, M. C., Martin, B. J., & Roemer, G. W. (2004). Use of matrix population models to estimate the efficacy of euthanasia versus trap-neuter-return for management of free-roaming cats. *Journal of the American Veterinary Medical Association, 225*(12), 1871–1876.

Foley, P., Foley, J. E., Levy, J. K., & Paik, T. (2005). Analysis of the impact of trap-neuter-return programs on populations of feral cats. *Journal of the American Veterinary Medical Association*, *227*(11), 1775–1781.

Lane, V., Liu, I., Casey, K., Flanagan, D., Murata, K., Munro, C., et al. (2007). Inoculation of female american black bears (ursus americanus) with partially purified porcine zona pellucidae limits cub production. *Reproduction, Fertility and Development*, *19*(5), 617–625.

Lewis, D. L., Breck, S. W., Wilson, K. R., & Webb, C. T. (2014). Modeling black bear population dynamics in a human-dominated stochastic environment. *Ecological Modelling*, *294*, 51–58.

McCarthy, R. J., Levine, S. H., & Reed, J. M. (2013). Estimation of effectiveness of three methods of feral cat population control by use of a simulation model. *Journal of the American Veterinary Medical Association*, *243*(4), 502–511.

Miller, P. S., Boone, J. D., Briggs, J. R., Lawler, D. F., Levy, J. K., Nutter, F. B., Slater, M., & Zawistowski, S. (2014). Simulating free-roaming cat population management options in open demographic environments. *PLoS One*, *9*(11), e113553.

Robertson, S. A. (2008). A review of feral cat control. *Journal of feline medicine and surgery*, *10*(4), 366–375.

第 6 章

種間相互作用があるモデル

> “自然界における一年あたりの苦痛の総量は，まともに考えられる量をはるかに越えている。私がこの一文を考えている瞬間にも，何千もの動物が生きたまま食われているし，恐怖に震えながら命からがら逃げている動物もいるだろうし，身体の内部からいらだたしい寄生虫に徐々にむさぼり食われているものもいる。また，あらゆる種類の何千という動物が餓えや渇きや病気で死につつあるのだ。そうにちがいない。たとえ豊饒のときがあるとしても，それは自動的に個体数の増加につながり，結局は飢餓と悲惨という自然な状態に戻るのである。”
>
> — Richard Dawkins,
> *River Out of Eden: A Darwinian View of Life, 1995*

生物の個体数の制御が難しい理由や，環境主義者などが，野生への介入が危険であるという理由の一つは，生物は様々な相互作用を含む**生態系（ecosystem）**の中に存在していることです。生態系とは一般に，生物と，非生物も含めたそれを取り巻く環境全体との相互作用を含む系を指しますが，生物同士の関係だけを見ても，異なる種の間には複雑な依存関係が存在します。そのため，ある生物種の数を抑制すると，それに依存して生活している別の生物種の数も減ったり，反対に別の種の数が増えてしまったりすることがあり得ます。あるいは，介入により一時的に個体数が変化しても，元の状態に復元してしまうこともあります。この章では，こうした異種間の相互作用をモデルし，その影響をどのように

調べたらよいかを学びます。

6.1 Lotka-Volterra 方程式

6.1.1 捕食者-被食者モデル

ある生物種が，別の種を捕食して生活している状況を考えます。x を食べられる方の種，**被食者（prey）**の個体数，y を食べる方の種，**捕食者（predator）**の個体数とします。

まず，捕食者の影響を反映した被食者側の個体数の変動をモデルしてみましょう。Malthus モデルを思い出すと，個体数の変化率は，出生率と死亡率を合わせた正味の成長率によって決まるのでした。今はその成長率を a と記すことにしましょう（今まで r で表してきた量です）。今度は，これら自然な出生と死亡に加えて，捕食されることによる死亡項を加えましょう。単純に頻度で考えると，それは個体数 x だけでなく，捕食者の数 y にも比例するでしょうから，比例係数を b として $-bxy$ と置けます。すると，x の時間変化は

$$\frac{dx}{dt} = ax - bxy \tag{6.1}$$

となります。b は捕食の影響の強さを調整するパラメータになります。

他方，捕食者の場合，死亡率ではなく増加率の方が被食者の数に依存します。今考えているような単純なモデルでは，被食者が絶滅したら，捕食者も食料がなく絶滅しますから，その出生率は x に依存するはずです。よって，捕食者の成長率は c を適当なパラメータとして，cx と置いてみましょう。これに，死亡率 d による自然な減少の影響を加えると

$$\frac{dy}{dt} = cxy - dy \tag{6.2}$$

となります。このように，二種の捕食-被食の関係を含んだ最も単純なモデルを記述する式 (6.1) と (6.2) は，合わせて **Lotka-Volterra 方程式（Lotka-Volterra equations）**と呼ばれます。Lotka-Volterra 方程式は，なんだか文字がたくさんあってわかりづらいという人は，いちど係数を全部 1 にして

$$\begin{cases} \dfrac{dx}{dt} = x - xy \\ \dfrac{dy}{dt} = xy - y \end{cases} \tag{6.3}$$

という形式で眺めていると構造がわかりやすくなるでしょう。この場合，一つ目の式は，被食者の個体数 x が成長率 1 で成長しつつ，捕食者の割合に比例して捕食されることを表しており，二つ目の式は，捕食者の個体数が，捕食の割合だけ増加しつつ，死亡率 1 で減少することを表しています。実際には，被食者の成長率（a），捕食者と被食者が出合い捕食が起こる割合（b），捕食が捕食者の成長率に与える影響（c），そして捕食者の死亡率（d）は様々なので，一般的な場合を考えると (6.1) および (6.2) になります。

6.1.2 アイソクライン

Lotka-Volterra 方程式は，二つの変数が絡みあった複雑な非線形微分方程式の組で，解析的な解を求めることはできません（A.1.3 を参照してください）。しかし，いくつかの可視化のテクニックを利用することで，それが記述する系の振る舞いをよく理解することができます。

被食者と捕食者の個体数の組み合わせが，私たちの考えている系の状態を表します。そしてこの状態は，(x, y) を座標とする抽象的な空間を，Lotka-Volterra 方程式に従って動き回る点として捉えることができます。まず，ロジスティック方程式のところで導入した，時間変化しない解である平衡点を可視化しましょう。ここでも時間微分をゼロと置くことで平衡点が見つかりますが，Lotka-Volterra 方程式は二つの変数を含むので，平衡点も二次元空間（平面）内の一点になります。$dx/dt = 0$ となる点は，$x = 0$ か

$$y = a/b \tag{6.4}$$

を満たす点になります。同様に，$dy/dt = 0$ となるのは，$y = 0$ か

$$x = d/c \tag{6.5}$$

となる点です。したがって，両方の条件を同時に満たす平衡点は，$(0,0)$ と $(d/c, a/b)$ です。両方とも個体数がゼロの点 $(0,0)$ は，両種とも絶滅した状態に対応し，両者とも個体数がゼロでない平衡点は**共存平衡点（coexistence equilibrium point）**と呼ばれます。

関数の傾きが一定である点を結んだ線を**アイソクライン（isocline）**といい，特にその傾きの値がゼロになる線を**ヌルクライン（nullcline）**といいます。今

考えている系の個体数が 0 ではない領域でのヌルクラインは，$x = d/c$ を満たす y 方向の直線と，$y = a/b$ を満たす x 方向の直線となり，これらの交点が共存平衡点になります。

直線は，D.1.2 で説明している `axhline` や `axvline` を使えば引けるのですが，後の拡張を考えて，ちょっと面倒ですが以下のようにします：

```
a = 1.2
b = 0.3
c = 0.5
d = 1.5

n_array = np.linspace(0, 10, 20)
isocline_y = np.full_like(n_array, a/b) # dx/dt = 0
isocline_x = np.full_like(n_array, d/c) # dy/dt = 0

plt.figure(figsize=(6, 4))
plt.plot(isocline_x, n_array, label='dy/dt=0')
plt.plot(n_array, isocline_y, label='dx/dt=0')
plt.show()
```

サンプルコード 6.1　アイソクラインの描画スクリプト。

最初に適当な範囲（0–10）を適当な数（20）で割った配列を `np.linspace` で作り，それと同じサイズで一定の値（a/b と d/c）を持つ配列を `np.full_like` を使って作成しています。

実行結果は，図 6.1 のようになります。改めて，これらの直線の交点が，二種が共存する平衡点を示します。上のモデルの構造から，被食者の数 x が増えるほど，捕食者の成長率 cx は増加しますから，グラフの右のほうでは点は上方向に動くことが想像できます。しかし，捕食者の数 y が増えすぎると，被食者の死亡率が成長率を上回り，被食者の数 x が減少し始めるので，右上の領域では，点の動きが左方向に転換することが想像できます。このように，x と y は互いに影響しあうため，(x, y) の組に対応する平面上の点は，一方向に移動するのではなく，向きを変えながら移動します。ヌルクラインはそれを境に変化率の符号が変わる線なので，これらの線を境に，向きを転換しながら，共存平衡点の周りで反時計回りの循環が起きることが予測できます。

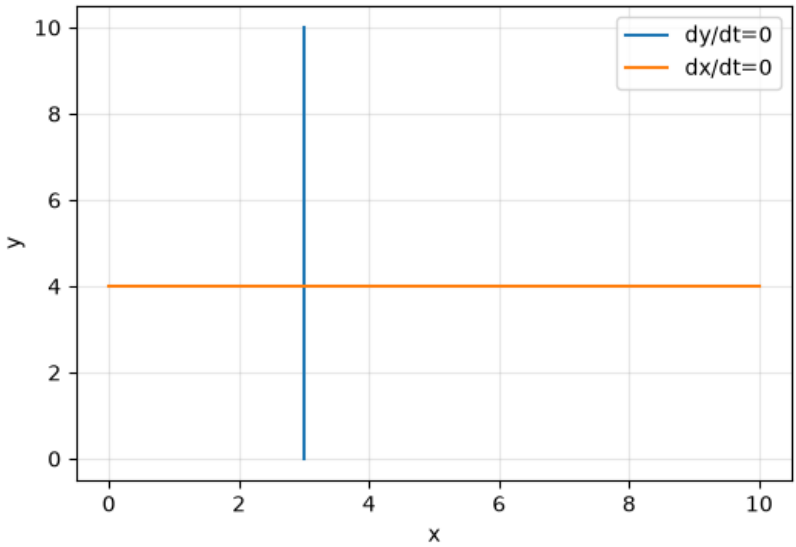


図 6.1 サンプルコード 6.1 の実行例。一部，サンプルコードでは省略している図の装飾などを使用しています。

6.1.3 ベクトル場

ヌルクラインを使うことで，平衡点を視覚的に示すとともに，解の大まかな動きを予測することができました。ただ，これらの線だけでは細かな運動の様子はわかりませんので，より詳細な情報を図示していきましょう。

点の移動は，$dx/dt > 0$ なら右方向，$dx/dt < 0$ ならば左方向になります。同時に，$dy/dt > 0$ なら上方向に，$dy/dt < 0$ ならば下方向に移動します。よって，適当な点に対応する値の組 (x, y) を式に代入した時，dx/dt と dy/dt の符号により，どの方向に動いていくかがわかります。また，dx/dt と dy/dt の大きさが，瞬間的な移動の速さに対応します。したがって，各点での点の移動の様子は，x 方向と y 方向にそれぞれ dx/dt と dy/dt の長さを持つ矢印を用いて可視化することができます。

矢印を指定する値の組は，第 5 章ですでに導入したベクトルとして

$$\boldsymbol{v} = \begin{pmatrix} \dfrac{dx}{dt} \\ \dfrac{dy}{dt} \end{pmatrix} \tag{6.6}$$

と定義できます。ベクトルはしばしば，大きさだけでなく方向を持つものと説明され，矢印に対応付けられますが，今説明した用法がその理由です。またこのことから，ベクトルは $\vec{v}$ のように頭に矢印をつけて記載されることも多いです。しかしここでは，もう一つの一般的な慣例に従い，ベクトルを太字で表すことにします。またここでは，行ベクトルと列ベクトルの区別は必要ないので，成分で表すとき，同じものを横に並べて書いたりもします。一般に，ベクトル $\boldsymbol{v} = (v_x, v_y)$ に対応する矢印の長さは，三平方の定理から

$$v = |\boldsymbol{v}| = \sqrt{v_x^2 + v_y^2} \tag{6.7}$$

で与えられます。数学的にはこれを，ベクトルの**ノルム（norm）**と呼びます。また，ベクトルをそのノルムで割り

$$\frac{\boldsymbol{v}}{v} = \frac{\boldsymbol{v}}{\sqrt{v_x^2 + v_y^2}} \tag{6.8}$$

とすることで，対応する矢印の長さを 1 に揃える（規格化する）ことができます。

ベクトル (6.6) を特定の点でのベクトルではなく，空間座標が与えられたらそこに対応するベクトルを割り当てる関数 $\boldsymbol{v}(x, y)$ とみなし，各点に対応する矢印を割り振っていくと，まるで麦畑（wheat field）のいたるところに麦が生えているように，平面のいたるところに矢印が書き込まれます。このように，空間の各点にベクトルを割り当てる関数は，**ベクトル場（vector field）**と呼ばれます。

6.1.4　ベクトル場の可視化

では，ベクトル場 $\boldsymbol{v}(x, y)$ を描画してみましょう。これには，`matplotlib.pyplot` の `quiver` を使います。矢印がきれいに図に収まるように，(6.8) により長さを 1 に規格化して描画します[*1]。パラメータ a から d の設定は省略します。

[*1] |quiver|では，長さの基準は自動で調整されますので，この場合の規格化はむしろ，すべての矢印の長さを揃え，向きの情報だけを視覚化するために行います。

```
import numpy as np
import matplotlib.pyplot as plt

# set parameters
 （略）

# grid points
x_max, y_max = 10, 10
x = np.linspace(0.1, x_max, 20)
y = np.linspace(0.1, y_max, 20)
X, Y = np.meshgrid(x, y)

# Lotka-Volterra equations
vx = a * X - b * X * Y  # dx/dt
vy = c * X * Y - d * Y  # dy/dt

# normalize the vector
norm = np.sqrt(vx**2+vy**2)
vx_norm = vx / norm
vy_norm = vy / norm

# plot the vector field
plt.quiver(X, Y, vx_norm, vy_norm, color='lightgray')
plt.xlim(-0.5, x_max)
plt.ylim(-0.5, y_max)
plt.show()
```

サンプルコード 6.2　ベクトル場の描画スクリプト。

初めに，図を描く範囲を配列 x と y で指定し，meshgrid を使って，それらから矢印の始点となる格子点 X と Y を作っています。ベクトル場は文字通り空間の至る所に（つまり無限個の）ベクトルを割り当てるのですが，描画の際にはこのように格子点の数で，図示する矢印の数を調整します。次に，その格子点ごとの，ベクトルの成分 vx と vy を作り，規格化して vx_norm と vy_norm としています。後は，格子点の座標とベクトルの成分を順に quiver の引数に入れるだけで矢印が描画されます。これに，先のヌルクラインのスクリプトを合わせて実行することで，図 6.2 のような図が得られます。

これで，かなり解の運動が視覚的にわかりやすくなりました。上でおおまかに予測した通り，反時計回りの動きが見て取れます。ただ，これでもまだ，平衡点から外れた点から出発した状態が，絶滅に向かうのか，共存平衡点に向かうの

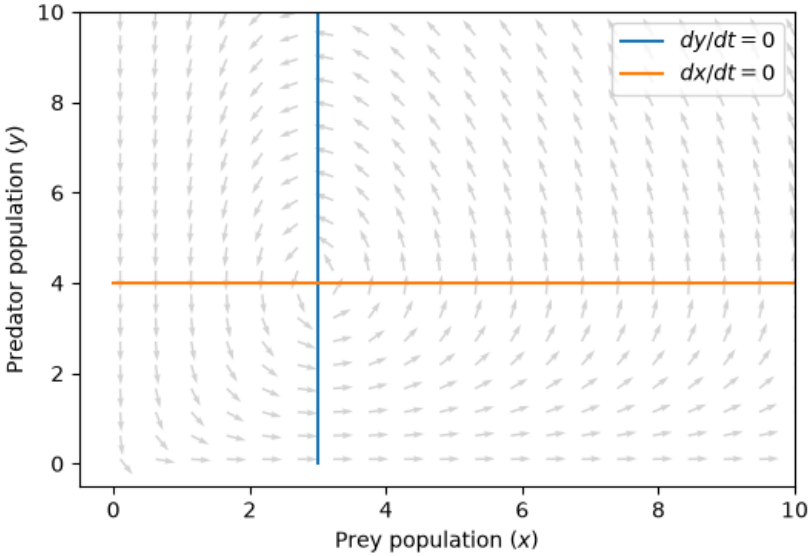


図 6.2　サンプルコード 6.2 の実行例。一部，サンプルコードでは省略している図の装飾などを使用しています。

か，あるいはそのどちらにも向かわず循環を続けるのか，といったことはわかりません。以下で，その部分まではっきりさせられるようにします。

6.2　解の軌道

6.2.1　積分曲線と軌道

図 6.2 における矢印を川の流れに見立てて眺めて見ると，点の動きは，それらをつないでできる曲線に沿って流されていく様子として想像できるかと思います。そのような曲線のことを，ベクトル場の**積分曲線（integral curve）**といいます。そして，積分曲線は方程式 (6.1) および (6.2) にしたがって点がたどる**軌道（orbit）**を表すことになります。

この軌道を調べることで，ある点（特定の状態）から出発する系が，最終的にどこに向かって流されていくのかを明らかにできます。それでは，このような

解の軌道を可視化する方法を見ていきましょう。

6.2.2 軌道計算

軌道は，各時刻での $(x(t), y(t))$ を集めたものなので，原理的には，初期値 $(x(t_0), y(t_0))$ を定めたうえで，Lotka-Volterra 方程式の積分

$$\begin{aligned} x(t) &= x(t_0) + \int_{t_0}^{t} \frac{dx(t)}{dt} dt \\ y(t) &= y(t_0) + \int_{t_0}^{t} \frac{dy(t)}{dt} dt \end{aligned} \tag{6.9}$$

をすれば求められます。これをコード化する戦略は，ループを使って時間を進め，その中で微分方程式部分を計算する関数を呼び

```
# basic parameters
 （略：a--dの定義）

def rate_of_change(x,y):
    dxdt = a * x - b * x * y
    dydt = c * x * y - d * y
    return dxdt, dydt

# 軌道の配列と時間変数の用意
ti = 0
tf = 20
n  = 1000
dt = (tf-ti) / n
x_orb = np.zeros(n)
y_orb = np.zeros(n)

# 初期値
x_orb[0] = 3
y_orb[0] = 3

for i in range(1,n):
    x = x_orb[i-1]
    y = y_orb[i-1]

    dxdt, dydt = rate_of_change(x,y)
     （ここで積分）
```

サンプルコード 6.3　軌道描画コードの基本構造。

とするのがいいでしょう。

問題は積分の部分です。単純に考えると，これを数値的に行うには，積分を有限の時間刻み幅 Δt を含んだ離散和で置き換えて

$$\begin{aligned} x(t+\Delta t) &= x(t) + \frac{dx(t)}{dt}\Delta t \\ y(t+\Delta t) &= y(t) + \frac{dy(t)}{dt}\Delta t \end{aligned} \tag{6.10}$$

とすれば求められそうです。これは，基本的には第 3 章で使ったサンプルコード 3.1 を二変数に拡張するだけで実現できます。ただこの方法には，§ 3.4.3 で説明したような時間刻みに関する問題があります。式 (6.10) をそのまま実装すると，Δt を非常に小さくとならない限り，誤差が蓄積してしまって，正確な軌道が得られなくなるのです。Malthus モデルのところでは解の具体的な値よりも大まかな（定性的な）振る舞いが知りたかっただけなのでよかったのですが，わずかな軌道のずれが結果に大きな影響を与える現在の解析などでは，この方法は十分な方法ではありません。

このような問題に対して用いられる一般的な方法として，以下のような計算法を用います。とりあえず積分部分を以下のように実装してください。詳細は Appendix B.2 で説明していますが，要するに数値的に高精度で計算するためのテクニックなので，必ずしも理由を理解する必要はありません。

```
for i in range(1,n):
    x = x_orb[i-1]
    y = y_orb[i-1]

    k1_x, k1_y=rate_of_change(x, y)
    k2_x, k2_y=rate_of_change(x+0.5*dt*k1_x, y+0.5*dt*k1_y)
    k3_x, k3_y=rate_of_change(x+0.5*dt*k2_x, y+0.5*dt*k2_y)
    k4_x, k4_y=rate_of_change(x+dt*k3_x, y+dt*k3_y)

    x_orb[i] = x + (dt/6.0) * (k1_x+2*k2_x+2*k3_x+k4_x)
    y_orb[i] = y + (dt/6.0) * (k1_y+2*k2_y+2*k3_y+k4_y)

# ついでの描画部分も追加
plt.plot(x_orb, y_orb)
plt.plot(x_orb[0], y_orb[0], marker="o", color="k")
plt.show()
```

その結果，図 6.3 のようなグラフが生成されます。以下の図では，異なる初期

位置（図中の黒点）から始めた結果を重ねて示していますが，上のコードでは適当に初期位置を定めると，そこから出発する，同じような閉じた一つの曲線が描かれるはずです。この結果から，平衡点からずれたどこかの点から出発した状態は，平面をぐるっと回って元の点に戻ることがわかります。理論的にもこうになることが示せます。つまり，この場合の個体数は，それぞれ増えたり減ったりを繰り返すものの，やがてまた元の状態に戻り，決して絶滅することも，定常な共存状態に至ることもありません。

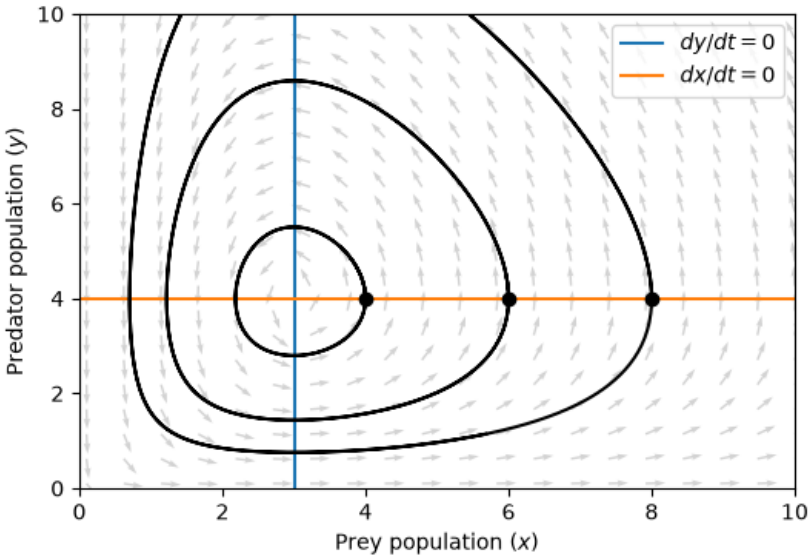


図 6.3 Lotka-Volterra 方程式の解の軌道を表すグラフ。一部，サンプルコードでは省略している図の装飾などを使用しています。

6.2.3 環境収容力の反映

Lotka-Volterra モデルでは，被食者の方は，捕食者の影響がない場合 Malthus 成長する形になっています。しかし，第 4 章で議論したように，密度効果による成長の制限が，非現実的な成長を抑制する上で重要な役割を果たします。そこで，このモデルにも密度効果の影響をを加えていきましょう。

第 4 章で Malthus モデルからロジスティックモデルに変形したときと同じように，成長率に $(1-x/K)$ の因子をかけてみましょう。すると，被食者の個体数の発展方程式は

$$\frac{dx}{dt} = ax\left(1-\frac{x}{K}\right) - bxy \tag{6.11}$$

となります。これに応じて，ヌルクラインは

$$y = \frac{a}{b}\left(1-\frac{x}{K}\right) \tag{6.12}$$

に変更されます。この式は，$y=-\alpha x+\beta$ の形なので，xy 平面上で，傾きが負の直線になります。

変更したモデルでの軌道を描画するには，上で使用したコードを少し修正するだけで OK です。まず，冒頭に環境収容力を

```
    K=10
```

と追加し，ヌルクラインの部分（サンプルコード 6.1）を

```
#isocline_y = np.full_like(n_array, a/b)
isocline_y  = np.full_like(n_array, a/b) - (a/b)*n_array/K
```

と変更します。そして変化率の計算部分（サンプルコード 6.3 とベクトル場計算部分）をそれぞれ

```
def rate_of_change(x,y):
    #dxdt = a * x - b * x * y
    dxdt = a * x * (1 - x / K ) - b * x * y
```

および

```
#vx = a * X - b * X * Y  # dx/dt
vx = a * X * (1 - X / K ) - b * X * Y  # dx/dt
```

とすれば完了です。

結果は，図 6.4 のようになるはずです。密度効果を取り入れた結果，解の軌道は共存平衡点に吸い込まれていくような軌道に変わりました。つまり，この系では，時間がたつと捕食者と被食者の数が自然にバランスして，定常な状態に至ります。

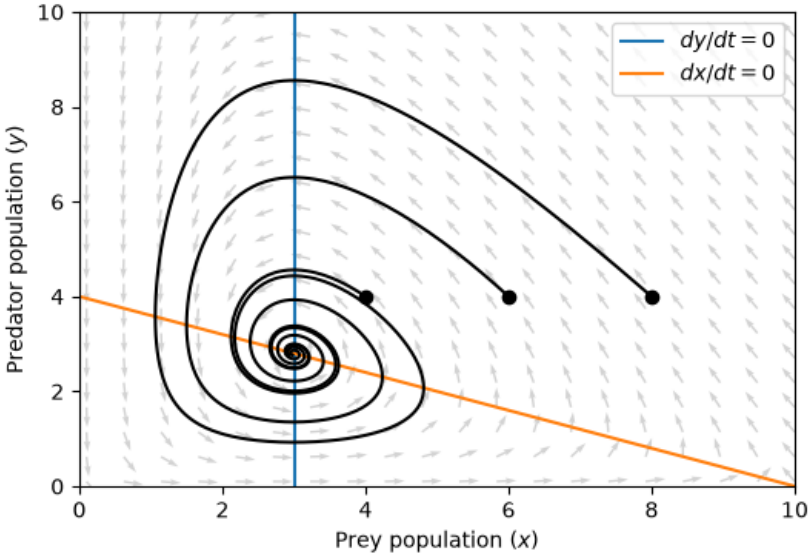


図 6.4　環境収容力を反映した場合の Lotka-Volterra 方程式の解の軌道を表すグラフ。一部，サンプルコードでは省略している図の装飾などを使用しています。

6.3 平衡点の安定性

6.3.1 安定性とは

これまで議論してきたモデルには，確率論的な影響は含まれませんので，平衡点に達したら，そこから変化が生じることはありません。しかし，現実の生態系は，少数の方程式だけで記述できるものではなく，どれだけ理想的な状況でも，様々な変動の影響が加わります。そのため，平衡点に達しても，その周りでわずかな変化が生じた場合，引き続きその平衡点にとどまるとは限りません。この時の振る舞いを決めるのが，平衡点の**安定性（stability）**です。

安定性は，生物の個体数に限らず，様々な物理的対象について議論されます。その概念を説明する際によく使われるのが，図 6.5 のような概念図です。左右の

図は，それぞれ谷と山に置かれた球を図示したもので，異なる性質を持つ平衡点にある状態を表しています。左図のように，谷の底に球がある場合，少し力を加えて位置をずらしても，また元の位置に転がりかえってくることが想像できます。このように，状態をわずかに変化させても変化が増大しなかったり元に戻ったりするような平衡点は，**安定（stable）**な平衡点であるといわれます。他方，右図のように山の頂点にある球は，わずかでも左右に動かすと，そのまま転げ落ちて平衡点からどんどん離れて行ってしまいます。このような平衡点は，**不安定（unstable）**であるといわれます。



図 6.5　谷（左）と山（右）に置かれた球で，平衡点の安定性を示す概念図。

例えば，図 6.5 の地形を，地球環境の状態を指定するパラメータとすると，大きく見れば，地球の環境は過去 1 万年ほど一種の安定な平衡点の周辺に維持されてきたとみなせるでしょう (Steffen et al., 2018)。環境主義者が恐れているのは，たまたま収まっているこの平衡点から脱することです。一方で，環境主義に批判的な人たちが雑に主張するのが，環境は絶えず変化してきただろうということです。確かにこれは事実です。平衡点は無数になり，ある平衡点を脱してもまた別の平衡点に落ち着きます。地球の生態系もこれを繰り返してきました。しかし，これも初めから要点を外した指摘です[*2]。

何より彼らの両方が無視しているのが，その生態系に含まれる動物たちの苦しみです。安定な平衡点にあるからといって苦しみが少ないということにはなりません。むしろ，持続的に苦しみが生産される続ける状態ともいえます。とはいえ，反対にそこから脱したからと言って苦しみが終わるわけでもありません。状態変化の過程でも多くの苦しみが生み出されるし，一つの平衡点を脱し

[*2] 環境主義者は，環境変化が何かしらの理由で問題であると主張するわけですが，地球環境が絶えず様々な時間スケールで変動してきたという純粋な事実を指摘したところで，環境主義者の主張に反論する形にはなりません。

ても，別の平衡点に収まることが予測できます。

実際の環境は無数のパラメータで決定されるため，平衡点は一次元ではなく，大きな次元を持つ空間内の一点になります。私たちが目指すべきは，できるだけ苦しみの少ない経路をたどりながら，最終的に生物の個体数がゼロとなる平衡点に生態系を移行させることです。このような変化を詳細に予測し，実現することは不可能ですが，それでもシミュレーションによる解析は，闇雲に取り組むよりも不確実性を減らし，妥当なシナリオを絞り込む上で重要な役割を果たします。

6.3.2 安定性の計算法

平衡点周辺での軌道の性質を調べることで，平衡点の安定性を確かめることができます。最初の例として，§ 4.2 で扱ったロジスティック方程式に Allee 効果を加えたモデルを考えましょう。式 (4.12) を再掲すると

$$\frac{dN}{dt} = rN\left(\frac{N}{a} - 1\right)\left(1 - \frac{N}{K}\right) \tag{4.12}$$

となります。ここで，$r > 0$, $K > a > 0$ です。平衡点の周りだけを調べるので，個体数 N を，平衡点 N_0 とそこからのわずかなずれ δN の和として，$N = N_0 + \delta N$ と置きます。このずれを表す部分は，形式的には**摂動（perturbation）**と呼ばれるもので，今は平衡点の周辺を調べるので，非常に小さな量であると仮定します。

平衡点の安定性の定義を思い出すと，平衡点からのずれ δN が，時間が経っても増大しないのならば，その平衡点は安定です。特に，時間とともに平衡点に収束していくならば，その平衡点は**漸近安定（asymptotically stable）**と言われます。反対に，ずれが生じたとき，それが時間とともに増大していくならば，その平衡点は不安定です。ですから，これらのずれの時間発展の性質を調べれば，平衡点の安定性が分かります (山本 & 中村, 1998)。

展開 $N = N_0 + \delta N$ を用いると，式 (4.12) は

$$\frac{d(N_0 + \delta N)}{dt} = r(N_0 + \delta N)\left(\frac{N_0 + \delta N}{a} - 1\right)\left(1 - \frac{N_0 + \delta N}{K}\right) \tag{6.13}$$

となります。この式を整理しますが，このとき，δN は微小量であるから，その二次以上の項（$(\delta N)^2$ や $(\delta N)^3$ など）は無視できるとし，δN の 1 次の項までを

残します。すると

$$\begin{aligned}\frac{d(N_0+\delta N)}{dt} =& rN_0\left(\frac{N_0}{a}-1\right)\left(1-\frac{N_0}{K}\right)\\ &+r\delta N\left(-\frac{r}{aK}N_0^3+\frac{r(a+K)}{aK}N_0^2-rN_0\right)\end{aligned} \tag{6.14}$$

が得られます。ここで，平衡点は

$$\frac{dN_0}{dt}=rN_0\left(\frac{N_0}{a}-1\right)\left(1-\frac{N_0}{K}\right)=0 \tag{6.15}$$

を満たすこと利用して，対応する部分を消去すると

$$\frac{d\delta N}{dt}=r\delta N\left(-\frac{r}{aK}N_0^3+\frac{r(a+K)}{aK}N_0^2-1\right) \tag{6.16}$$

が残ります。このように，非線形な方程式を，平衡点などの基準点からのずれの 1 次の式に近似する手続きのことを**線形化（linearization）**といいます。

線形化された式 (6.16) の右辺は，δN の係数をまとめて

$$A\equiv r\left(-\frac{r}{aK}N_0^3+\frac{r(a+K)}{aK}N_0^2-1\right) \tag{6.17}$$

とすると

$$\frac{d}{dt}\delta N=A\delta N \tag{6.18}$$

と表せます。これは Malthus 方程式そのものです。つまり，ずれ δx が増大していくか減衰していくかを調べ，平衡点の安定性を確かめるという目的は，Malthus 方程式 (6.18) の成長率 A の符号を調べることで達成できることが分かります。

実際に, 式 (4.12) の平衡点 $N=0,K$ および a を代入してみると（$r>0, K>a>0$ に注意して）

$$A(N=0)=-r<0 \tag{6.19}$$

$$A(N=K)=-\frac{r(K-a)}{a}<0 \tag{6.20}$$

$$A(N=a)=\frac{r(K-a)}{K}>0 \tag{6.21}$$

となり，前者の二つは時間とともにずれが減衰するので安定なのに対し，Allee 閾値と等しい場合はずれが成長するため，不安定であることが分かります。

6.3.3 Lotka-Volterra 方程式の安定性

今，1 変数の方程式について行った解析を，2 変数の Lotka-Volterra 方程式についても適用してみましょう。被食者と捕食者の個体数 (x, y) を，共存平衡点における個体数 (x_0, y_0) と，そこからの微小なずれ $(\delta x, \delta y)$ を用いて

$$\begin{aligned} x &= x_0 + \delta x, \\ y &= y_0 + \delta y \end{aligned} \tag{6.22}$$

とします。

表現 (6.22) を用いて Lotka-Volterra 方程式 (6.1) および (6.2) を表すと

$$\frac{d}{dt}(x_0 + \delta x) = a(x_0 + \delta x) - b(x_0 + \delta x)(y_0 + \delta y) \tag{6.23}$$

$$\frac{d}{dt}(y_0 + \delta y) = c(x_0 + \delta x)(y_0 + \delta y) - d(y_0 + \delta y) \tag{6.24}$$

となります。ここでも，δx と δy の二次以上の項（δx^2 や $\delta x \delta y$ など）は無視できるとします。そして，定義より平衡点では

$$\left.\frac{dx}{dt}\right|_{x=x_0, y=y_0} = ax_0 - bx_0 y_0 = 0 \tag{6.25}$$

$$\left.\frac{dy}{dt}\right|_{x=x_0, y=y_0} = cx_0 y_0 - dy_0 = 0 \tag{6.26}$$

であることから，対応する部分を消去することで

$$\frac{d}{dt}\delta x = a\delta x - b(x_0 \delta y + y_0 \delta x) \tag{6.27}$$

$$\frac{d}{dt}\delta y = c(x_0 \delta y + y_0 \delta x) - d\delta y \tag{6.28}$$

が得られます。これらの式は第 5 章で扱った行列を用いると

$$\begin{pmatrix} \frac{d}{dt}\delta x \\ \frac{d}{dt}\delta y \end{pmatrix} = \begin{pmatrix} a - by_0 & -bx_0 \\ cy_0 & cx_0 - d \end{pmatrix} \begin{pmatrix} \delta x \\ \delta y \end{pmatrix} \tag{6.29}$$

と行列方程式で表すことができます。この式における変換行列を，**Jacobi 行列（Jacobian matrix）**と呼びます。

1 変数の問題では，線形化した方程式の成長率が正ならば時間とともに成長，負ならば減少するというものでした。また，個体数の分布を導入した推移行列モデルでは，成長率に対応するものとして，行列の固有値が導入されました。すでに学んだこれらのことから，式 (6.29) の Jacobi 行列の固有値の実部が正であれば平衡点は不安定，負であれば漸近安定と言えます。ただし，Lotka-Volterra 方程式の場合，固有値は純粋な虚数（$z = x + iy$ における実部 x がゼロ）になります（Appendix C.2 で説明した計算の仕方により直接確かめられます）。これは，図 6.3 からも確認したように，Lotka-Volterra 方程式の解が，共存平衡点の周りで閉じた軌道を描くことに対応し，共存平衡点からわずかにずれた点から出発する軌道についても全く同じであることを意味しています。このように，安定だが漸近安定ではない平衡点は，**中立安定（neutrally stable）**と言われることがあります。

環境収容力を導入した場合，図 6.4 のように，解の軌道は共存平衡点に吸い込まれていくような軌道になりました。つまり，この場合の共存平衡点は，漸近安定な平衡点であることが予測されます。これを数値的に確かめてみましょう。

練習問題 6.1. 式 (6.11) を線形化して式 (6.29) に対応する式を作り，Jacobi 行列を求めてください。回答は以下のコード内で確認できます。

環境収容力を反映した場合の Jacobi 行列の固有値を計算するために，コードでパラメータ a から d と K を定義する部分よりも下に，以下のコードを追加して実行してみてください。これにより，固有値の実部が負になることが確かめられます。環境収容力を含まない元の Lotka-Volterra 方程式の場合を調べたければ，K を含む因子を消すか，あるいは K を極端に大きくして計算してみてください。このように，解の具体的な軌道ではなく，安定性だけが知りたい場合には，非線形な元の方程式ではなく，線形化した式の性質を調べるだけで十分になります。

```
x0 = d / c
y0 = (a / b) * (1 - x0 / K)

J11 = a * ( 1 - 2 * x0 / K ) - b * y0
J12 = -b * x0
J21 = c * y0
J22 = c * x0 - d
```

```
eigenvalues = np.linalg.eigvals([[J11, J12], [J21, J22]])

print(f"(x0, y0) = ({x0:.4f}, {y0:.4f})")
print(f"Eigen values: {eigenvalues[0]:.4f}, {eigenvalues[1]:.4
  f}")
```

6.4 Rosenzweig-MacArthur モデル

6.4.1 機能的応答

Lotka-Volterra 方程式では，捕食者の成長率に対応する量は，単純に被食者の数に比例する形 cx になっています。よって，被食者の数が 100 倍になれば，成長率も 100 倍になります。これは明らかに非現実的です。そこで，捕食効率の限界に関連する制限を考えます。

これまで，被食者が単位時間あたりに捕食者一体に食べられる数を bx としてきました。これを抽象的な関数 $f(x)$ に置き換えると，方程式は

$$\frac{dx}{dt} = ax\left(1 - \frac{x}{K}\right) - yf(x) \tag{6.30}$$

$$\frac{dy}{dt} = kyf(x) - dy \tag{6.31}$$

と書けます。捕食者の成長率も，捕食率に比例すると仮定し，新たな定数 k を用いて $kyf(x)$ としています。元のように $f(x) = bx$ だった場合，$k = c/b$ という関係になります。この関係からもわかりやすいように，k は捕食に対し，成長率がどれだけ効率的に上がるかに対応する係数となっています。

関数 $f(x)$ は，被食者の密度（あるいは個体数 x）に対し，一体の捕食者が単位時間あたりに被食者を捕食する数がどう変化するかを表しており，**機能的応答（functional response）**と呼ばれます。具体的な振る舞いは捕食者と被食者の性質によるので，機能的応答には様々な形がありえます。元の bx という形も，I 型と呼ばれる機能的応答の一つです。別の代表的な形の一つが II 型と呼ばれるもので，被食者が増えても，一個体の捕食者が単位時間あたりに食べる量には上限があるため，捕食速度が次第に頭打ちになる（飽和する）形

$$f(x) = \frac{bx}{1 + hx} \tag{6.32}$$

です。この式は **Holling の円盤方程式（Holling's disc equation）**と呼ばれます。あるいはこれは，酵素の反応速度を表す **Michaelis-Menten 方程式（Michaelis-Menten equation）**として知られる方程式と同じ形をしているので，Michaelis-Menten 型とも呼ばれます。

h は，獲物を見つける効率と食べるのにかかる時間を決めるパラメータです。パラメータ b がその探索効率を表しており，τ のように食べる時間に対応するパラメータを別に定義し，$h = b\tau$ の形に書かれることも一般的ですが，ここでは簡潔化のために τ は用いません。$h = 0$ とすれば，捕食速度に上限のない元の式 $f(x) = bx$ に戻ります。反対に hx を大きくしていくと，分母においてその項が支配的になり

$$f(x) = \frac{bx}{1+hx} \simeq \frac{bx}{hx} = \frac{b}{h} \tag{6.33}$$

のように，被食者の数 x によらない形になります。この II 型の機能的応答を用いると，式は

$$\frac{dx}{dt} = ax\left(1 - \frac{x}{K}\right) - \frac{bxy}{1+hx} \tag{6.34}$$

$$\frac{dy}{dt} = \frac{kbxy}{1+hx} - dy \tag{6.35}$$

となります。このモデルを **Rosenzweig-MacArthur モデル（Rosenzweig-MacArthur model）**と呼びます。

このとき，$dx/dt = 0$ となるヌルクラインは，$x = 0$ と

$$y = \frac{a}{b}\left[1 + \left(h - \frac{1}{K}\right)x - \frac{h}{K}x^2\right] \tag{6.36}$$

のように，x についての二次関数になります。$dy/dt = 0$ のヌルクラインは，$y = 0$ と

$$x = \frac{d}{kb - hd} \tag{6.37}$$

という直線のままです。

6.4.2　富栄養化のパラドクス

Rosenzweig-MacArthur 型の方程式 (6.34) および (6.35) を描画してみましょう。$k = c/b$ とします。ここでも必要な変更は，新しくパラメータ h を追加し，対応する方程式部分とヌルクライン部分を

```
def rate_of_change(x, y):
    dxdt = a * x * ( 1 - x / K ) - b * x * y  / ( 1 + h * x )
    dydt = c * x * y / ( 1 + h * x ) - d * y
    return dxdt, dydt

vx = a * X * (1 - X / K ) - b * X * Y / ( 1 + h * X )  # dx/dt
vy = c * X * Y / (1 + h * X) - d * Y  # dy/dt

isocline_y = np.full_like(n_array, a/b) + (a/b) * ( ( h - 1 /
  K) * n_array - ( h / K ) * n_array**2 )
isocline_x = np.full_like(n_array, d / ( c - h * d ) )
```

に変更するだけです。

ここでは，被食者の環境収容力を上げ，$K = 15$ として計算してみます。その他のパラメータは $a = 1.2$，$b = 0.3$，$c = 0.5$，$d = 0.5$，$h = 0.2$ とします。被食者の環境収容力を上げると，維持できる被食者の数が増えるので，捕食者からしてもエサが増えることが予測されます。しかし，結果は図 6.6 のようになります。

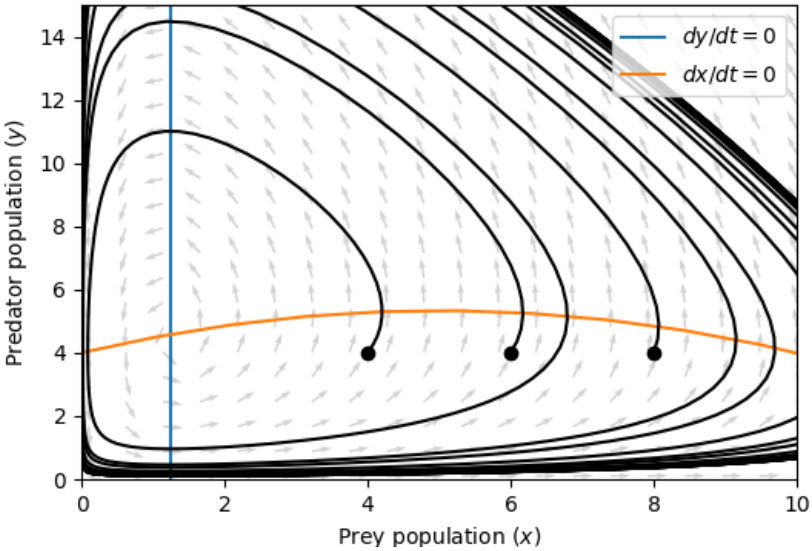


図 6.6　一部，サンプルコードでは省略している図の装飾などを使用しています。$a = 1.2, b = 0.3$，$c = 0.5, d = 0.5$，$K = 15$，$h = 0.2$ にした場合の図です。

図 6.6 からは，環境収容力だけを導入した場合とは反対に，機能的応答の影響は，軌道が共存平衡点から離れていく方向に仕向けることが見て取れます。そのため，軌道が始点に戻ることなく，より外側を通って周回し，結果的に絶滅のリスクが増大しています[*3]。予想に反するこの結果の原因は，h の導入により捕食の速度に限界が与えられたことです。被食者がどんどん増えても，捕食がそれに追いつかず，被食者の個体数 x は大きく成長します（軌道がグラフの右側に大きく振れるのに対応します）。しかし，それに遅れて捕食者の個体数 y が増加してくると，今度はたくさんの捕食者が大量に被食者を消費し始めます（軌道がグラフの左側に向かって急カーブします）。こうして，軌道のふり幅が大きくなり，系全体が不安定化するのです。

このように，被食者が利用できる栄養を増やし，環境収容力 K を大きくすると，逆に系が不安定化し，絶滅のリスクが高まる現象を，**富栄養化のパラドクス（paradox of enrichment）**と呼びます。これは単純な実験環境下でも実際に観察されていますが，もっと複雑な自然界では，ここまでわかりやすい形ではほとんど観察されるものではないようです (Vandermeer & Goldberg, 2007)。それでも，§ 4.3 で説明した理想的なモデルの役割と同様に，理想的に近い条件下で起こる現象というものを理解していくことには意味があります。

6.5　中位捕食者解放効果

本章ではこれまで，複数種の相互作用を表すパラメータの設定次第で，個体数変動が複雑に変化することを見てきました。現実の生態系は，これらの議論で用いた基本的な数理モデルよりはるかに複雑であることから，実際の個体数制御の取り組みにおいて，どんな影響が生じるかを正確に予測することがいかに困難なことか想像できるでしょう。実際に人間が生態系に介入して個体数制御を行おうとした結果，反直感的な効果により意図しない影響が生じた例がいくつもあります。その中の代表的な一例が，**中位捕食者解放効果（mesopredator release effect）**と呼ばれるものです。

[*3] 共存平衡点が不安定な場合でも，確率的な影響を含まない今のモデルでは，0 より大きい初期値から始まった個体数が厳密に 0 になるわけではなく，最終的に閉じた軌道に落ち着きます［最終的に収束するこのような閉じた軌道を**リミットサイクル（limit cycle）**と呼びます］。しかし，確率的なゆらぎの影響を含めれば，0 に達する可能性があります。

6.5.1 中位捕食者解放効果とは

ネコが強力な捕食者で，鳥やネズミや昆虫など，多くの動物を殺すことはよく知られています。特に，**外来種（alien species）**として離島に連れ込まれたネコの影響力は大きく，多くの鳥の絶滅やそのリスク増大を招いてきました。こうしたことからも，ネコの個体数制御は重要な問題として取り組まれてきたのですが，その中で，逆説的な効果も観察されてきました。それは，一度導入された後でネコの個体数を減らすと，鳥の個体数の減少が抑止されるどころか，むしろ進んでしまうのです。

この直感に反する結果の原因は，ネズミのような**中位捕食者（mesopredator）**の存在にありました。ネズミはネコという上位の捕食者に対しては被食者ですが，鳥に対しては卵やヒナを中心に狙う捕食者ともなります。このような食物連鎖において中間的な位置にある生物が中位捕食者と呼ばれます。ネコは，鳥に限らず，ネズミを含めた他の様々な動物を捕食するので，ネコがいる環境ではネズミの個体数が低く抑えられるのですが，ネコがいなくなるとこの中位捕食者の影響が高まり，鳥の絶滅が進んでしまうのです。これが，中位捕食者解放効果と呼ばれるものです。

Courchamp et al., 1999 は，ネコと鳥とネズミの三種が相互作用する系を考え，この効果を数理的に示しました。このモデルはこれまで私たちが学んだモデルの延長として構成できるため，ここで彼らの結果を再現してみましょう。

6.5.2 二種モデル

まず，鳥とネズミだけの場合と，鳥とネコだけの場合のモデルを考え，後から三種目を加えるようにしましょう。基本となる方程式は，成長率をロジスティック方程式に除去項 P を加えた

$$\frac{dN}{dt} = rN\left(1 - \frac{N}{K}\right) - P \tag{6.38}$$

になります。ここで，N は個体数，r は成長率，K は環境収容力です。下で方程式の見た目が少しずつ変わりますが，基本はこの形ですので複雑に考えすぎないようにしてください。

初めに，鳥とネコの系を考えます。鳥とネコの個体数をそれぞれ，B と C とし，それぞれの成長率を r_b と r_c とします。また，鳥の環境収容力を K_b，ネコが鳥を捕食する割合を μ_b とします。これらを (6.38) に入れれば，鳥の個体数の式が得られます。ネコの環境収容力は，§ 4.1.4 で扱ったケースのように，純粋に得られるエサの量で決まるとします。つまり，鳥の個体数 B をネコ一体あたりが食べる量 μ_b で割った $K_c = B/\mu_b$ をネコの環境収容力とします。ネコは上位捕食者なので，捕食による減少項はありません。よって，この系の発展方程式は

$$\begin{cases} \dfrac{dB}{dt} = r_b B\left(1 - \dfrac{B}{K_b}\right) - \mu_b C \\ \dfrac{dC}{dt} = r_c C\left(1 - \dfrac{\mu_b C}{B}\right) \end{cases} \tag{6.39}$$

で与えられます。

続いて鳥とネズミの系を考えます。捕食者がネコからネズミに変わるだけなので，基本的には同じ形の式で書けます。ただ，ここではネズミは鳥以外のエサも食べることを考慮に入れます。鳥以外のすべてのエサを S とし，ネズミはその割合に応じてエサを食べ分けるとすると，鳥がネズミに捕食される割合は $B/(S+B)$ と書けます。ネズミ一匹当たりが食べる鳥とそれ以外のエサの量をそれぞれ η_b と η_s とすると，ネズミの環境収容力 K_r は，$S/\eta_s + B/\eta_b$ で与えられます。式に現れるのはこの逆数 $1/K_r = 1/(S/\eta_s + B/\eta_b) = \eta_b\eta_s/(\eta_b S + \eta_s B)$ であることに注意し，ネズミの個体数と成長率を R および r_r とすると

$$\begin{cases} \dfrac{dB}{dt} = r_b B\left(1 - \dfrac{B}{K_b}\right) - \dfrac{B}{S+B}\eta_b R \\ \dfrac{dR}{dt} = r_r R\left(1 - \dfrac{\eta_b\eta_s R}{\eta_b S + \eta_s B}\right) \end{cases} \tag{6.40}$$

となります。

6.5.3　三種モデル

鳥とネコの系の式と鳥とネズミの系の式が，それぞれ (6.39) と (6.40) のように得られたので，これらを基に，鳥とネコとネズミの三種からなる系の式を構成します。ここでも，上の式を導いたのと同じように，捕食者は複数のエサの種類がある場合，そのエサの割合に応じて捕食することを仮定します。結果を先に

示すと，次のようになります。

$$\begin{cases} \dfrac{dB}{dt} = r_b B\left(1-\dfrac{B}{K_b}\right) - \dfrac{B}{S+B}\eta_b R - \dfrac{B}{B+R}\mu_b C \\ \dfrac{dR}{dt} = r_r R\left(1-\dfrac{\eta_b\eta_s R}{\eta_b S+\eta_s B}\right) - \dfrac{R}{B+R}\mu_r C \\ \dfrac{dC}{dt} = r_c C\left(1-\dfrac{\mu_b\mu_r C}{\mu_r B+\mu_b R}\right) \end{cases} \tag{6.41}$$

順番に意味を説明していきます。鳥の場合は単純です。鳥とネズミの系の式 (6.40) と比べると，ネコによる捕食の影響が加わるので，対応する項 $-B/(B+R)\mu_b C$ が加わっただけの形になっています。あるいは (6.39) を基準に考えると，捕食項 $-\mu_b C$ が，鳥とネズミの両方に分配され，$-B/(B+R)\mu_b C$ となったと考えることもできます。ネズミは，鳥とネズミの系の式 (6.40) から見ると，ネコにより捕食されることによる減少項 $-R/(B+R)\mu_r C$ が加わっています。ネコの式は，エサが鳥とネズミの二種類になるので，環境収容力が B/μ_b から $B/\mu_b + R/\mu_r$ に変更されています。

この系には，K_b と S および r，η，μ に添え字が付いた多くのパラメータが含まれます。これらのパラメータの組み合わせ（条件）次第で，三種のうちの一種が絶滅する場合もありますし，三種が共存する場合もあります。あるいはネズミだけが絶滅を逃れる場合もあります。そして，ある条件下では，ネズミだけだったら鳥を絶滅に追い込むが，ネコが存在することでそれが防がれるという状況も表れます。

6.5.4 三種共存シミュレーション

これを再現するために，まずは三種が共存するパラメータを用意しましょう。Courchamp et al., 1999 の Fig. 2 を基にパラメータを探ると，表 6.1 のようになります。また，初期の個体数は鳥，ネズミ，ネコでそれぞれ 10^5，10^2，1 としています。これらを用いて三種系の式 (6.41) を解くコード 6.4 を実行すると，図 6.7 のような結果が得られます。ネコとネズミの個体数は 100 倍して描画していることに注意してください。

```
eps = 1e-8

r_b, r_r, r_c = 1.5, 4.0, 0.75
K_b, S = 100000, 10000
```

r_b	r_r	r_c	K_b	S	μ_b	μ_r	η_b	η_s
1.5	4.0	0.75	100000	10000	54	200	120	60

表 6.1　三種が共存するパラメータの組み合わせ。

```
mu_b, mu_r = 54, 200
eta_b, eta_s = 120, 60

B0, R0, C0 = 100000, 100, 1

def func(y):
    B, R, C = y
    dBdt = r_b * B * (1 - B/K_b) - (B/(S + B + eps)) * eta_b *
       R - (B/(B + R + eps)) * mu_b * C
    dRdt = r_r * R * (1 - (eta_b * eta_s * R)/(eta_b * S +
      eta_s * B + eps)) - (R/(B + R + eps)) * mu_r * C
    dCdt = r_c * C * (1 - (mu_b * mu_r * C)/(mu_r * B + mu_b *
       R + eps))
    return np.array([dBdt, dRdt, dCdt])

t_start, t_end = 0, 50
nsteps = 10000
dt = (t_end - t_start) / nsteps
time = np.linspace(t_start, t_end, nsteps)

y_arr = np.zeros((nsteps, 3))
y_arr[0] = np.array([B0, R0, C0])

for i in range(1, nsteps):

    k1 = func(y_arr[i-1])
    k2 = func(y_arr[i-1] + 0.5 * dt * k1)
    k3 = func(y_arr[i-1] + 0.5 * dt * k2)
    k4 = func(y_arr[i-1] + dt * k3)

    y_arr[i] = y_arr[i-1] + (dt / 6.0) * (k1 + 2*k2 + 2*k3 +
      k4)
```

```
plt.plot(time, y_arr[:, 0], 'b-', label='Bird (Prey)')
plt.plot(time, y_arr[:, 1]*100, 'r--', label='Rat (
  Mesopredator) x100')
plt.plot(time, y_arr[:, 2]*100, 'g-.', label='Cat (
  Superpredator)x100')
```

```
plt.ylabel('Population')
plt.yscale('log')
plt.legend()
plt.grid(True, alpha=0.5)
plt.show()
```

サンプルコード 6.4 三種系の式を解くコード。

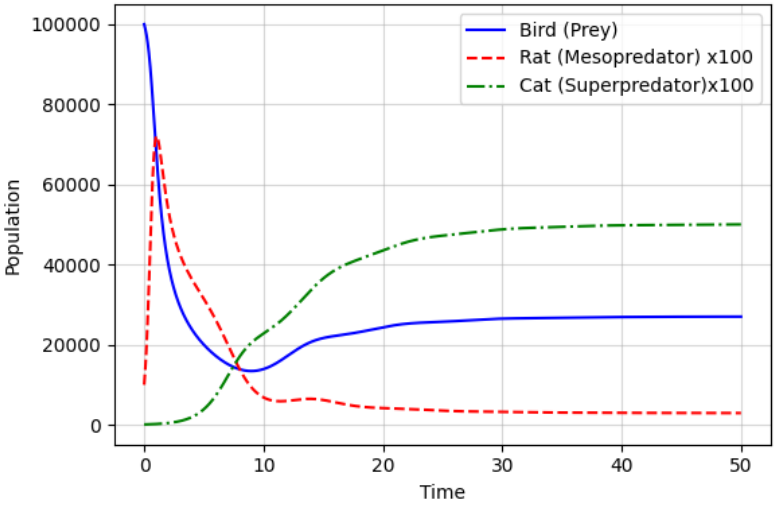


図 6.7 式 (6.41) を解くコード 6.4 の実行結果。青線，赤線，緑線はそれぞれ，鳥，ネズミ，ネコの個体数を表します。ネズミとネコの個体数は 100 倍して描画しています。使用パラメータは表 6.1 を参照してください。

まずこの図からわかることを考察してみましょう。はじめ，10^5 いた鳥（青線）は，ネズミ（赤線）とネコ（緑線）が導入されることで個体数を減らしていきます。導入されたネズミとネコは，最初指数関数的に個体数を増やしていることが見て取れます。そして，繁殖サイクルがより短いネズミの方が，ネコよりも早く増えています。しかし，ネコの個体数が増えてくると，ネズミの成長が止まり，減少に転じます。その後，あるところまでネズミの個体数が減ってきたところで，三種の個体数の変化が小さくなっていき，平衡点に達します。

6.5.5　中位捕食者解放効果シミュレーション

続いて，中位捕食者解放効果を見るため，ネコとネズミの個体数変動の式に，それぞれ除去項 $-\lambda_c C$ と $-\lambda_r R$ を加えます。係数 λ_c と λ_r は，それぞれネコとネズミを取り除く効率を表します。これらを反映し，時刻が t_{cntrl} を過ぎたら，介入によるネコとネズミの除去を有効にするようにし，前段階の計算結果から介入によるどのような変化が起きるかを調べるようにします。そのために，コード内のパラメータ設定の部分を以下のように変更しましょう。

```
eta_b, eta_s = 120, 60
lam_r, lam_c = 0.0, 1.0 # 追加

B0, R0, C0 = 100000, 100, 1
t_start, t_end, t_cntrl = 0, 100, 50 # t_cntrlを追加
```

ネズミの除去率は $\lambda_r = 0$ としています。そして，関数部分に時間を渡し，除去効果を有効にするよう以下のように変更します。

```
def func(y,t):
    B, R, C = y
    dBdt = r_b * B * (1 - B/K_b) - (B/(S + B + eps)) * eta_b *
       R - (B/(B + R + eps)) * mu_b * C
    dRdt = r_r * R * (1 - (eta_b * eta_s * R)/(eta_b * S +
      eta_s * B + eps)) - (R/(B + R + eps)) * mu_r * C
    dCdt = r_c * C * (1 - (mu_b * mu_r * C)/(mu_r * B + mu_b *
       R + eps))
```

```
    if t>t_cntrl:
        dRdt += - lam_r * R
        dCdt += - lam_c * C

    return np.array([dBdt, dRdt, dCdt])

for i in range(1, nsteps):

    t = time[i]
    k1 = func(y_arr[i-1], t)
    k2 = func(y_arr[i-1] + 0.5 * dt * k1, t)
    k3 = func(y_arr[i-1] + 0.5 * dt * k2, t)
    k4 = func(y_arr[i-1] + dt * k3, t)
```

```
        y_arr[i] = y_arr[i-1] + (dt / 6.0) * (k1 + 2*k2 + 2*k3 +
          k4)
```

これを実行した結果が，図 6.8 になります。時間が 50 のところまでは前の結果と同じです。それ以降で，ネコの除去が開始されます。ネコの個体数を急速に減らすと，ネズミの個体数も一気に増加し，それに応じて鳥が絶滅に向かう様子が見て取れると思います。まさしくこれが，中位捕食者解放効果です。

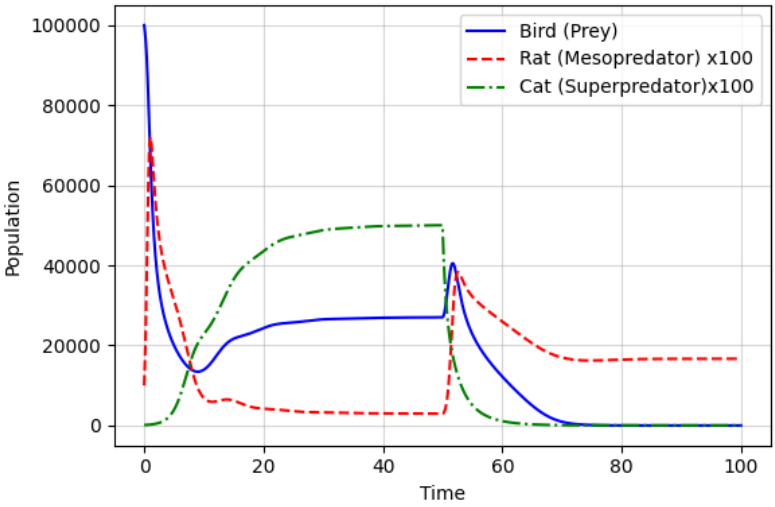


図 6.8 $t_{cntrl} = 50$ および $\lambda_c = 1.0$ として実行した場合の結果。

6.6 不妊虫放飼の競争モデル

数理モデルとコーディングについて，本章で説明すべきことは前の節までででほぼ説明し終わりました。最後に本節では，概念的な補足を兼ねてもう一つだけ問題を考えます。コーディングについては新しいことはないので省略し，簡単な数理モデルとそれを用いたシミュレーション結果の例だけを議論します。ここで扱うのは，**不妊虫放飼法（sterile insect release technique）**や，あるいはより端的に **sterile insect technique（SIT）**と呼ばれる，昆虫の不妊化法です。

6.6.1　不妊虫放飼とは

不妊虫放飼（SIT）は，農薬などに頼らず，人間たちが「害虫」とみなす特定の昆虫の個体数を抑制し，場合によっては根絶を目的として利用されてきた方法の一つです。その基本的な仕組みは単純です。まず，対象となる虫を施設内で人工的に大量増殖させ，放射線を照射して不妊化した個体（主にオス）を作成します。そして，この不妊化された個体を野生の生息地へ放ちます。野外に放たれた不妊オスは，野生のオスと競合しながら野生のメスと交尾を行います。野生オスの個体数を N，放飼する不妊オスの個体数を S とし，メスが野生オスとランダムに交尾すると仮定すると，メスが野生種のオスと交尾できる確率は

$$p = \frac{N}{N+S} \tag{6.42}$$

に低下します (Barclay, 2005)。不妊オスと交尾したメスが産んだ卵は孵化しないため，繁殖につながりません。このように，繁殖につながる有効な交尾率を下げることで，個体数を減らします。もし不妊のオスを継続的に放飼し，S を一定に保ち続けると，野生の個体数 N が減少するにつれて割合 $\frac{N}{N+S}$ がどんどん小さくなるため，SIT は個体数が減るほど効率が上がっていくという強力な性質を持っています。

6.6.2　単一方程式モデル

不妊虫の個体数 S が与えられたとき，野生種の個体数変動がどう影響を受けるかを調べましょう。通常のロジスティック方程式は

$$\frac{dN}{dt} = aN\left(1 - \frac{N}{K}\right) = \underbrace{aN}_{\text{正味の増加}} - \underbrace{a\frac{N^2}{K}}_{\text{過密死}} \tag{6.43}$$

の形で与えられるのでした。過密効果による死亡項 $\propto N^2$ は，個体数 N に比例し，かつ過密の影響によりさらに N がかかって N の 2 次の項になっていると理解できます。この単純なロジスティック方程式を出発点にして，不妊虫の個体数を反映させた式を構成しましょう。

正味の増加項は，「出生」-「死亡」に分けられます。出生項は SIT による繁殖

成功率の低下を考慮に入れると

$$a\frac{N}{N+S}$$

と書けます。死亡率は単純に，比例係数 $b>0$ を用いて $-bN$ とします。過密による死亡項は，野生種自身の個体数 N に比例し，過密効果は全個体数 $N+S$ に比例するとすると，係数を $k>0$ として

$$kN(N+S)$$

とできます。これらをまとめると

$$\frac{dN}{dt}=\left(\frac{aN}{N+S}-b\right)N-kN(N+S) \tag{6.44}$$

が得られます (Murray, 2002)。

6.6.3 競争モデル

野生種の個体数の発展方程式 (6.44) に，不妊種の個体数の発展方程式を加えれば，これまで扱ってきたのと同様の競争モデルを作れます。不妊種の放出の仕方には，実践的に様々な方法があり，それに応じて様々なモデル化が考えられます (Barclay, 2005; Ben Dhahbi et al., 2020)。ここでは単純に，野生種の個体数 N に比例して不妊虫を放出する場合を仮定した形

$$\frac{dS}{dt}=\gamma N-bS \tag{6.45}$$

を用いましょう。$\gamma>0$ は比例係数で，不妊種の死亡率 b は野生種と同じであることを仮定しました。このモデルはまた，人為的な不妊種の放出だけでなく，何らかの理由で野生種から γ の割合で不妊虫で生まれてくるとした場合にも適用できます。

式 (6.44) と (6.45) を，$a=0.8$，$b=0.2$，$k=0.001$，$\gamma=0.8$ とした場合のグラフが，図 6.9 になります。この場合，安定な共存平衡点が存在し，軌道はそこに収束します。対して，他のパラメータは固定し，$\gamma=0.8$ とした場合のグラフが，図 6.10 になります。この場合，共存平衡点はなくなり，すべての軌道は，絶滅に対応する原点 $(0,0)$ に向かいます。

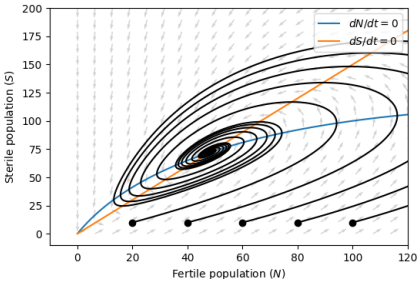


図 6.9　$a = 0.8$，$b = 0.2$，$k = 0.001$，$\gamma = 0.3$ として，式 (6.44) および (6.45) に従う軌道と，ヌルクラインを示したグラフ。

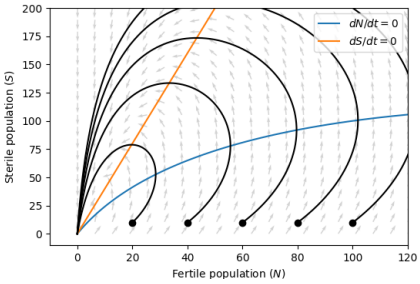


図 6.10　$a = 0.8$，$b = 0.2$，$k = 0.001$，$\gamma = 0.8$ として，式 (6.44) および (6.45) に従う軌道と，ヌルクラインを示したグラフ。

References

Barclay, H. (2005). Mathematical models for the use of sterile insects. In *Sterile insect technique: Principles and practice in area-wide integrated pest management* (pp. 147–174). Springer.

Ben Dhahbi, A., Chargui, Y., Boulaaras, S. M., Ben Khalifa, S., Koko, W., & Alresheedi, F. (2020). Mathematical modelling of the sterile insect technique using different release strategies. *Mathematical Problems in Engineering, 2020*(1), 8896566.

Courchamp, F., Langlais, M., & Sugihara, G. (1999). Cats protecting birds: Modelling the mesopredator release effect. *Journal of Animal Ecology, 68*(2), 282–292.

Murray, J. D. (2002). *Mathematical biology i: An introduction* (3rd, Vol. 17). Springer.

Steffen, W., Rockström, J., Richardson, K., Lenton, T. M., Folke, C., Liverman, D., Summerhayes, C. P., Barnosky, A. D., Cornell, S. E., Crucifix, M., et al. (2018). Trajectories of the earth system in the anthropocene. *Proceedings of the national academy of sciences, 115*(33), 8252–8259.

Vandermeer, J. H., & Goldberg, D. E. (2007). **個体群生態学入門: 生物の人口論** (佐藤一憲, 竹内康博, 宮崎倫子, & 守田智, Trans.) [原著: Population Ecology: First Principles (Princeton University Press, 2003)]. 共立出版.

山本, 義., & 中村, 孔. (1998). **解析力学** *i&ii*. 朝倉書店.

巌佐, 庸. (1998). **数理生物学入門: 生物社会のダイナミックスを探る（改装版）** [原著: HBJ 出版局, 1990 年]. 共立出版.

桑村雅隆. (2015). **パターン形成と分岐理論：自発的パターン発生の力学系入門**. 共立出版.

第 7 章

確率論的記述

> “世界はお前には決して解決できない問題で溢れている。子供たちはお前には予測できない運命を背負ってる。全ての親は罪人だ。有罪を宣告せよ。”
>
> — Mistro,
> *Every Cradle*

これまで扱ってきたモデルはすべて，初期値が与えられれば，時間発展の規則に従って，未来の値が完全に一つに決まるものでした。このような時間発展は，**決定論的（deterministic）**であると言われます。実際，私たちの日常のスケールでは，ものごとは決定論的に進行します。一方で，ものごとの変化には，無数の要因がかかわっており，それらすべてを厳密に考慮することはできません。例えば，エサとなる木の実がたくさん付いたとか付かなかったとかいうのも，個体数の変動には重要な要素となります。生物が空間的に移動している様子を考えたとき，たまたま少し前に獲物となる別の動物が近くを通ったため，それを追いかけて進むとか，たまたま風が吹いて音が鳴り，驚いて進路を変えるとかいうこともあるでしょう。これらを正確に予測してモデルに加えていくことなど不可能です。

そこで，こうした，不確実性の高い要素は，ランダムな変数で置き換えてしまうことが非常に一般的な方法になります。そして，そうしたランダムな変数の値以外は条件を同じにした多数のシミュレーションを実行し，平均やばらつきなど，それらの統計的性質から対象の性質を調べる方法を，**Monte Carlo 法**

（Monte Carlo method）と呼びます。本章では，こうした**確率性（stochasticity）**を取り入れた**確率論的モデル（stochastic model）**と，そのコーディングの方法を学びます。

7.1　ランダムウォーク

7.1.1　拡散係数

ある生物個体の移動を，最も単純な確率モデルで表現してみましょう。毎ステップ，運動がランダムに決まる過程を**ランダムウォーク（random walk）**といいます。簡単のために x 軸に沿った 1 次元の場合を考え，個体は各ステップごとに，正か負の方向（右か左）に Δx だけ移動するとします。つまり，正の方向に動く確率を p_R，負の方向に動く確率を p_L とすると，どちらも同じ値で $p_R = p_L = 0.5$ となります。これらの確率は時間がたっても同じであるとします。

ここで，確率を次のように理解することにします。まず，実在する個体は一体ですが，互いに相互作用しない無数のそっくりの個体の集まりを考えることとします。このような対象の仮想的なコピー集団のことを**アンサンブル（ensemble）**と呼びます。次に，アンサンブルのうち，毎ステップある割合の個体は右に移動し，残りは左に移動します。このとき，アンサンブルの数を無限とみなす極限で，その割合がそれぞれ p_R と $p_L = 1 - p_R$ に収束するとします（ここで $0 \leq p_R, p_L \leq 1$）。ただし，ある個体に着目するとき，その個体が右と左どちらに移動するグループにいるかはわからないものとします。つまり，確率 p_R と p_L は，理想的な条件の下で，ある時間ステップにアンサンブルのうちそれぞれ右と左に移動する個体数の割合を意味します。

初期位置を原点 $x = 0$ にとれば，1 ステップごとの変位 Δx のアンサンブル平均は

$$\langle \Delta x \rangle = p_R \Delta x + (-\Delta x) p_L = 0 \tag{7.1}$$

となります。毎ステップこの平均は変わらないため，任意のステップ数 n を経た後でも，位置 x の平均 $\langle x \rangle$ は 0 のままです。

他方，負の量でも二乗した値は正の値になるので，1 ステップごとの位置変化の 2 乗 $(\Delta x)^2$ は

$$\left\langle (\Delta x)^2 \right\rangle = (\Delta x)^2 p_R + (-\Delta x)^2 p_L = (\Delta x)^2 \tag{7.2}$$

と有限な値になります。よって，n ステップ後には

$$\langle x^2 \rangle = \langle (\Delta x)^2 \rangle n = (\Delta x)^2 n \tag{7.3}$$

となります。

式 (7.3) は，一般的に，確率変数 x の**分散（variance）**と呼ばれる量

数学公式 7.1 (分散の定義式)**.**

$$\sigma^2 = \langle (x - \langle x \rangle)^2 \rangle = \langle x^2 \rangle - \langle x \rangle^2 \tag{7.4}$$

に対応します。今の場合，$\langle x \rangle = 0$ としているので，(7.4) の右辺の二項目はゼロとなり，式 (7.3) は分散と一致します。

(7.3) は，ランダムに動く個体の位置のばらつきが，時間とともに大きくなっていくことを示しています。その広がりの速さを測る量として，**拡散係数（diffusion coefficient）**

$$D \equiv \frac{(\Delta x)^2}{2\Delta t} \tag{7.5}$$

という量が用いられます。これと $t = n\Delta t$ を用いれば，二乗変位の平均は

$$\langle x^2 \rangle = 2Dt \tag{7.6}$$

と表せます。そして，このルートを取ることで，時刻 t 後の典型的な変位を

$$\sqrt{\langle x^2 \rangle} = \sqrt{2Dt} \tag{7.7}$$

と見積もることができます。

7.1.2 ランダムウォークシミュレーション

本当の意味でランダムな値を生成することはできませんが，実際上はそれを模した**擬似乱数（pseudorandom numbers）**が用いられます。Python には，そのためのモジュールとして，`random` があります。これを用いて，ランダムウォークをシミュレーションし，可視化してみましょう。

ここでは，各ステップで $\Delta x = +1$ か $\Delta x = -1$ のどちらかを選ぶだけだとします。この場合，同じモジュールの `random` の関数だけでも，ランダムウォークの基本要素である，確率的に移動の方向を決める部分を実装できる関数は

複数あります。直感的にわかりやすいのは $0 \le r < 1$ の乱数 r を与える関数 `random.random` を用いることです。もう一つは，リストから要素を一つ選ぶ関数 `random.choice` を用いることです。以下，$\Delta x = \pm 1$ を `step` として実装した例です。

```
import random

# 方法1
if random.random() < 0.5:
    step = 1
else:
    step = -1

# 方法2
step = random.choice([ 1, -1 ])
```

どちらでも同じ結果が得られます。

二つ目の方法を採用すると，N 体からなるアンサンブルのランダムウォークは，以下の単純なコードで計算して記録できます。

```
import random
import matplotlib.pyplot as plt

N = 1000
nsteps = 500

path_ensemble = []

for i in range(N):
    x = 0
    path = [x]
    for t in range(nsteps):
        dx = random.choice([-1, 1])
        x += dx
        path.append(x)

    path_ensemble.append(path)
```

ここでは初めに，軌道のアンサンブル `path_ensemble` を用意し，アンサンブルの各個体 `i` について順番に軌道を計算します。軌道 `path` は `nsteps` の時間ステップに渡り，ランダムに `dx` が -1 か 1 を決めて足しこんでいくだけです。

描画するには以下のスクリプトを足せばいいだけです。ループの数は描画す

る軌道の数で，あまり多く選ぶと図が見えにくくなるので調整してみてください。結果は図 7.1 のようになります。原点から出発した軌道の束が，もにょもにょと変化しながら広がっていっているような様子が見て取れます。このように，正負の方向にランダムに移動しているだけであっても，統計的に見ると，時間とともに分布が広がっていくのです。

```
for i in range(20):
    plt.plot(path_ensemble[i], color='k', alpha=0.5)
plt.xlabel('Time step')
plt.ylabel('x')
plt.grid(True, alpha=0.5)
plt.show()
```

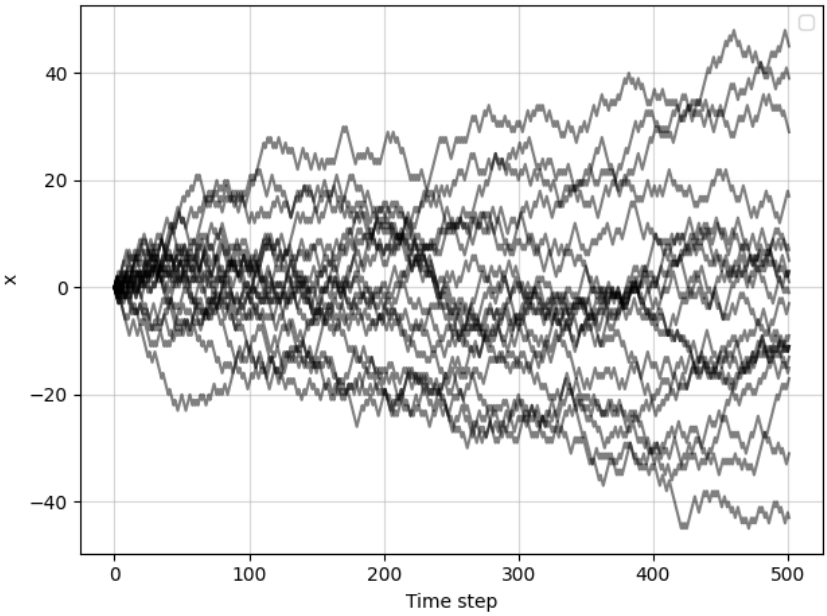


図 7.1　$\Delta t = 1$，$\Delta x = \pm 1$ とした 1 次元ランダムウォークの様子。

7.1.3　分散のシミュレーション

図 7.1 で描画したのは，アンサンブル中の各個体の軌道です。続いて，これらの軌道の統計的性質である分散 (7.3) あるいは (7.4) を計算してみましょう。拡散係数はこれを $2t$ で割ったものなので，拡散係数を求めるシミュレーションと考えても良いです。今は $\Delta t = 1$，$(\Delta x)^2 = 1$ および $t = n\Delta t = n$ としているので，理論的な予測では，$\langle x^2 \rangle = n = t$ という直線になるはずで，この直線の傾きの半分 0.5 が拡散係数になります。

```
time = np.arange(nsteps + 1)
path_mean = []
var_list = []

for t in range(nsteps+1):

    sum_x = 0
    for i in range(N):
        sum_x += path_ensemble[i][t]

    x_mean = sum_x / N
    path_mean.append(x_mean)

    sum_diff_sq = 0
    for i in range(N):
        sum_diff_sq += (path_ensemble[i][t] - x_mean) ** 2

    var = sum_diff_sq / N
    var_list.append(var)

var_theory = time

plt.plot(var_list,   label="Simulation")
plt.plot(var_theory, label="Theoretical")
plt.xlabel('Time step')
plt.ylabel('variance')
plt.legend()
plt.grid(True, alpha=0.5)
plt.show()
```

サンプルコード 7.1　変位二乗の分散計算用スクリプト。

分散を計算するには，各時刻におけるアンサンブル中の個体の位置とその平均が

必要で，その情報は軌道のアンサンブル path_ensemble に記録されています。path_ensemble は，個体を区別する次元と，時刻を区別する次元からなる二次元の配列ですので，時間と個体の二重のループ中で，個体 i の時刻 t における位置 path_ensemble[i][t] を一つずつ取り出し，各時間点での平均 x_mean を計算しています。その後，この平均と各個体の位置の差の二乗の和 sum_diff_sq を計算し，それを個体数 N で割ることで，分散 var を求めています。分散の時間変化を見たいので，それをリスト var_list に記録しています。これを描画した結果が図 7.2 です。

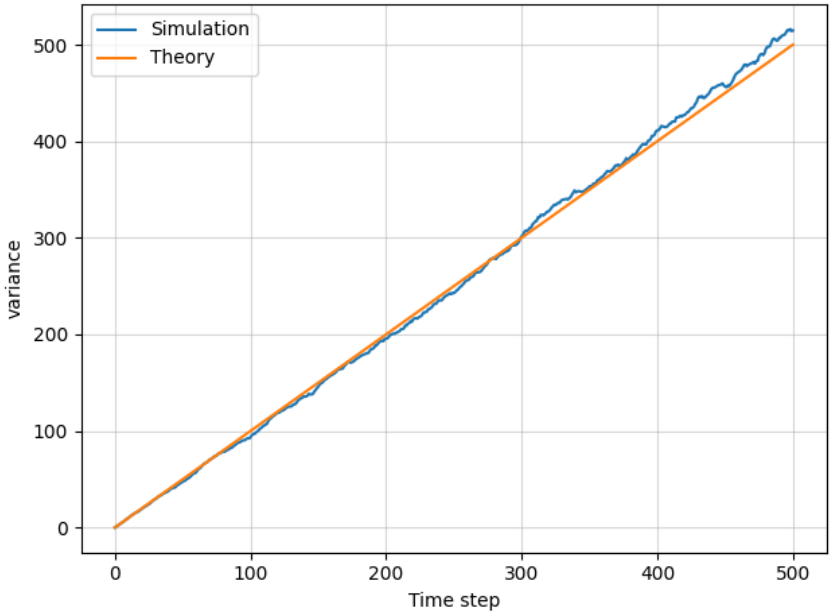


図 7.2　$\Delta t = 1$，$\Delta x = \pm 1$ とした 1 次元ランダムウォークの分散。青線がシミュレーション結果，オレンジ線が理論予測を表します。

図 7.2 が示すシミュレーション結果（青線）は，直線の周りでゆらいでおり，理論予測（オレンジ線）からわずかにずれていますが，おおむね直線的になっています。これは N=1000 とした結果ですが，これを小さくしたり大きくしたりして，結果がどう変わるかを確かめてみてください。アンサンブルが大きくなるにつれて，シミュレーション結果が理論予測と高精度で一致していく様子が確

認できるはずです。

コーディングの補足 7.1. 分散を計算するサンプルコード 7.1 は，手順を明示するためにループを使って書きましたが，NumPy の関数 var を用いれば，ループ部分

```
for t in range(nsteps+1):
    （略）
    var_list.append(var)
```

をすべて

```
arr = np.array(path_ensemble)
var_list = np.var(arr, axis=0)
```

で置き換えられます。

7.2　確率密度関数

7.2.1　Gauss 分布

生物個体のアンサンブルのランダムウォークのシミュレーションにより，個体が存在する範囲が時間とともに広がっていく様子を見ました。ただ，その分布をよく見てみると，均等ではなく，どの時刻でも平均位置に近いほど多くの個体が集まっていることが見て取れます。ここでは，この分布に着目して考えてみましょう。

N 個の仮想的な個体からなるアンサンブルを考えます。ある時刻 t において，アンサンブルに含まれる個体のうち，位置が x から $x+dx$ の間にある数を N で割ったものを $\rho(x,t)dx$ と表すことにします。N で割ったのは，全範囲で積分した結果が 1 になるという条件（規格化条件）

$$1 = \int_{-\infty}^{\infty} \rho(x,t)dx \tag{7.8}$$

を満たすようにするためです。これにより，$\rho(x,t)dx$ は，時刻 t に個体が位置が x から $x+dx$ の間に見つかる確率として扱えるようになります。この $\rho(x,t)$ を**確率密度関数（probability density function）**と呼びます。しばしば pdf と省略されます。

ランダムウォークに従うアンサンブルの確率密度は

$$\rho(x,t) = \frac{1}{\sqrt{4\pi Dt}} \exp\left(-\frac{x^2}{4Dt}\right) \tag{7.9}$$

となることが示せます。これは，**Gauss 分布（Gaussian distribution）**や**正規分布（normal distribution）**と呼ばれる最も基本的な確率分布

数学公式 7.2 (Gauss 分布)**.**

$$\rho(x,t) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x-\langle x\rangle)^2}{2\sigma^2}\right] \tag{7.10}$$

そのものになっています。Gauss 分布は，平均 $\langle x\rangle$ を中心に，分散 σ^2 が大きいほど平坦で左右に広がった釣鐘(つりがね)型の分布になります。実際，(7.10) において，$\langle x\rangle = 0$ および $\sigma^2 = 2Dt$ を代入すると，(7.9) に一致することを確かめてみてください。

7.2.2 シミュレーションとの比較

実際に前節でやった軌道の分布が Gauss 分布になっていることを確かめてみましょう。ばらつきのあるデータから分布を作る最も簡単な方法の一つは，`matplotli.pyplotb` のヒストグラムを描画する `hist` を使うことです。

まず，特定の時刻での分布を計算するため，適当な時間ステップを指定して，その時刻の分布を取り出します。

```
arr   = np.array(path_ensemble)
it    = nsteps
arr_t = arr[:, it]
```

以下のようにすることで，これをヒストグラムとして描画できます。

```
plt.hist(arr_t, bins=50, density=True, color='C0', edgecolor='
  k')
```

ここで，最初の引数に，描画するデータを入れます。次に `bins` で，ヒストグラムのビンの数を指定します。このとき，アンサンブルの数 N に対し，ビンの数が多すぎても少なすぎても，きれいな分布を表現できませんので，N とビン数を調整してみてください。次のオプション `density` も重要で，これを `True`

にすることで，規格化条件 (7.8) を満たすようスケールを自動調整してくれます。`color` と `edgecolor` は，ヒストグラムの塗りつぶしの色と枠の色を指定します。

$N = 10000$ とし，ステップ 10000 でこれを描画したのが，図 7.3 です。赤線は，解析的な Gauss 分布 (7.9) を表します。ビンの数もアンサンブルのサイズ N も有限であるため，ヒストグラムは赤線と完全に一致はしませんが，良い精度で一致していることが確認できます。

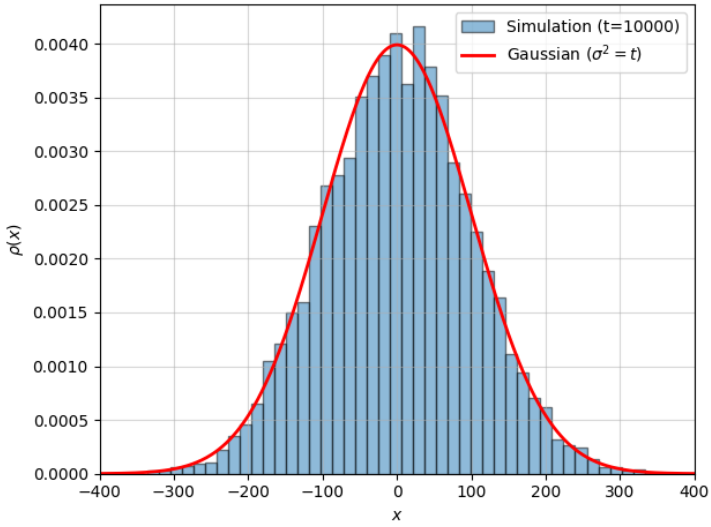


図 7.3　$N = 10000$ とし，ステップ 10000 でランダムウォークするアンサンブルの分布をヒストグラムで表したもの。赤線は Gauss 分布 (7.9) を表します。

7.3　確率論的なクマの行列モデル

§ 5.4 で，Lewis et al., 2014 を基に，クマの推移行列を作り，人為的なエサと殺処分の影響を含めた，長期的な個体数変動を調べました。ここではさらに，自然のエサの量の変動を確率論的にモデルし，長期的な個体数の変動をシミュレーションしてみましょう。

§ 5.4 で作成した推移行列を作るコードの下に，時間発展計算用のコードを追

加していきます。まず，シミュレーション設定のパラメータを

```
nsteps = 50
nensemble = 500
p_good = 0.75
```

と定義します。ここで，nsteps は時間ステップ数で，このモデルでは 1 ステップ 1 年であるため，50 年の計算をします。nensemble はアンサンブル数で，計算結果の妥当性を評価するためには，それ以上値を増やしても結果が定性的に変わらなくなるくらい十分な大きさに取る必要があります。p_good は，各年で，自然なエサが豊作である確率で，元論文に合わせて 0.75 としています。気候変動の予測に応じてこの値を調整することで，気候変動の影響を評価できます。

続いて，初期条件と空の配列を用意します。ここでは例としてベースラインシナリオの個体数計算のためのコードを示します。

```
N0 = np.array([20, 30, 30, 30])
Nmean_base = np.zeros(nsteps + 1)
Nmean_base[0] = np.sum(N0)
```

N0 は初期の個体数のベクトルで，要素はそれぞれのライフステージの個体数を表します。Nmean_base が，各時刻での個体数を記録するための配列です。初期値として，初期の全ステージでの総個体数を代入しています。

後は，試行ごとに，個体数の時間発展を計算します。アンサンブルと時間ステップの二重ループになります。

```
for i in range(nensemble):

    N_base = N0.copy()

    for t in range(nsteps):

        p = random.random()

        if p < p_good:
            N_base = A_base_good @ N_base
        else:
            N_base = A_base_poor @ N_base

        Nmean_base[t+1] += np.sum(N_base)

Nmean_base[1:] /= nensemble
```

```
plt.plot(Nmean_base, label='Baseline', linestyle='-')
plt.xlabel('Year')
plt.ylabel('Total Population')
plt.legend()
plt.grid(True, alpha=0.5)
plt.show()
```

ここでは，試行 i ごとに，初期状態のベクトルを N_base として取り出し，時間ループを計算しています。このとき，配列をそのまま N_base = N0 として複製しようとすると，同じデータ（実体）に新たに名前が貼り付けられるだけになってしまい，N_base の変更が元の N0 にまで反映されてしまうようになります。そのため，.copy() を用いることで，中身が同じ別の配列 N_base を新たに作成する必要があります。

時間ループの中では，毎ステップ乱数 p を振り，p < p_good であれば豊作，そうでなければ不作の場合の行列を用いて $N(t+1) = AN(t)$ を計算しています。アンサンブルの個々の履歴は保存せず，毎ステップごとに Nmean_base にアンサンブルの合計を足しこんでいき，ループを抜けた後にアンサンブル数で割ることで，最終的な統計平均値のみを計算しています。なお，初期値（$t = 0$ の要素）は全試行で共通に計算されているため，最後の処理（/= nensemble）は最初（0 番目）の要素を除いた [1:] の範囲に対して行っています。

コードを実行した結果は，図 7.4 のようになります。図ではベースラインシナリオ（青線）に加えて，殺処分率が低い場合（low removal；緑線）と高い場合（high removal；オレンジ線）のシナリオも同時に計算して描画しています。このシミュレーション設定では，ベースラインシナリオでも時間とともに個体数が減少していくのがわかります。Low removal シナリオでは，人間社会との接触の影響にうち，殺処分よりも人為的な食糧の影響が上回り，個体数が増加しています。対して High removal シナリオでは，殺処分の影響が上回り，ベースラインシナリオよりも速く個体数を減らしています。シミュレーションでは豊作の確率を毎年ランダム 75% としましたが，論文でも議論されている通り，この先，気候変動により不作の確率が増加していくことも予測されます。また，不作の年はランダムではなく，何年も続く可能性もあります。これらの可能性を踏まえると，より現実的なシナリオはもっと個体数の減少が速いものになるかもしれません。

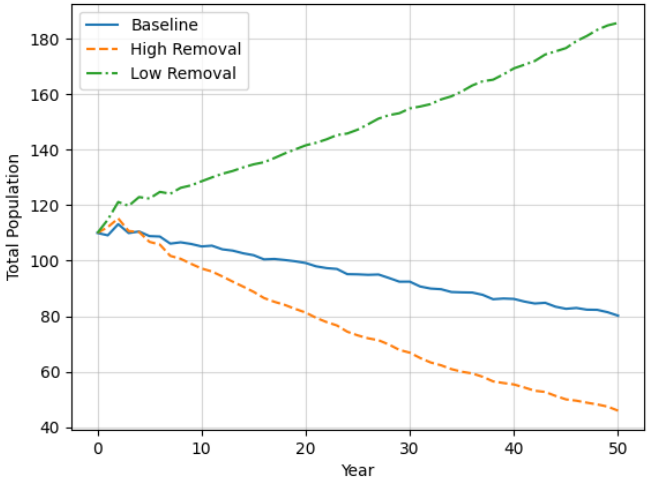


図 7.4　自然なエサが豊作である確率を $p_{good} = 0.75$ とした場合の，50 年に渡るクマの個体数の時間発展。青線，オレンジ線および緑線は，それぞれベースラインシナリオ，低殺処分率シナリオおよび高殺処分率シナリオに対応します。

改めて，この計算では統計平均のみを計算しており，個々の試行の結果は保存していないため，結果のばらつきや広がりは調べていません。詳細な解析を行うためには，そうした量の計算も必要になります。また，計算に用いたパラメータは，北米におけるアメリカグマに関するフィールド調査のデータに基づいており，そのまま日本のヒグマやツキノワグマの個体数予測には用いれないことに注意してください。しかし，本書で提示した数理モデルやコード自体は，他地域および多種の個体数の動態を調べるための基礎として使用できるため，関心のある読者は，ぜひ国内でのデータを収集し，解析に取り組んでください。

References

Gardiner, C. W. (2009). *Handbook of stochastic methods: For physics, chemistry and the natural sciences* (fourth). Springer-Verlag.

Lewis, D. L., Breck, S. W., Wilson, K. R., & Webb, C. T. (2014). Modeling black bear population dynamics in a human-dominated stochastic environment. *Ecological Modelling, 294*, 51–58.

第 8 章

空間分布と拡散モデル

> “現在，世界中で何十億もの野生の脊椎動物と，何千兆もの野生の無脊椎動物が苦しんでいる。これは差し迫った悲劇であるが，残念ながらこれが最悪の事態というわけでもない。未来派の中には，テラフォーミングや意図的なパンスペルミアといった手法を用いて、野生生物を他の惑星へ広げることを提案している。将来的に，多数の野生動物やその他の感覚を持つ生物を含みうる無数の現実のシミュレーションを実行することを構想する人々もいる。通常，こうした動物たちが経験することになる主観的な苦痛については，ほとんど考慮がなされていない。野生動物の苦しみを軽減するための最も容易かつ重要な方法は，それ（の存在域）を広げないことである。”
>
> — Brian Tomasik,
> *Why Vegans Should Care about Suffering in Nature, 2015*

生物の個体数の変動を予測したり制御したりするうえで，環境中で，個体群がどのように分布しているか，あるいはその分布がどのように変化していくかも重要な情報になります。第 7 章で説明した，仮想的なコピー集団の分布を解析するための数学的概念は，実際の個体群の分布を調べるのにも利用できます。そこで本章では，これらの概念を利用しつつ，新たな概念も加え，個体群の空間的分布とその変化を記述する方法を学びます。

8.1 密度分布

8.1.1 密度と密度分布

空間的な分布を考える場合, 個体数そのものよりも, 個体数の**密度分布（density distribution）**を扱う方が有用なことが多くなります。密度は，単位領域当たりの数を測る量です。例えば，1 次元的な広がりを考え，個体数 N が長さ L の範囲で直線状に分布していたとすると，全体での密度は

$$n = \frac{N}{L} \tag{8.1}$$

となります。つまり，単位長さあたりどれだけの個体数がいるかを表します。

しかし，私たちが知りたいのは，どのあたりで密度が高く，どのあたりで密度が低いかという密度の「分布」です。区間 L を等間隔 Δx の小区間に分割し，各区間内の個体数を N_i とします。すると，i 番目の区間の個体数密度は

$$n_i = \frac{N_i}{\Delta x} \tag{8.2}$$

により定義できます。これが密度分布です。これに区間の幅をかけて和を取れば，全個体数が数えあげられます

$$N = \sum_i \frac{N_i}{\Delta x} \Delta x \tag{8.3}$$

連続的な分布が知りたい場合は，この分割を限りなく小さくとればいいでしょう。しかし，例えば個体のサイズと同程度まで小さくとると，各区間に 1 個体いるかいないか，というような，本来得たい情報を与えるような関数ではなくなってしまいます。そこで，分布の空間的な変化を捉えられる程度には小さく，得られる分布が十分滑らかになるくらいには，各区間に十分な個体数がいるような極限の取り方をしますただ，そのためには，個体数が十分たくさんいないといけないことになります。バクテリアなどを考える場合にはそれでいいかもしれませんが，動物の個体数などではそこまでの数がいることは稀です。一つの考え方としては，第 7 章で導入した，同じような系を無数に集めたアンサンブルを用い，平均化した量を見ているとみなすことです。その場合，$n(x,t)$ は，確率密度 $\rho(x,t)$ に個体数をかけた $N\rho(x,t)$ と同一視されます。

その結果，密度分布は，各点 x の連続的な関数 $n(x)$ として扱えるようになります。これを領域全体で積分すれば，全個体数 N に等しいため

$$N = \int_0^L dx n(x) \tag{8.4}$$

が成り立ちます。以下，密度分布のことを単純に密度とも呼びます。

8.2 偏微分

密度分布は，一般に個体の移動や出生および死亡などにより時間的に変化します。よって，空間一次元だけを考えても，空間座標 x だけでなく，時間 t にも依存し，$n(x,t)$ という二変数の関数になります。そのため，例えば密度の空間変化率（x について微分）の値も，特定の時刻ごとでしか定まりません。そこで，関数 $n(x,t)$ について，t を固定して，x 方向の変化率を計算する演算を，d ではなく，やわらかい記号 ∂ を使って

$$\frac{\partial n}{\partial x} \equiv \lim_{\Delta x \to 0} \frac{n(x+\Delta x, t) - x(x,t)}{\Delta x} \tag{8.5}$$

と表します。反対に，ある位置を固定して，その位置での密度の時間変化を計算したい場合には

$$\frac{\partial n}{\partial t} \equiv \lim_{\Delta t \to 0} \frac{n(x,t+\Delta t) - x(x,t)}{\Delta t} \tag{8.6}$$

とします。一般に，関数 $f(x,y)$ に対し

数学公式 8.1 (偏微分)**.**

$$\frac{\partial f}{\partial x} \equiv \lim_{\Delta x \to 0} \frac{f(x+\Delta x, y) - f(x,y)}{\Delta x} \tag{8.7}$$

を，関数 f の x に関する**偏微分（partial derivative）**と呼びます。これは，変数が三つ以上になっても同じです。やっていることはこれまでと本質的に変わりません。混乱を避けるために新たな表記を導入しただけで，ある一点での変化率を求めるだけです。

8.3　拡散現象

8.3.1　連続の式

ある 1 次元的な空間に拡がりをもって分布している生物集団を考えましょう。また，考えている時間内で，出生も死亡も起きず，個体数は一定であるとします。すると，ある領域の中の個体数が変化するのは，その領域を個体が出入りする場合に限られることがわかります。そして，その領域での正味の個体数の増減は，入ってくる個体数と出ていく個体数の差に等しいはずです。このことを数式で表してみましょう。

まず，考える領域全体の総個体数を N とすると，その変化率は dN/dt と表せます。これを，単位時間に領域に出入りする個体数の正味の数を Q と表せば，上で述べた関係が

$$\frac{dN}{dt} = Q \tag{8.8}$$

と表現できます。

式 (8.8) の左辺は，密度を用いて表すと

$$\frac{d}{dt}N(t) = \frac{d}{dt}\int_0^L n(x,t)dx = \int_0^L \frac{\partial n(x,t)}{\partial t}dx \tag{8.9}$$

です。密度 $n(x,t)$ は時間 t に加えて空間 x にも依存するため，固定した空間点での時間変化は偏微分 $\partial/\partial t$ で表されます。それを領域全体で積分したものは，領域内の全個体数 N の変化率と等しく，空間座標には依存しないので通常の微分 d/dt で表されます。式 (8.9) は，保存式 (8.8) の左辺について，個体数の変化率の大域的な表現 dN/dt と，空間各点での局所的な変化率 $\partial n/\partial t$ を関係づける式になっています。

Q についても同じような関係を探ってみましょう。領域全体を幅 Δx の M 個の区間に分割します。そのうち，一つの区間に着目し，その小領域の左端の空間座標を x_i とすると，右端の座標は $x_{i+1} = x_i + \Delta x$ となります。また，単位時間あたりに座標 x_i を通過する個体数を $j(x_i)$ とします。この量は，左から右に移動するときに正の値を取ることとします。すると，左端から単位時間に領域に「入る」個体数は $j(x_i)$ で，右端から「出ていく」個体数は $j(x_i + \Delta x)$ なので，単

位時間当たりの正味の個体数の変化は

$$q_i = -[j(x_i + \Delta x) - j(x_i)] \tag{8.10}$$

となります。ここで，入り量が出ていく量より多い場合に正の値となるように，右辺に負号がついています。これをすべて足し合わせたものが，式 (8.8) の右辺 Q に等しくなります。

幅 Δx が小さければ

$$j(x+\Delta x) - j(x) = \frac{j(x+\Delta x) - j(x)}{\Delta x}\Delta x \simeq \frac{\partial j}{\partial x}\Delta x \tag{8.11}$$

と近似できるので，求める和は（符号に注意して）

$$Q = -\sum_{i=0}^{M-1}[j(x_{i+1}) - j(x_i)] = -\sum_{i=0}^{M-1}\left(\frac{\partial j}{\partial x}\right)_{x=x_i}\Delta x \simeq -\int \frac{\partial j}{\partial x}dx \tag{8.12}$$

と近似できます。これと，式 (8.8) および (8.9) から

$$\int_0^L \frac{\partial n(x,t)}{\partial t}dx = -\int_0^L \frac{\partial j(x,t)}{\partial x}dx \tag{8.13}$$

が得られます。あるいはこの式の両辺は，同じ積分範囲に関する積分であり，その範囲の取り方によらず成り立つため，積分の中身同士を結び付けて

$$\frac{\partial n(x,t)}{\partial t} + \frac{\partial j(x,t)}{\partial x} = 0 \tag{8.14}$$

と表現することもできます。式 (8.13) および (8.14) は，**連続の式（continuity equation）**と呼ばれるもので，それぞれ積分形と微分形と呼ばれます。連続の式は流れのある量の極めて一般的な式で，生物個体に限らず，物質や確率など，様々な量の保存則を表すのに利用されます。

8.3.2 拡散とは

生物が生息域を広げていく様子は，最も単純な場合，**拡散（diffusion）**としてモデルできます。拡散は日常的にも耳にする言葉でしょうし，実際に様々な場面でよく観察される現象です。例えばコーヒーにミルクを加えたとき，何もしなくとも勝手に広がっていくのも主に拡散によるものです。このような物質の

自然な広がりが起こる理由は，§ 7.1 で学んだランダムウォークの描像に基づいて理解することができます。

ランダムウォークする対象の統計的な分布は時間とともに広がっていくことを見ました。そこで扱ったのは仮想的な個体の集団でしたが，実際の生物や粒子の集まりでも，相互作用の影響が無視できて，運動が十分ランダムに近ければ，その分布の変化はランダムウォークに基づいて記述できます。

ある空間で，密度の高いところと低いところがあり，各個体はランダムウォークしているとすると，密度の高いところから広がっていく個体数の方が，密度の低いところから高いところに広がっていく個体数よりも多くなります。その結果，時間とともに，密度の高いところと低いところの差がなくなっていきます。そして密度が一様になると，ある領域から別の領域に移る数はどこでも平均的に見れば同じになるので，それ以上変化が見えなくなります。

改めて，ここには方向性を持つ力が加わっているわけではなく，個々の対象を見ればどれも同じようにランダムに動いているだけです。ただ，全体で見ると密度に偏りがある場合，純粋に統計的な結果として，密度の高いところから低いところへの移動が起こるのです。これが拡散の仕組みです。

8.3.3　拡散方程式

個体の移動が拡散として記述できる場合，密度の差に比例して，密度を均一化する方向に流れが生じるため，個体数の流れは

$$j = -D\frac{[n(x+\Delta x) - n(x)]}{\Delta x} \simeq -D\frac{\partial n}{\partial x} \tag{8.15}$$

と書けるでしょう。拡散の強さを表す係数 D は，§ 7.1 で導入した拡散係数 (7.5) と一致することが示せます。以下，D は時間的にも空間的にも一定であると仮定します。式 (8.15) は，密度の差がなくなると流れがゼロになる形になっていることを確かめてみてください。また，密度が周りより大きい場合は流出が起こるため，右辺に負号がついていることに注意してください。

式 (8.15) を連続の式 (8.14) に代入すると，**拡散方程式（diffusion equation）**

$$\frac{\partial n}{\partial t} = D\frac{\partial^2 n}{\partial x^2} \tag{8.16}$$

が得られます。拡散によって密度が変化する様子を記述するこの方程式も，様々

な対象に用いられるとても一般的な方程式になります。

これまで，空間 1 次元の場合に絞って話をしてきましたが，2 次元以上に広げることは容易です。詳細は省きますが，空間を xy の 2 次元とした場合，拡散方程式は

$$\frac{\partial n}{\partial t} = D\left(\frac{\partial^2 n}{\partial x^2} + \frac{\partial^2 n}{\partial y^2}\right) \tag{8.17}$$

となります。

8.4 拡散方程式のシミュレーション

8.4.1 1 次元拡散方程式の差分化

一階微分 $\partial n/\partial x$ は Δx が小さければ

$$\frac{\partial n}{\partial x} \simeq \frac{n(x+\Delta x) - n(x)}{\Delta x} \tag{8.18}$$

と近似できるのでした。二階微分は，この一階微分の差を取って，Δx で割ったものの極限なので

$$\begin{aligned}\frac{\partial^2 n}{\partial x^2} &\simeq \frac{1}{\Delta x}\left(\frac{n(x+\Delta x) - n(x)}{\Delta x} - \frac{n(x) - n(x-\Delta x)}{\Delta x}\right) \\ &= \frac{n(x+\Delta x) - 2n(x) + n(x-\Delta x)}{(\Delta x)^2}\end{aligned} \tag{8.19}$$

と近似できます。時間微分は $\partial n/\partial x \simeq [n(x, t+\Delta t) - n(x,t)]/\Delta t$ なので，1 次元の拡散方程式は，

$$n(x, t+\Delta t) = n(x,t) + D\frac{n(x+\Delta x, t) - 2n(x,t) + n(x-\Delta x, t)}{(\Delta x)^2}\Delta t \tag{8.20}$$

と差分化できます。

数値計算をする際には，1 ステップ当たりの時間幅 Δt を大きくとればとるほど計算が速く進みます。しかし，あまりに Δt が大きいと，近似の精度が悪くなります。さらに，拡散方程式の場合

$$\alpha \equiv D\frac{\Delta t}{(\Delta x)^2} \leq \frac{1}{2} \tag{8.21}$$

という制限を課す必要があります。

8.4.2　1 次元拡散方程式のシミュレーション

1 次元拡散方程式に従う密度の時間変化を，アニメーションにより可視化してみましょう。アニメーションの説明に移る前に，基本的なパラメータ設定を記述しておきましょう。

```
L = 100         # 空間のサイズ
nx = 100        # 空間の分割数
dx = L / nx     # 空間刻み幅
D = 1.0         # 拡散係数
dt = 0.1        # 時間刻み幅
alpha = D * dt / (dx ** 2)
```

そして，密度を記録する空の配列と初期値を

```
n = np.zeros(nx)
n[nx//2] = 10.0
```

としておきます。初期値は見やすいように中央（nx//2）のセルにだけ値を与えています。

続いて，描画の準備を行います。

```
fig, ax = plt.subplots()
line, = ax.plot(n)
ax.set_xlabel("x")
ax.set_ylabel("Density")
```

ここではまず，図全体 fig と，その中の描画領域 ax を作ります。その後，初期値を line, = ax.plot(n) として描画します。このとき，line, のカンマ「,」はタイポではありません。ここでは一本の線しか描画していませんが，ax.plot は複数の線を書くことができるため，一般に線のリストを返すようになっています。後の処理のためには，リストではなくその要素が必要になるので

```
line_list = ax.plot(n)
line = line_list[0]
```

とする必要があるのですが, これを省略した書き方をすると line, = ax.plot(n) で済ますことができます。

さて，こうして準備ができたら，あとはこの line をフレームごとに更新していくことで，アニメーションを生成できます。アニメーションは，

matplotlib.animation モジュールの FuncAnimation クラスによって実行されます。そのため，コードの先頭で

```
import matplotlib.animation as animation
```

を加えます。そして，以下のようにフレームの更新とアニメーション生成を実装します。

```
def update(frame):

    n[1:-1] = n[1:-1] + alpha * (n[2:] - 2*n[1:-1] + n[:-2])

    line.set_ydata(n)

ani = animation.FuncAnimation(fig, update, frames=1000,
  interval=30)
plt.show()
```

FuncAnimation の引数には，対象となる図 fig と，更新のための関数 update（名前は自由に決めていい）に加え，アニメーションのフレーム数（frames）と，フレームごとの更新間隔（interval）を入れます。update 更新で，フレームを進めながら拡散方程式 (8.20) を解き，set_ydata(n) により，新たに計算された値を line に書き込んでいきます。

拡散方程式を解く部分ですが，境界条件は $n(0) = n(L) = 0$ なので，配列の両端の要素 n[0] と n[nx-1]=n[-1] を除外した計算になっています。M =nx とし，空間位置のラベルを添え字にして丁寧に書くと，式 (8.20) は

$$\begin{aligned} n_1(t+\Delta t) &= n_1(t) + \alpha\,[n_2(t) - 2n_1(t) + n_0(t)] \\ n_2(t+\Delta t) &= n_2(t) + \alpha\,[n_3(t) - 2n_2(t) + n_1(t)] \\ &\vdots \\ n_M(t+\Delta t) &= n_M(t) + \alpha\,[n_{M+1}(t) - 2n_M(t) + n_{M-1}(t)] \end{aligned} \tag{8.22}$$

と M 本の式になります。しかし，これをループを使って書くよりも，上述のようなスライスを用いることで，表 8.1 のように配列要素の位置がずれ，簡易的な書き方ができます。コードがすっきるするだけでなく，Python では一般に，ループを使って要素ごとに計算するより配列ごと一括で計算する方が計算も速いため，可能な場合はこのような書き方が推奨されます。

表 8.1　スライスした配列の要素位置。

要素位置	0	1	2	3
`n[1:-1]`	n_1	n_2	n_3	n_4
`n[2:]`	n_2	n_3	n_4	n_5
`n[:-2]`	n_0	n_1	n_2	n_3

実行すると，アニメーションが生成されます。図 8.1 はそのスナップショットを重ねて描画したものになります。中心に集中していた密度が，釣り鐘型となり，時間とともに広がっていく様子が確認できると思います。この分布は，Gauss 分布 (7.9) になっており，この時間発展は，図 7.1 で描画したランダムウォークする個体の集団の分布の広がりを別の視点から見たものに対応します。ただし，密度の全領域積分は 1 ではなく個体数 N に等しくなるため，密度分布は N を含み

$$n(x,t) = \frac{N}{\sqrt{4\pi D t}} \exp\left(-\frac{x^2}{4Dt}\right) \tag{8.23}$$

で与えられます。

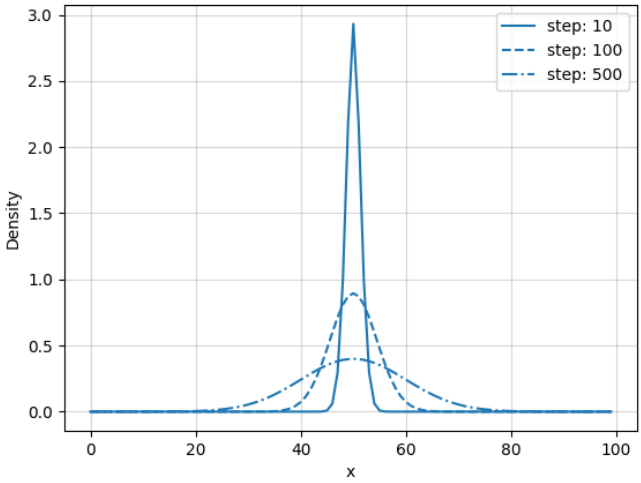


図 8.1　1 次元拡散方程式に従う密度の時間変化。

アニメーションをファイルに保存するには

```
ani.save("fig_diffusion_1d.gif", writer="pillow")
```

あるいは

```
ani.save("fig_diffusion_1d.mp4", writer="ffmpeg")
```

のようにします。ただし，mp4 の場合は，ffmpeg をインストールする必要があります。

8.5 反応拡散モデル

拡散モデルでは与えられた分布が単に広がっていく様子だけを記述するのでした。境界条件として，領域の端で $n = 0$ とすると，分布は時間とともに平坦化されながら，境界から個体が流出していき，最終的には境界内の個体数はゼロになります。しかし実際の生物は，個体群の中で出生や死亡が起こり，個体数の変動を伴います。あるいは種によっては，人為的な導入や除去などによっても個体数が変動します。こうした効果を加えることで，個体数の密度が変動しながら移動していく様子をモデルできます。特に，農作物の保護や環境主義的思想などの人間中心主義的観点からは，生物群が新たな領域に侵入（invasion）してくる様子や条件が詳しく調べられてきました。本節では，このような振る舞いを調べるための基本的なモデルを示します。

8.5.1 反応拡散方程式

上で述べた，個体数の増減を表す効果を拡散方程式に加えます。これらの効果をまとめて S とし，拡散方程式に加えると

$$\frac{\partial n}{\partial t} = D\frac{\partial^2 n}{\partial x^2} + S \tag{8.24}$$

が得られます。この新たに加えられた項 S は一般に，**ソース項（source term）**と呼ばれます。n を化学物質の濃度としたとき，ソース項は化学反応による生成および消滅を表すのに用いられるため，この形の式は**反応拡散方程式（reaction diffusion equation）**と呼ばれます (Livi & Politi, 2025; Phillips et al., 2009)。またこのとき，ソース項は反応項とも呼ばれます。

8.5.2　Fisher-KPP 方程式

ソース項として，これまで用いてきた最も一般的な形である，ロジスティック成長項を加えてみましょう。成長率を $r>0$，環境収容力を $K>0$ とすると

$$\frac{\partial n}{\partial t} = r\left(1-\frac{n}{K}\right)n + D\frac{\partial^2 n}{\partial x^2} \tag{8.25}$$

となります。この形の式は，Fisher 方程式や，Kolmogorov-Petrovsky-Pikunov 方程式，あるいは前者の名前と後者の頭文字から **Fisher-KPP 方程式（Fisher-KPP equation）**と呼ばれます。

Fisher-KPP 方程式を，$0 \le x < \infty$ の範囲で，境界条件 $n(0)=K$，$n(\infty)=0$ で解いてみましょう。初期値として，$x=l>0$ のところまで $n(x)=K$，それより後（$x \ge l$）は $n(x)=0$ とします。$r=0.5$，$K=10.0$，$D=1.0$ として解いた結果が，図 8.2 になります。この図より，密度分布は形状を保ったまま右に伝播（でんぱ）して行く様子が確認できます。このように，形状を保ったまま一定の速度で伝播する解を，**進行波解（traveling wave solution）**と呼びます。Fisher-KPP 方程式の進行波解の伝播速度は

$$c = 2\sqrt{Dr} \tag{8.26}$$

となります (Kot, 2001)。

8.5.3　Nagumo 方程式

第 4 章では，密度効果として，環境収容力による制限に加え，Allee 効果を反映したモデルを考えました。ここでも，§ 4.2 と同じ考え方で，それを取り入れてみましょう。Fisher-KPP 方程式 (8.25) に，因子 $(n/K-a)$ を導入すれば

$$\frac{\partial n}{\partial t} = r\left(\frac{n}{K}-a\right)\left(1-\frac{n}{K}\right)n + D\frac{\partial^2 n}{\partial x^2} \tag{8.27}$$

が得られます。ここで，$a>0$ は Allee 閾値です。この形の式を，**Nagumo 方程式（Nagumo equation）**と呼びます。

Nagumo 方程式 (8.27) の全体を K で割ると

$$\frac{\partial}{\partial t}\frac{n}{K} = r\left(\frac{n}{K}-a\right)\left(1-\frac{n}{K}\right)\frac{n}{K} + D\frac{\partial^2}{\partial x^2}\frac{n}{K} \tag{8.28}$$

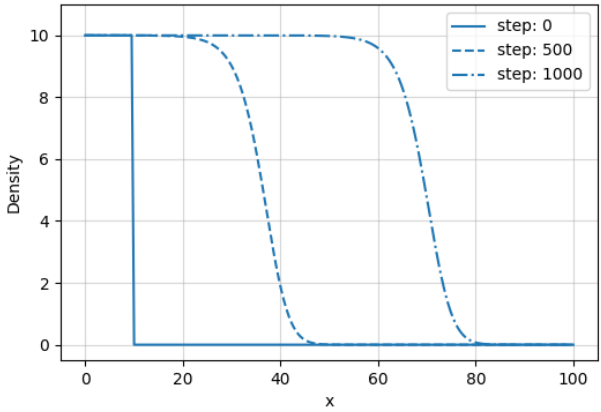


図 8.2　$r = 0.5$，$K = 10.0$，$D = 1.0$ として Fisher-KPP 方程式 (8.25) を計算した例。はじめ，$x = 10$ のところまで $n(x) = K$，それより後では $n(x) = 0$ とした密度分布が，滑らかな形状になり，時間とともにその形状を保ったまま右方向に伝播していきます。

となります。ここで，規格化された密度を $u \equiv n/K$ とすれば，方程式は

$$\frac{\partial u}{\partial t} = r\,(u-a)\,(1-u)\,u + D\frac{\partial^2 u}{\partial x^2} \tag{8.29}$$

と簡略化されます。さらに，時間変数 t と空間変数を x を $\overline{t} = rt$ および $\overline{x} = x\sqrt{r/D}$ 規格化すれば，$r = D = 1$ とした形の式

$$\frac{\partial u}{\partial \overline{t}} = (u-a)\,(1-u)\,u + \frac{\partial^2 u}{\partial \overline{x}^2} \tag{8.30}$$

が得られます。文献によっては，特に微分方程式としての基本性質を調べるような文脈では，この形の式が用いられるので注意してください。

Nagumo 方程式 (8.27) の進行波解の伝播速度は

$$c = \sqrt{2rD}\left(\frac{1}{2} - a\right) \tag{8.31}$$

で与えられます。つまり，$a > 1/2$ であれば x 軸に沿って正の方向に，$a < 1/2$ であれば負の方向に伝播します。x 軸に沿って正の方向に伝播し，ある領域に侵入してくる生物種を考えたとき，正方向の波は侵入波（invasion wave），負の方向

の波は絶滅波（extinction wave）に対応します。このとき，$a = 1/2$ の場合は侵入と絶滅の傾向が釣り合って，定常な分布が保たれる状態を表します。

8.6　不妊虫放飼と侵入波・絶滅波

8.6.1　単一方程式モデル

前節で述べた，侵入波と絶滅波を考えるにあたって，§ 6.6 で扱った不妊虫放飼法（SIT）のモデルを用いることにしましょう。そのために，Nagumo 方程式ではなく，§ 6.6 で扱った式 (6.44) に基づく

$$\frac{\partial n}{\partial t} = \left(\frac{rn}{n+s} - b\right) n - kn(n+s) + D\frac{\partial^2 n}{\partial x^2} \tag{8.32}$$

を用います (Lewis & Van Den Driessche, 1993)。この式にも進行波解が存在し，不妊虫の密度 s に応じて，正方向に伝播する侵入波と，負方向に伝播する絶滅波に分かれます。

ここで，規格化量

$$\overline{x} = x\sqrt{\frac{r}{D}}, \quad \overline{t} = rt, \quad \overline{n} = n\frac{k}{r}, \quad \overline{s} = s\frac{k}{r}, \quad A = \frac{b}{r} \tag{8.33}$$

を導入し，式 (8.32) 全体に k をかけて整理すると

$$\frac{\partial \overline{n}}{\partial \overline{t}} = \overline{n}\left(\frac{\overline{n}}{\overline{n}+\overline{s}} - A - (\overline{n}+\overline{s})\right) + \frac{\partial^2 \overline{n}}{\partial \overline{x}^2} \tag{8.34}$$

と変形できます。以下，オーバーバーは省略して，同じ式を

$$\frac{\partial n}{\partial t} = \left(\frac{n}{n+s} - A - (n+s)\right) n + \frac{\partial^2 n}{\partial x^2} \tag{8.35}$$

と表すことにします。

8.6.2　解の性質

不妊虫がない場合（$s = 0$），式 (8.35) は

$$\frac{\partial n}{\partial t} = (1 - A - n)\, n + \frac{\partial^2 n}{\partial x^2} \tag{8.36}$$

となり，2 次の減少項を持つ，Fisher-KPP 方程式と構造的に同種の方程式になります。図 8.2 からも確認できるように，Fisher-KPP 方程式の進行波解の主要部分は平坦な分布（$\partial n/\partial x = 0$）になり，時間的にも変化しない（$\partial n/\partial t = 0$）ので，空間的に一様な定常解を調べることで，その性質が確認できます。

空間的に一様な定常解は $f(n) = (1 - A - n)n = 0$ で決まります。この解は，$n = 0$ と，$n = 1 - A$ の二つあり，一つ目は絶滅状態，二つ目が野生種が集団発生（outbreak）した状態に対応します。では，これに不妊虫が導入された場合（$s > 0$），解の性質はどう変化するでしょうか。

不妊虫の密度が $s > 0$ の場合，ソース項は

$$f(n, s) = \left(\frac{n}{n+s} - A - (n+s)\right) n \tag{8.37}$$

です。これがゼロとなる条件を探るため，$(n+s)$ をかけて整理すると

$$n[(n+s)^2 + A(n+s) - n] = n[n^2 + (2s + A - 1)n + s(A+s)] = 0 \tag{8.38}$$

となり，解の公式から，$n = 0$ と

$$n = -s + \frac{1 - A \pm \sqrt{(1-A)^2 - 4s}}{2} \tag{8.39}$$

の合計三つの解が得られます。n は野生種の個体数を表すので，現実の存在状態に対応するためには実数でなくてはいけません。そのためには，平方根の中身がゼロ以上である必要があり

$$s \leq s_c \equiv \frac{(1-A)^2}{4} \tag{8.40}$$

という条件が課されます。$n = 0$ の場合に加え，不妊種の個体数 s が閾値 s_c を超えた場合（$s > s_c$）も，野生種が絶滅した状態に対応します。そこで，以下では $0 < s < s_c$ の場合を考えます。

8.6.3 侵入波・絶滅波のシミュレーション

式 (8.35) の進行波解の伝播速度を解析的に求めることはできず，数値的な見積もりも少し込み入っているので，詳細は元論文 (Lewis & Van Den Driessche, 1993) にある記述を参照してください。ここでは直接シミュレーションによって，侵入波と絶滅波の様子を調べてみましょう。

用いるコードは，1 次元拡散方程式のシミュレーションに用いたものと大きな違いはなく，基本的には増えた変数を加えて，方程式を書き替えるだけです。パラメータ設定は

```
L = 100
nx = 200
dx = L / nx
x_grid = np.linspace(0, L, nx)
D = 1.0
s = 0.15
A = 0.1
s_c = (1 - A)**2 / 4
dt = 0.1
frames = 1500
alpha = D * dt / (dx ** 2)
```

とします。ただしここで，上で述べた不妊種の個体数の閾値に関する判定を行い，野生種の個体数の定常値を決定します

```
if s < s_c:
    n_plus = (1 - A - 2*s + np.sqrt((1 - A)**2 - 4*s)) / 2
else:
    n_plus = 0

n = np.zeros(nx)
if n_plus > 0:
    n[0:50] = n_plus
```

後は，方程式を実装して，描画します。

```
line, = ax.plot(x_grid, n)
ax.set_xlabel("x")
ax.set_ylabel("Density")

def update(frame):

    diffusion = alpha * (n[2:] - 2 * n[1:-1] + n[:-2])
    reaction  = dt * ( n[1:-1] / (n[1:-1] + s) - A - (n[1:-1]
      + s) ) * n[1:-1]
    n[1:-1] = n[1:-1] + diffusion + reaction

    n[0] = n[1]
    n[-1] = n[-2]

    line.set_ydata(n)
```

```
ani = animation.FuncAnimation(fig, update, frames=frames,
  interval=30)
ani.save(f"fig_sit_single_eq_{s:1g}.gif", writer="pillow")
```

これを実行した例が，図 8.3 および図 8.4 になります。図 8.3 は，$s = 0.15$ とした場合の結果で，右向きの進行波が生じています。これは，野生種が x 軸の正の領域に侵入していく様子に対応します。対して，図 8.4 は，$s = 0.2$ とした場合の結果で，左向きの進行波が生じています。これは，x 軸の正の領域から放出される不妊種の影響で，野生種の侵入が阻害され，x 軸の負の領域に押し返されていく様子に対応します。

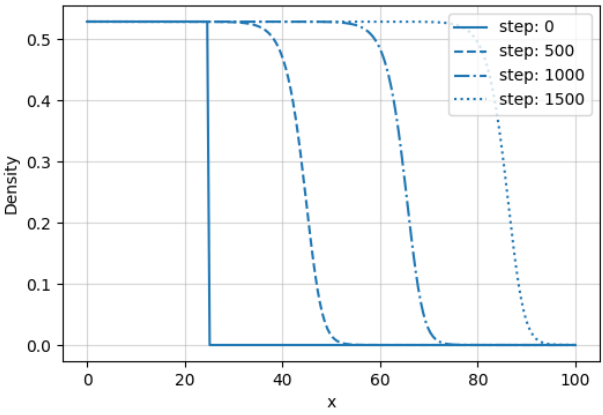


図 8.3　式 (8.35) を，$D = 1.0$，$A = 0.1$，$s = 0.15$ として解いた例。侵入波に対応する右向きの進行波が生じます。

8.7　エントロピーと生物

拡散とは，方向性を持つ力が働いているわけではないのに，その統計的な性質により，分布が自然と一様化していく傾向であることを学びました。これは自然界で普遍的に見られるもので，物質の空間分布だけでなく，温度や電位など，様々な量の分布が同じように一様化していく傾向を持ちます。この性質は，**エントロピー（entropy）**と呼ばれる基本的な物理量に関連するものです。エネル

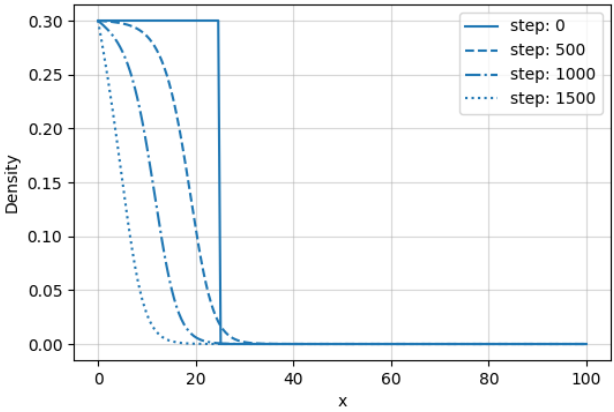


図 8.4　式 (8.35) を，$D = 1.0$，$A = 0.1$，$s = 0.2$ として解いた例。絶滅波に対応する左向きの進行波が生じます。

ギーや物質のやりとりがない空間の中では，その中の物質の密度も温度も一様になったとき，エントロピーは最大値を取ります（ここでは重力などの影響は無視します)。エントロピーの定義はここでは述べませんが，拡散によってエントロピーが生成される速度を表す量（エントロピー生成）は

$$\sigma_{\mathrm{diff}} = -\frac{j}{n}\frac{\partial n}{\partial x} = \frac{D}{n}\left(\frac{\partial n}{\partial x}\right)^2 \geq 0 \tag{8.41}$$

で表されます［二つ目の等式で，式 (8.15) を利用しました］[*1]。この量は常にゼロ以上であり，密度の偏りがなくなり，エントロピーが最大になったところでゼロになります。このように，拡散のような現象は，系のエントロピーを最大値に向けて増大させる性質の反映とみなすことができます。

生物は一見この性質とは対極にある存在に思えます。生物の体は様々な組織からなり，その一つ一つで，多様な物質の密度（濃度）や温度の分布が高精度に調整され，維持されています。このような構造の形成と維持を可能にするの

[*1] 正確には，密度勾配 $\partial n/\partial x$ ではなく，化学ポテンシャル $\mu = \mu_0 + T\ln n$ の勾配 $\partial\mu/\partial x$ を温度で割った量から計算されます。ここで μ_0 は一定の基準値で，T は温度です。ただ，温度が一様であれば，結果的に得られる式は (8.41) と一致します（Boltzmann 定数を $k_B = 1$ とする単位系を仮定しています)。

は，外部との物質やエネルギーの交換です。反応拡散方程式 (8.24) を仮定すれば，これらの外部とのやり取りは，反応項あるいはソース項に含めることで表現できます。外界とのやり取りに関連するエントロピー変化まで含めると，系の全エントロピー S の変化 dS/dt は

$$\frac{dS}{dt} = \frac{d_i S}{dt} + \frac{d_e S}{dt} \tag{8.42}$$

と書けます (Katchalsky & Curran, 1965; Nicolis & Prigogine, 1977)。ここで，$d_e S/dt$ は外部とのやり取りに伴うエントロピー変化で，$d_i S/dt$ は，系内部でのエントロピー生成 $\sigma = \sigma_{\mathrm{diff}} + \sigma_{\mathrm{chem}}$ を系全体で積分したものです（σ_{chem} は反応項に由来する寄与です)。内部のエントロピー変化は常に $d_i S/dt \geq 0$ となりますが，外部との交換項 $d_e S/dt$ は，物質やエネルギーの供給により負の値を取ることができ，全体として $dS/dt = 0$ とし，低エントロピー状態を維持したり，秩序を構成して $dS/dt < 0$ とすることが可能です。つまり，物質やエネルギーを与えれば，構造を作り，維持することが可能になるのです。例えば冷蔵庫は，電気の形でエネルギーを与えることで，部屋と異なる温度を維持する装置です。生物も同じで，主に食事により取り込んだ物質やエネルギーを利用することで，複雑な身体の管理をしています。しかし，細胞の入れ替えや修正に伴うエラーが蓄積することで，病気や老化が起こり，最終的には死に至ります。

原理的には，より多くの資源を投入すれば，より高精度な身体の修復や維持ができます。しかし，いくら飽食な環境に置かれても，私たち動物の身体は劣化を続けます。なぜでしょうか？それは，生物が身体の状態を理想的なまま維持し続ける方向に進化してきたわけではないからです。Richard Dawkins の言葉を借りれば，生物の身体は，遺伝子の複製に用いられる「乗り物」でしかありません (Dawkins, 1976, 1982)。そのため，生殖して子孫を育てるまで生きられれば，その乗り物を維持する進化的利益は失われていきます。むしろ，限られた資源の中で，遺伝子の複製に貢献しない形質は，繁殖や生存に使える資源を奪うことになり，自然淘汰により切り捨てられていきます。その結果，私たちの身体は，「私たち」自身の観点からは欠陥だらけでメンテナンスが難しく，苦しみを多く生み出す劣悪なものとなっているのです。

References

Dawkins, R. (1976). *The selfish gene*. Oxford University Press.

Dawkins, R. (1982). *The extended phenotype* [(1987). 延長された表現型: 自然淘汰の単位としての遺伝子. 日高敏隆ほか訳. 紀伊國屋書店.]. Oxford university press Oxford.

Katchalsky, A., & Curran, P. F. (1965). *Nonequilibrium thermodynamics in biophysics* [(1975). 生物物理学における非平衡の熱力学. 青野修ほか訳. みすず書房.]. Harvard University Press.

Kot, M. (2001). *Elements of mathematical ecology* (Vol. 1). Cambridge University Press Cambridge, UK.

Lewis, M., & Van Den Driessche, P. (1993). Waves of extinction from sterile insect release. *Mathematical Biosciences*, *116*(2), 221–247.

Livi, R., & Politi, P. (2025). *Nonequilibrium statistical physics: A modern perspective* (2nd). Cambridge University Press.

Nicolis, G., & Prigogine, I. (1977). *Self-organization in nonequilibrium systems: From dissipative structures to order through fluctuations* [(1980). 散逸構造. 小畠陽之助 & 相沢洋二 訳. 岩波書店.]. Wiley.

Phillips, R., Kondev, J., Theriot, J., & Garcia, H. (2009). *Physical biology of the cell* [(2011). 細胞の物理生物学. 笹井理生ほか訳. 共立出版.]. Garland Science.

Tomasik, B. (2015). Why vegans should care about suffering in nature [https://reducing-suffering.org/why-vegans-should-care-about-suffering-in-nature/].

第 9 章

エージェントベースモデル (ABM)

> “種は抽象的な存在で，何かを経験することはなく，したがってまた，個体が経験しうるのと同じように，不当な扱いを受ける対象でもない。個体のみが良い経験や悪い経験をすることが可能なのであり，それゆえ配慮すべき対象となるのである。”
>
> —Animal Ethics,
> *Why we should give moral consideration to individuals rather than species*

9.1 エージェントとは

9.1.1 エージェントベースモデル

生物集団は，生物の個体（individual）が集まって構成されるものです。各個体は，年齢や性別など，様々な性質によって特徴づけられ，それに応じて，他の個体と複雑なやりとりをします。このように，各個体ごとに紐づけられるパラメータ（内部状態）を持ち，周囲と相互作用をするものを，**エージェント（agent）**と呼びます。

これまで学んできたモデルでは，個体の総数に第一に着目して解析するものでした。対して，**個体ベースモデル（individual-based model; IBM）**あるいは**エー**

ジェントベースモデル（agent-based model; ABM）と呼ばれるものでは，個体同士の相互作用を含む，エージェントとしての個体の振る舞いを追跡し，その結果としての個体数の変動を調べます (小高, 2018)。

9.1.2　本章の内容

私たち Applied Efilism Society は，エージェントベースの野良ネコの個体数抑制効果を調べるコード NORA を開発してきました[*1] 。本章では，このコードのモデルの基本部分を簡略化したプログラミングを例に，実際にコードを作成しながら進めていきます。ただし，初めはネコの生態を反映せず，どんな生物の動態をモデルするにしても基本となる枠組みを形作ります。最初に最も基本的な Malthus 型の成長をエージェントベースでモデルします。続いて，環境収容数を導入してロジスティック型の成長に拡張した後，年齢や生存態といった属性を取り入れる方法を示します。その後は，具体的なネコの生態を反映する要素を取り入れ，野良ネコの個体数制御をシミュレーションするための具体的なモデルを実装していきます。このモデルでは，簡単のために，メスネコのみの状態を追跡します。

9.2　オブジェクト指向

9.2.1　オブジェクト指向

ABM では，多くの個体（エージェント）を同時に扱います。Python で，個体ごとの内部状態を別々に管理するのに有用なのが，**クラス（class）**です。クラスとはいわば設計図のようなものです。例えば，プログラム上でネコを模したエージェントを扱う場合，`Cat` というクラスを作っておけば，それを呼ぶだけで，「ネコ A」,「ネコ B」というように，実体を生成することができるようになります。この実体のことを，**インスタンス（instance）**や**オブジェクト（object）**と呼びます (Althoff, 2018)。

これまでの章では，数学の関数 $f(x)$ と同じように，関数に変数を入れて渡す

[*1] NORA はオープンソースのプログラムで，https://github.com/AppliedEfilism/NORA よりダウンロードできます。

f(x) のような形で処理を実行してきました。例えば，ネコの集まりを考え，各個体の年齢を関数を使って管理しようとしたら，関数 age(x) を作り，個体の識別番号を x として渡すという処理をすることになります。これに対し，オブジェクトを中心にした，**オブジェクト指向（Object-Oriented）**というプログラミング設計の流儀があり，ABM はこちらの方が適している場合が多いです。オブジェクト指向なプログラムでは，ネコの個体に対応するそれぞれのオブジェクトが，自分で年齢をはじめとする属性を持つ形で記述します。

漠然とした説明をするより具体例を見ながらの方が分かりやすいと思うので，実際にサンプルコードを作りながらその趣旨を見ていきましょう。先に結果を見たい方は，サンプルコード 9.1 を先に実行してみて，後戻りしてコードの意味を確認しながら読んでもよいでしょう。

9.2.2　クラスの使用例

オブジェクト指向の基本要素となるクラスは，主に以下の二つの要素で構成されます。

- 属性（データ）: 年齢や生存状態など，その個体が持つパラメータ。
- メソッド（処理）: クラスに属する関数で，その個体が行う処理。

特に，クラスから新しい個体を生み出す際に最初に自動で実行される特殊なメソッドを__init__（初期化メソッド）と呼び，ここで個体の初期状態を設定します。実際に以下で例を見ることで，意味が分かると思います。メソッドの利用にあたって，必要に応じて関数の基本に関する説明§ 2.4.4 を参照してください。

試しに，クラス Cat を定義し，オブジェクトを生成してみましょう。§ 2.4.5 で説明したように，いくつかのファイルで一つのコードを構成することにし，クラスの定義は agent.py というファイルで行うことにします。ついでに，パラメータを指定する config.py も作っておきましょう。

```
# config.py

N0 = 10 # 初期個体数
```

オブジェクトを生成する際に，年齢を 0，生存状態を真（True）にすることに

します。これらは，次のようなコードで実現できます。

```
# agent.py
class Cat:
    def __init__(self):
        self.age = 0
        self.is_alive = True
```

ここで，self という名前でその特定のオブジェクトを表しています。self.age および self.is_alive のように self.xxx の形に書かれている xxx が属性です。このように，オブジェクト自身が年齢や生死という内部状態（属性）を更新するという設計になっています。これが，オブジェクト指向の本質です。

実際にオブジェクトを生成するには，これを main.py から呼び出します。

```
# main.py
import config
from agent import Cat

def main():

    cats = []  # 空のリストを用意
    for i in range(config.N0):
        cat = Cat() # オブジェクトを生成
        cats.append(cat) # それを格納

if __name__ == "__main__":
    main()
```

ここで，Cat() により，agent.py 内の初期メソッド__init__が呼ばれ，オブジェクトが一つ作成されます。それを cat という名前で受け取り，リスト cats に格納します。それを，ループを回して N0 回行っているので，結果的に N0 個のオブジェクトを要素とするリスト cats が出来上がります。

コーディングの補足 9.1. 空のリストの生成からループ全体の一連の処理は

```
cats = [Cat() for _ in range(config.N0)]
```

という一行で済ますこともできます。ここで，ループに用いるインデックスは，単にカウントのために使われるだけなのでどんな文字でも構いません。逆に i などの文字を用いると，f(i) のように，何かの関数に代入して使われるのかな？と勘違いさせてしまうこともあります。なので，このようにただカウントを回すためだけに使う場合には，アンダースコア「_」を使うことも一般的です。

続いて，オブジェクトの処理を追加してみましょう。class Cat の中に次の関数を加えます。

```
# agent.py
    def update(self):

        # すでに死亡した個体についてはパス
        if not self.is_alive:
            return

        # 年齢を1単位増やす
        self.age += 1
```

まず，if not self.is_alive により生死を判定し，すでに死亡している個体の場合には何もせず return しています。生きている個体については，self.age += 1 で年齢を 1 単位増やしています。改めて，このようなクラス専用の関数が，メソッドです。メソッドを実行するには

```
    cat=cats[0]
    cat.update()
```

のようにします。ここでは，リスト cats の 0 番目の要素を cat として取り出して，メソッド update を実行しています。

9.2.3 その他応用例

オブジェクトを生成する際，何か値を渡して利用することもできます。例えば，生成の際にそれぞれ名前やサイズを指定してみたいと思ったら

```
    cat1 = Cat("Tama", 5)
    cat2 = Cat("Kuro", 10)
```

のように引数に名前とサイズを表す値を渡すようにします。これを受け取って処理するには，

```
class Cat():
    def __init__(self,name,size):
        self.age = 0
        self.is_alive = True
        self.name = name
        self.size = size
```

のようにします。まず受けとる側でも`__init__(self,name,size)`と変数を書き加えています。このとき，渡す側にはない`self`を一番左に残したまま，その次から新たに加えた変数を書き加えることに注意してください。メソッドの中身では，受け取った値を使って，新たな属性`name`と`size`をオブジェクトに与えています。こうして与えた属性は

```
print(cat1.name, cat1.age)
print(cat2.name, cat2.size)
```

のように出力して確認できます。

初期メソッド`__init__`の内容を拡張するだけでなく，新たなメソッドを作成して加えることもできます。どんな具合か確かめてみたい場合は，例えば以下のようなメソッドを加えて実行してみてください。

```
class Cat():
    def __init__(self,name,size):
     (省略)

    def play(self):
        print("Playing!") #printが実行される

    def meow(self):
        return "Meow!" #文字列をreturnする

cat1.play() # メソッドの処理により，直接printされる

# meowは文字を返すだけなので，printしないと何も起きない
print(cat2.meow())
```

9.3 サンプルコード：ネコの個体数シミュレーションコード

前の節で用意したコードを拡張して，実際に個体数の変動を計算する ABM のプログラムを作ってみましょう。乱数を振って個体（エージェント）ごとに確率論的に繁殖と死亡を判定し，個体数の変動を追跡するモデルをコード化します。よくリストの操作をするため，必要であれば § 2.5.1 を参照しながら読み進めてください。

9.3.1 時間ループの追加

まず，時間変化を追うために，時間ステップのループを追加しましょう。そのために，総ステップ数を config.py に加えます。

```
# config.py

NSTEP = 100 # 総ステップ数
```

このパラメータを使って，main.py に時間ループを加えます。

```
# main.py

def main():
    # 最初の個体群を生成してリストに格納
    cats = [Cat() for i in range(config.N0)]

    # ここに追加
    for step in range(config.NSTEP):
```

9.3.2 個体数変化の記録

今度は，個体数の変化を追うために，生きている個体の総数をカウントするようにしましょう。まず，最初に用意したリスト cats を，生きた個体を集めたリストとして利用することにします。そして，ループ内でそのステップにおける cats の要素数を表す変数 N_current を追加します。また，各ステップの N_current を要素とするリスト N_history を作れば，個体数の変化を追跡でき

ます。実際にコードに反映すると

```
# main.py

    # 空のリストを用意
    N_history = []

    for step in range(config.NSTEP):

        # そのステップでの個体数
        N_current = len(cats)

        # 個体数の履歴のリストに追加
        N_history.append(N_current)
```

となります。もちろんこのままでは，個体数は初期の値のまま変化しませんので，続いて個体数の変化の原因となる，死亡と繁殖の過程を実装していきます。

9.3.3　繁殖と死亡の実装

まず，1 ステップあたりの死亡確率 P_DEATH と，繁殖確率 P_REPRODUCE を config.py に加えます。

```
# config.py
P_DEATH = 0.1 # 死亡率
P_REPRODUCE = 0.15 # 繁殖率
```

これを用いて死亡と繁殖の判定をし，処理を実行する関数を実装しましょう。最初に，agent.py 内の update を次のように拡張しましょう。

```
# agent.py
import random

class Cat:

    def update(self):

        # すでに死亡した個体についてはパス
        if not self.is_alive:
            return

        # 死亡判定
        if random.random() < config.P_DEATH:
            self.is_alive = False
```

```
        # 年齢を1単位増やす
        self.age += 1
```

先頭で，乱数を利用するために random をインポートするのを忘れないでください。年齢の情報は今や利用していないため，更新にも意味はないのですが，後の拡張のために残してあります。新たに加えた部分では，乱数 random.random() の値が，死亡確率 config.P_DEATH を下回った場合，個体の生存状態を False に変更するようにしています。

同じように，update の下に，繁殖を行う別の関数を以下のように加えましょう。

```
    def reproduce(self):

        # すでに死亡した個体についてはパス
        if not self.is_alive:
            return None

        if random.random() < config.P_REPRODUCE:
            baby = Cat()
            return baby
        return None
```

ここでも，最初に死亡した個体についてはパスしますが，繁殖がなされなかった場合は，「値がない」ことを返す必要があるため，None を返しています。あるいはこの部分は丸まる削除して，次の if 文の条件を

```
    if self.is_alive and random.random() < config.P_REPRODUCE:
```

と書き換えてもいいです。この if 文内では，乱数 random.random() の値が，繁殖確率 config.P_REPRODUCE を下回った場合，Cat を読んで新たなオブジェクト baby を生成し，それを返します。あるいはよりシンプルに

```
    if random.random() < config.P_REPRODUCE:
        return Cat()
```

としても OK です。最後の，条件に合わなかった場合のための return None も忘れないようにしましょう。

9.3.4　メインループとの統合

繁殖と死亡を扱う関数が出来たため，あとはこれをメインループで毎ステップ呼び出し，新たに誕生した個体と死亡した個体を反映して個体数を更新すればいいです。以下のコードをメインループ内の `N_history.append(N_current)` 以下に追加しましょう。

```
    # 新しく生まれる個体のリストを用意
    new_cats = []

    for cat in cats:
        # 各個体の状態を更新
        cat.update()

        # 繁殖過程の処理
        offspring = cat.reproduce()

        # Noneでなければリストに追加
        if offspring is not None:
            new_cats.append(offspring)

    # 生き残った個体だけを抽出
    cats = [cat for cat in cats if cat.is_alive]

    # 新しく生まれた個体のリストと結合
    cats.extend(new_cats)
```

ここでは，最初に新しく生まれる個体のために空のリストを用意しています。その後，リスト `cats` 内の要素を一つずつ取り出して，メソッド `update` と `reproduce` を実行しています。そして，`reproduce` の結果，繁殖が行われたら，新たに生まれた個体を `new_cats` に加えます。ループが終了したら，リスト `cats` のうち，生き延びた個体だけ（`if cat.is_alive`）を集めて，新たにリスト `cats` を構成します。最後に，`cats` と新たな個体 `new_cats` を結合します。

9.3.5　可視化と考察

コードをまとめ，最後に可視化の処理を加えると，以下の三つのファイルのようになります。

```
# config.py
N0 = 50
NSTEP = 100
P_REPRODUCE = 0.15
P_DEATH = 0.1
```

```
# main.py
import matplotlib.pyplot as plt
import config
from agent import Cat

def main():

    cats = [Cat() for _ in range(config.N0)]

    N_history = []

    for step in range(config.NSTEP):
        N_current = len(cats)
        N_history.append(N_current)

        new_cats = []

        for cat in cats:

            cat.update()

            offspring = cat.reproduce()
            if offspring is not None:
                new_cats.append(offspring)

        cats = [cat for cat in cats if cat.is_alive]
        cats.extend(new_cats)

    # 結果の可視化
    plt.plot(N_history)
    plt.xlabel("time")
    plt.ylabel("population")
    plt.grid(True, alpha=0.5)
    plt.show()

if __name__ == "__main__":
    main()
```

```
# agent.py
import random
import config

class Cat:
    def __init__(self):
        # 個体が生まれたときの初期状態
        self.age = 0
        self.is_alive = True

    def update(self):

        if not self.is_alive:
            return

        if random.random() < config.P_DEATH:
            self.is_alive = False

        self.age += 1

    def reproduce(self):

        if self.is_alive and random.random() < config.
          P_REPRODUCE:
            return Cat()
        return None
```

サンプルコード 9.1　エージェントベースの Malthus 的モデルのプログラム例。

これを実行すると，図 9.1 のようなグラフが得られるはずです。繁殖確率と死亡確率の差の分が，正味の成長確率になるというモデルなので，これは，第 3 章で差分や微分方程式を用いて記述した Malthus 的成長を，確率効果を加え，エージェントベースでモデルしたものに対応します。しかし，大まかに Malthus 的な成長を示していますが，確率的な要素のために，単調な曲線にはなっていないことがわかります。

9.3.6　環境収容力の導入

上で作成した基本的なモデルでは，繁殖確率も死亡確率も一定なので，繁殖確率が上回っている場合は，際限なく個体数が成長できます。しかし，現実には個体数が増えると，資源を巡る争いが増え，死亡確率が上昇するのが一般的です。

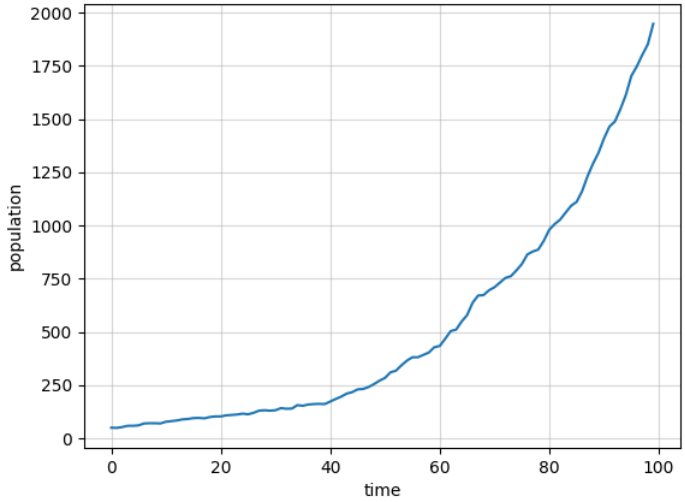


図 9.1　サンプルコード 9.1 の実行結果。

そこで，この効果を反映するために，第 4 章で導入した環境収容力を導入しましょう。

環境収容力は，個体数の大まかな上限を与えます。この性質は，個体数 N が環境収容力 K に等しくなると，繁殖確率と死亡確率が等しくなり，正味の成長確率がゼロになるようにすれば反映できます。これをモデルする一つの方法は，繁殖確率 p_b と死亡確率 p_d に対し，実効的な死亡確率 p_{eff} を

$$p_{\mathrm{eff}} = p_d - (p_d - p_b)\frac{N}{K} \tag{9.1}$$

とすることです (McCarthy et al., 2013)。こうすれば，N/K が小さいときは $p_{\mathrm{eff}} \simeq p_d$ のように実効的な死亡確率は生の死亡確率とほぼ等しいのに対し，$N/K = 1$ となると，$p_{\mathrm{eff}} = p_b$ となり，死亡確率が繁殖確率と等しくなります。

上の密度の依存する死亡確率を実装するには，まず `config.py` に環境収容力に対応するパラメータ `KCAP` を加えましょう。ついでに，効果をわかりやすくするためにパラメータを全体に調整しておきます。

```
# config.py
N0 = 50
NSTEP = 5000
```

```
KCAP   = 200 # NEW
P_REPRODUCE = 0.015
P_DEATH = 0.01
```

続いて，メソッド update を呼ぶ際に，N に対応するそのステップでの個体数を渡すようにしましょう。

```
# main.py
    cat.update(N_current)
```

最後に，それを update で受け，死亡確率を修正します。

```
    def update(self,N): # ←Nを追加

        # （略）

        p_d = config.P_DEATH
        p_b = config.P_REPRODUCE
        K   = config.KCAP

        P_EFF = p_d - ( p_d - p_b ) * N / K

        if random.random() < P_EFF:
            self.is_alive = False
```

N_current を受け取る際，簡単のために N という名前で受け取っています。

図 9.2 は，これらの変更を反映したコードの実効例を示しています。環境収容力 $K = 200$ 付近で，個体数の成長が頭打ちになっていることが確認できます。

9.3.7　ネコの生態の反映

これまでのモデルを基本として，ネコの生態と，不妊化による個体数抑制を反映するロジックを実装していきましょう。まず必要な設定変数をまとめて

```
# config.py
N0 = 100
NSTEP = 5000
KCAP   = 100
#P_REPRODUCE = 0.015
P_PREGNANCY = 0.1 # 妊娠確率
P_DEATH = 0.001 # 死亡率
P_DEATH_K = 0.005 # N=Kでの死亡率
MATURE_AGE = 180 # 成熟年齢
```

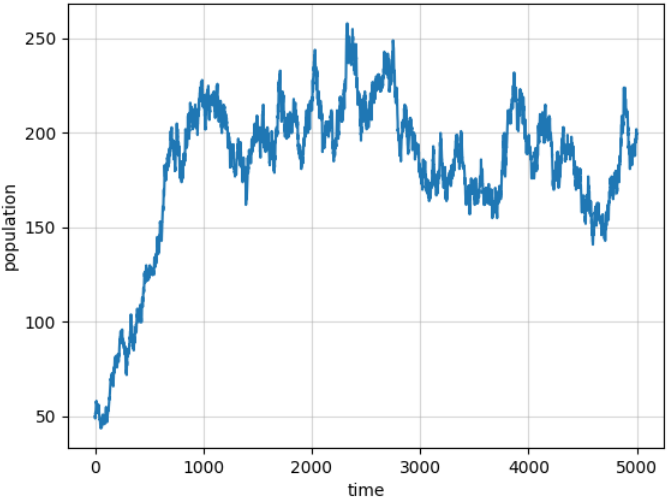


図 9.2　サンプルコード 9.1 に環境収容力の影響を加えた場合の実行例（$K = 200$）。

```
BREEDING_SEASON_LENGTH = 250 # 繁殖期の長さ
LITTER_SIZE = 2 # （メスの）産子数
TRAP_RATE = 5e-4 # 捕獲率
```

とします。変数の意味は，プログラム内のそれらを用いる部分で順に説明していきます。

メイン部分は，以下のように変更します

```
# main.py
import matplotlib.pyplot as plt
import config
import random
from agent import Cat

def main():

    cats = []
    for _ in range(config.N0):
        age = int(300*random.random())
        cats.append(Cat(age))

    N_history = []
```

```
    for step in range(config.NSTEP):
        N_current = len(cats)
        N_history.append(N_current)

        new_cats = []

        day_of_year = ( step % 365 ) + 1

        for cat in cats:

            cat.update(N_current)

            if day_of_year <= config.BREEDING_SEASON_LENGTH:

                offspring = cat.reproduce()
                if offspring is not None:
                    #new_cats.append(offspring)
                    new_cats.extend(offspring)

        cats = [cat for cat in cats if cat.is_alive]
        cats.extend(new_cats)
```

最初に変更した点は，初期のオブジェクトを生成する部分です。初期の個体群に年齢の分布を持たせるため，個体ごとに乱数を振って age を決め，引数として渡しています。

```
    age = int(300*random.random())
    cats.append(Cat(age))
```

次の変更点は，ループ内に加えた day_of_year です。これは日にちを数える変数 step を，1 から 365 までの自然数に変換し，一年のうち何日目かを扱えるようにします。その後，条件文

```
if day_of_year <= config.BREEDING_SEASON_LENGTH:
```

を加えることで，繁殖期（breeding season）の場合のみ，繁殖に関わるメソッド reproduce を呼び出すようにしています。また，返ってくるのは，オブジェクトではなくリストになるよう変更するため，新たな個体のリストに反映する際，リストの要素を追加する append ではなく，リストを結合する extend に変更をしています

```
#new_cats.append(offspring)
new_cats.extend(offspring)
```

初期メソッドにも，いくつかからの新たな属性を加えます。このとき，引数にデフォルト値を含めた age=0 を加えることも忘れないでください。

```
def __init__(self,age=0):
    self.age = age
    self.is_alive = True
    self.is_pregnant = False
    self.pregnancy_time = 0
    self.can_be_pregnant = False
    self.sterilized = False
```

状態更新のメソッドは，以下のように変更します。

```
    def update(self,N):

        if not self.is_alive:
            return

        p_d = config.P_DEATH
        #p_b = config.P_REPRODUCE
        p_b = config.P_DEATH_K

         (中 略)

        if random.random() < config.TRAP_RATE:
            self.sterilized = True

        if self.is_alive and self.age >= config.MATURE_AGE and
           not self.sterilized and not self.is_pregnant:
            self.can_be_pregnant = True

        return
```

新たに加えた条件文の一つ目は，捕獲と不妊化の処理になります。乱数を振り，確率 TRAP_RATE に収まる場合，不妊化の処理を施します。二つ目の条件文は妊娠可能性の判定です。生存している個体のうち，年齢が成熟年齢 MATURE_AGE 以上であり，不妊化しておらず，かつ妊娠していない個体のみ，妊娠可能な個体とします。

最後に，繁殖のメソッドを以下のようにします。

```
    def reproduce(self):

        if self.can_be_pregnant:
            if random.random() < config.P_PREGNANCY:
```

```
                self.is_pregnant = True
                self.can_be_pregnant = False

        elif self.is_pregnant:
            self.pregnancy_time += 1
            if self.pregnancy_time >= 60:
                newborns = []
                self.is_pregnant = False
                self.pregnancy_time = 0
                for _ in range(config.LITTER_SIZE):
                    newborns.append(Cat())
                return newborns
        return None
```

妊娠可能な個体の場合，乱数を振り，妊娠確率 P_PREGNANCY に収まる場合，その個体を妊娠させます。また，そのステップ以前に妊娠していた個体は，カウンター pregnancy_time を 1 つ増やします。そして，繁殖日数が妊娠期間の上限（ここでは 60 としています）に達したら，出産を行います。このとき，（メスの）産子数 LITTER_SIZE の数だけループを回して，新たな個体のオブジェクトを生成し，リストにまとめています。

9.3.8　捕獲率の違いによる比較

捕獲率 TRAP_RATE は，ある個体の 1 日当たりの捕獲確率 π を表しています。1 日に捕獲「されない」確率は $1-\pi$ で，1 年間（365 日）捕獲「されない」確率は $(1-\pi)^{365}$ なので，1 年間のうちに捕獲「される」確率に換算すると

$$P_{\text{trap}} = 1-(1-\pi)^{365} \tag{9.2}$$

となります。この捕獲率を $\pi = 0$，0.02 および 0.005（それぞれ年間にして $P_{\text{trap}} = 0$，0.52 および 0.84）とし，シミュレーションを実行してみます。結果を比較したのが，図 9.3 になります。

まず，不妊処置をしない場合（青線）から見ていきましょう。発情期間を導入したことで，1 年ごとに，個体数の増加の山ができるようになったことが確認できます。繁殖期の外では個体数が減りますが，全体として環境収容力 $K = 100$ を中心とした変動になっていることがわかります。

不妊化を実施した場合，$\pi = 0.002$ のケース（オレンジ線）では，平均的な個体数が 70% 程度まで減っていることがわかります。ただし，個体数は 50 から

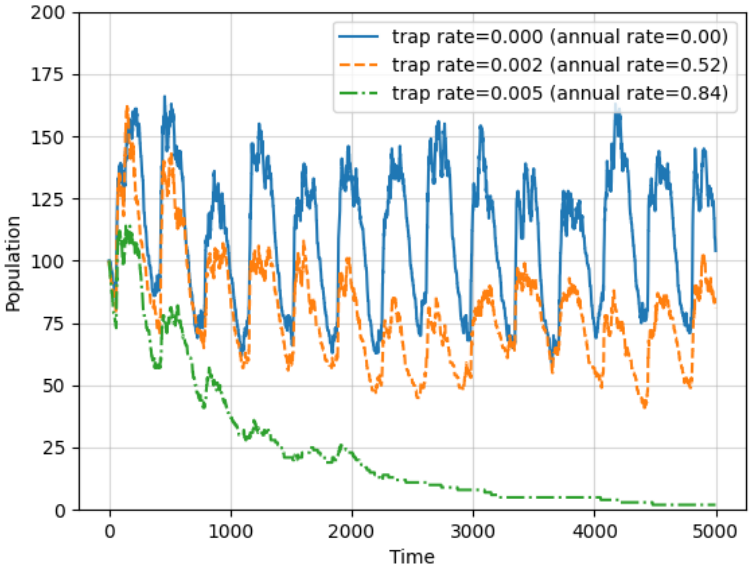


図 9.3　捕獲率を $\pi = 0$，0.02 および 0.005（それぞれ年間にして $P_{\text{trap}} = 0$，0.52 および 0.84）とした場合のシミュレーション例。

100 程度の範囲で安定しており，ゼロに向かう様子は見えません。対して，捕獲率を $\pi = 0.005$ まで上げたケース（緑線）では，個体数は効果的に減少しており，5000 日（約 14 年）後には，ほぼゼロに達しています。

このようにして，不妊化の効果と，必要な効率を評価することができます。ただし，ここでは，捕獲率ごとに 1 回ずつの計算結果を比較していますが，本来は確率的なシミュレーションなので，同じパラメータ設定で複数回のシミュレーションを実行し，統計量を評価しないといけないことに注意してください。また，用いたモデルもかなり単純化したものであり，野良ネコの生態を反映するのに必要な多くの要素が省略されていることにも注意が必要です。ここではメスのみを対象とするモデルを用いましたが，より現実的な解析を行うには，オスを含めたシミュレーションが必要です。また，年齢は繁殖可能性の判定のみに用いましたが，年齢ステージを設け，ステージごとの死亡率を設定しないと，現実的な集団の構成を再現できません。さらに，TNR による不妊化により，繁殖行

動が抑制され，死亡率が低下する効果なども反映されていません。

これらの性質を反映するための拡張は，本章で行ったことの延長として行えます。つまり，調べたい対象をオブジェクトの属性として与え，特定の条件の下でそれらの属性の更新をするメソッドを構築するだけです。必要な属性を加えたり，条件を精巧化したりするなどして，より現実的なモデルを構成してみてください。また，本章の冒頭で紹介した NORA コードは，これらの点を考慮したより詳細なモデルに基づくコードなので，関心のある方はぜひ利用してみてください。

References

Althoff, C. (2018). **独学プログラマー *Python* 言語の基本から仕事のやり方まで** (清水川貴之, Trans.). 日経 BP.

McCarthy, R. J., Levine, S. H., & Reed, J. M. (2013). Estimation of effectiveness of three methods of feral cat population control by use of a simulation model. *Journal of the American Veterinary Medical Association, 243*(4), 502–511.

小高, 知. (2018). ***Python* による数値計算とシミュレーション**. 株式会社 オーム社.

付録 A

解析学

A.1 微分方程式の基礎

A.1.1 一般解と特解

Malthus 方程式は，a を定数として

$$\frac{dy}{dx} = ay \tag{A.1}$$

という形をしていました。これを解くためには，まず両辺を y で割ります。すると

$$\frac{1}{y}\frac{dy}{dx} = a \tag{A.2}$$

となります。これを x で積分し，左辺の逆微分が $\ln y$ であったことを用いると

$$\ln y = ax + C \tag{A.3}$$

となります。ここで積分定数 C が現れます。対数の意味を思い出すと

$$y = e^{ax+C} = e^C e^{ax} \tag{A.4}$$

が得られます。e^C もまた定数なので，これを改めて C で表すと，最終的に

$$y = Ce^{ax} \tag{A.5}$$

となります。定数は他に条件がないと定められません。単に方程式を解けという数学の問題としてできるのは，この一般的な解の形を示すまでです。このよ

うな，積分定数を残したままの微分方程式の解を**一般解（general solution）**といいます。このとき，積分定数は元の微分方程式の階数と同じ数含まれることに注意してください。

これに対し，何らかの条件の下で，定数の値を決めた，具体的（particular）な解のことを**特解（particular solution）**といいます。Malthus 方程式の解に初期値 N_0 を設定した (3.36) は，特解の例です。

A.1.2 線形な方程式と非線形な方程式

Malthus 方程式が持つ一つの大事な性質は，もし y_1 と y_2 がこの方程式の解で

$$\frac{dy_1}{dx} = ay_1, \quad \frac{dy_2}{dx} = ay_2 \tag{A.6}$$

が成り立つなら，これらを足した

$$\frac{d}{dx}(y_1 + y_2) = a(y_1 + y_2) \tag{A.7}$$

も成り立つため，$y_1 + y_2$ という関数もまた同じ方程式の解になっているということです。もっというと，適当な定数 c_1, c_2 をつかって $c_1 y_1 + c_2 y_2$ としても解になります。解を定数倍と足し算をしても解ける微分方程式は，**線形（linear）**であるといいます。もし二種類の生物種がそれぞれ無関係に Malthus モデルに従うとき，それらの個体数を足したものも同じモデルに従うということです。あるいは，単一の種の個体数を適当に $N = N_1 + N_2$ と分けたり，定数倍しても，同じ方程式に従います。これは，線形な方程式に従う量の振る舞いにおいて，その量の絶対値は本質的ではないことを示しています。

対して，ロジスティクス方程式は

$$\frac{dy}{dx} = ay + by^2 \tag{A.8}$$

の形をしていました。これを満たす二つの解 y_1, y_2 についての式の和は

$$\frac{d}{dx}(y_1 + y_2) = a(y_1 + y_2) + b(y_1^2 + y_2^2) \tag{A.9}$$

である一方で，解の和 $y_1 + y_2$ を y として代入した結果は

$$\frac{d}{dx}(y_1 + y_2) = a(y_1 + y_2) + b(y_1^2 + y_2^2 + 2y_1 y_2) \tag{A.10}$$

となり，それぞれ異なる式になります。つまり，y_1, y_2 はそれぞれ解であっても，その和は一般に解ではないのです。このように線形でない方程式は，**非線形（nonlinear）**であると言われます。この場合，Malthus モデルと異なり，個体数を適当に分けて考えると，正しい記述ができないことを意味しています。ロジスティック方程式では，収容力 K に対する個体数の絶対値が大事な役割を果たすことを考えても，納得いくと思います。

このように，全体が単なる部分の和では書けないという性質が，生命のような複雑な存在を可能にするのですが，同時に数学的対象としての取り扱いも，一般に線形な方程式に比べはるかにむつかしくします。

A.1.3 解析解と数値解

Malthus 方程式のような，単純な線形方程式には，上で示したように式変形によって理論的に導ける解が存在します。このような解は，一般的な関数の組み合わせで表現できるため，具体的な数値を代入しなくても，その「姿」を記述することができます。このような解のことを**解析解（analytical solution）**といいます。

対して，非線形方程式では解析解が存在しないことの方が一般的です。ではどうやってその性質を調べるのかという問いに対する代表的な答えが，私たちが本書で行っている，コンピュータなどを用いた可視化や数値計算です。数値計算により，その都度，具体的な数値を用いて求められる解を**数値解（numerical solution）**といいます。このような解も，具体的な値をつないでグラフでその姿を表現することはできますが，それを私たちが知っている関数を組み合わせて厳密に表現することはできません。数値計算を行うのは，単に計算が難しいからではなく，そもそも原理的に手で計算することができない問題の方が多いからでもあるのです。

A.1.4 ロジスティック方程式の解

全体を N^2 で割ります。すると

$$\frac{1}{N^2}\frac{dN}{dt} = \frac{r}{N} - \frac{r}{K} \tag{A.11}$$

となります。ここで，$x = 1/N$ と変数変換すると

$$\frac{dx}{dt} + rx = \frac{r}{K} \tag{A.12}$$

となります。この式は，x に関する線形な方程式になっています。このように，非線形な方程式もうまく変数を変えることで線形な方程式に変換できることがあります。

この式は，もし最後の項がなければ

$$\frac{dx}{dt} + rx = 0 \tag{A.13}$$

という式になります。一階の微分方程式の場合，一般に $dx/dt + f(t)x = 0$ の形の式を，**同次式（homogeneous equation）**といいます。反対に，元の同次でない式は**非同次式（inhomogeneous equation）**と呼ばれます。

線形な非同次式の一般解は，その特解と同次式の一般解を合わせたものとして得られることが分かっています。そこでまず，同次式の一般解を求めましょう。とはいえ，この式は，Malthus 方程式の右辺に負号が着いただけなので，解も

$$x = Ce^{-rt} \tag{A.14}$$

のように簡単に得られます。

次に非同次式の特解を求めましょう。ロジスティック方程式が記述する現象から，方程式が満たすべき具体的な解が想像できます。ロジスティック成長する個体数は，$N = K$ に達すると変化しなくなるため，ロジスティック方程式はそのような状況を表す解を含んでいるはずです。よって $dx/dt = 0$ としてみると

$$x = \frac{1}{K} \tag{A.15}$$

という具体的な解，すなわち特解が見つかります。こうして，x の一般解が

$$x = Ce^{-rt} + \frac{1}{K} \tag{A.16}$$

と得られます。実際にこれが解になっていることは代入して確かめられます。

忘れてはいけないのは，私たちが求めているのは x ではなく $N = 1/x$ なので，分子と分母をひっくり返して一般解を得る必要がありますが，欲しいのはむし

ろ初期条件を満たす特解です。そのためには，x のまま計算を進めて，$t=0$ で，$x(0)=1/N_0$ となる定数を探す方が楽です。これは

$$C=\frac{1}{N_0}-\frac{1}{K} \tag{A.17}$$

であることがわかります。よって

$$\begin{aligned} x&=\left(\frac{1}{N_0}-\frac{1}{K}\right)e^{-rt}+\frac{1}{K}\\ &=\frac{e^{-rt}}{KN_0}\left[(K-N_0)+N_0e^{rt}\right] \end{aligned} \tag{A.18}$$

であり，改めて，この x の逆数が求める解 N なので

$$N=\frac{KN_0e^{rt}}{K-N_0(1-e^{rt})} \tag{A.19}$$

A.2　複素解析の基礎

A.2.1　複素数とは

$i^2=-1$ を満たす数 $i=\sqrt{-1}$ を**虚数単位（imaginary unit）**と呼びます。そして，この i と，二つの実数 x と y を用いて表される数

$$z=x+iy \tag{A.20}$$

を**複素数（complex number）**と呼びます。複素数 (A.20) のうち，x と y をそれぞれ**実部（real part）**および**虚部（imaginary part）**と呼び

$$x=\mathrm{Re}(z),\quad y=\mathrm{Im}(z) \tag{A.21}$$

のように表します。

複素数 $z=x+iy$ に対して，虚部の符号を変え

$$z^*=x-iy \tag{A.22}$$

とする操作を，z の**複素共役（complex conjugate）**と呼びます。(A.22) と同じものを，アスタリスク「$*$」の代わりに $\overline{\cdots}$ を使って $\bar{z}$ と表すこともあります。

複素数 z の絶対値は

$$|z|=\sqrt{x^2+y^2} \tag{A.23}$$

で定義されます。複素数の絶対値は実数であることに注意してください。z と z^* の積を取ると

$$\begin{aligned} zz^* = z^*z &= (x+iy)(x-iy) \\ &= x^2 + ixy - ixy + y^2 \\ &= x^2 + y^2 \\ &= |z|^2 \end{aligned} \tag{A.24}$$

となり，絶対値の二乗が与えられることが確認できます。

付録 B

有限差分と近似の精度

B.1 Taylor 展開

B.1.1 1 次近似

関数 $f(x)$ の点 $x=a$ における微分は，有限の幅 Δx にわたる変化率

$$\frac{f(a+\Delta x)-f(a)}{\Delta x}$$

の極限

$$\left.\frac{df(x)}{dx}\right|_{x=a} = \lim_{\Delta x\to 0}\frac{f(a+\Delta x)-f(a)}{\Delta x} \tag{B.1}$$

として定義されました。ここで，$df(x)/dx|_{x=a}$ は，微分した後に $x=a$ を代入することを意味しています。そうでないと，$f(a)$ はすでに具体的な値が定まったただの数なので，微分すると消えてしまいます。ただ，誤解の恐れがない場合には，それを $df(a)/dx$ のように表記することも一般的ですので，文献を読む際には注意してください。

さて，上の微分の定義は，$df(x)/dx|_{x=a}$ が

$$\left.\frac{df(x)}{dx}\right|_{x=a} = \frac{f(a+\Delta x)-f(a)}{\Delta x} + O(\Delta x) \tag{B.2}$$

と近似できることを意味しています。この右辺の二項目は，$\Delta x\to 0$ の極限で 0 になる量を表します。$\Delta x = x-a$ として (B.2) を変形すると

$$f(x) = f(a) + \left.\frac{df(x)}{dx}\right|_{x=a}(x-a) + O((x-a)^2) \tag{B.3}$$

が得られます。最後の項が，$\Delta x = x - a$ がかかることで，$x - a$ についての 2 次の誤差になっていることに注意してください。この表現は，x での関数 f の値 $f(x)$ を，a での値 $f(a)$ とその導関数により，誤差 $O((x-a)^2)$ の範囲で近似できることを示しており，関数 f の 1 次近似と呼びます。この式を微分すると

$$\frac{df(x)}{dx} = \left.\frac{df(x)}{dx}\right|_{x=a} + O((x-a)) \tag{B.4}$$

となります。$x = a$ を代入した後の項はすべて定数なので，微分の影響を受けないことに注意してください。ここで，$x = a$ とすると，右辺二項目以降は $x - a = 0$ よりすべて消えるので

$$\left.\frac{df(x)}{dx}\right|_{x=a} = \left.\frac{df(x)}{dx}\right|_{x=a} + 0 \tag{B.5}$$

と，つじつまの合う結果になります。

B.1.2 Taylor 級数

ところで，もし関数 $f(x)$ が一次関数（直線）であれば，傾きは一定なので，導関数と平均変化率は一致し，(B.2) は誤差なく一致するはずです。言い換えると，誤差は二階以上の微分に由来するものだということです。そのため (B.3) はより正確には

$$f(x) = f(a) + \left.\frac{df(x)}{dx}\right|_{x=a}(x-a) + c\left.\frac{d^2f(x)}{dx^2}\right|_{x=a}(x-a)^2 + O((x-a)^3) \tag{B.6}$$

と書けます。c はまだ未知の定数ですが，次のようにして決められます。この式を微分すると

$$\frac{df}{dx} = \left.\frac{df(x)}{dx}\right|_{x=a} + 2c\left.\frac{d^2f(x)}{dx^2}\right|_{x=a}(x-a) + O((x-a)^2) \tag{B.7}$$

となり，さらにもう一回微分することで

$$\frac{d^2f}{dx^2} = 2c\left.\frac{d^2f(x)}{dx^2}\right|_{x=a} + O((x-a)) \tag{B.8}$$

が得られます。ここでまた，$x = a$ を代入すれば，右辺二項目以降が消えて両辺が一致するはずですが，そのためには $c = 1/2$ でなくてはなりません。このことから，定数 c が決められます。

この議論を繰り返すと，関数 $f(x)$ は

$$f(x) = f(a) + \left.\frac{df(x)}{dx}\right|_{x=a}(x-a) + \frac{1}{2!}\left.\frac{d^2f(x)}{dx^2}\right|_{x=a}(x-a)^2 + \frac{1}{3!}\left.\frac{d^3f(x)}{dx^3}\right|_{x=a}(x-a)^3 + \cdots$$
$$= \sum_{k=0}^{\infty}\frac{1}{k!}\left.\frac{d^kf(x)}{dx^k}\right|_{x=a}(x-a)^k \tag{B.9}$$

と展開できることが示せます。ここで，$k! = k\cdot(k-1)\cdot(k-2)\cdots 1$ は k の階乗です。式 (B.9) の展開を，関数 f の $x=a$ 周りでの **Taylor 展開（Taylor expansion）**と呼びます。Taylor 展開は，数理的な解析において最も基本的な道具の一つになります。

B.2 数値積分

B.2.1 Euler 法

微分方程式

$$\frac{dx}{dt} = f(x,t) \tag{B.10}$$

の解 $x(t)$ を数値的に求めたいとします。これを，離散的な式

$$\frac{x(t+\Delta t)-x(t)}{\Delta t} = f(x(t),t) \tag{B.11}$$

で近似し，変形することで

$$x(t+\Delta t) = x(t) + f(x(t),t)\Delta t \tag{B.12}$$

が得られます。これを初期値から始めて繰り返し計算すれば，任意のステップでの値が得られます。このような数値積分法を，**Euler 法（Euler method）**といいます。

Euler 法は，微分を有限幅の平均変化率で置き換える 1 次近似 (B.3) に基づいているため，$(\Delta t)^2$ のオーダーの誤差が含まれます。特にこの方法で軌道を計算するためには，多数回（100 回，1000 回あるいはもっと多く）同じ計算をするため，誤差が蓄積していき，正しい解からどんどんずれて行ってしまいます。

刻み幅を最大限小さく取ればいいんじゃないかと思われるかもしれません。ですが，そうもいかない理由があります。時間刻み幅が小さくなるということ

はシミュレーション内で 1 ステップあたりに進む時間が短くなることを意味します。そのため，あまりに細かい刻み幅を用いると，まともな時間内に計算が終わらなくなってしまいます。また，最終的にはコンピュータが扱える最小の値という限界もあります。

B.2.2 改良 Euler 法

こうしたことから，ある程度の刻み幅を保ちながら，十分な精度で計算できる方法が求められます。一つの改良は，$x(t)$ と，求めたい $x(t+\Delta t)$ の中間の値 $x(t+\Delta t/2)$ を用いて

$$x(t+\Delta t) = x(t) + f(x(t+\Delta t/2), t+\Delta t/2)\Delta t \tag{B.13}$$

とすることです。これを，改良 Euler 法とか，2 次の **Runge-Kutta 法（Runge-Kutta method）**と呼びます。

B.2.3 4 次の Runge-Kutta 法

改良 Euler 法は，元の Euler 法よりは精度が高いのですが，一般的な数値計算においてはまだ十分ではありません。より一般的に用いられるのが，4 次の Runge-Kutta 法です。4 次の Runge-Kutta 法では，1 ステップの間に以下の 4 つの傾き k_1, k_2, k_3, k_4 を評価します。

$$k_1 = f(x(t),\, t) \tag{B.14}$$

$$k_2 = f\left(x(t) + \frac{\Delta t}{2} k_1,\, t + \frac{\Delta t}{2}\right) \tag{B.15}$$

$$k_3 = f\left(x(t) + \frac{\Delta t}{2} k_2,\, t + \frac{\Delta t}{2}\right) \tag{B.16}$$

$$k_4 = f(x(t) + \Delta t\, k_3,\, t + \Delta t) \tag{B.17}$$

これら 4 つの傾きを，次のような重みづけをして足し合わせ，次ステップの値を更新します。

$$x(t+\Delta t) = x(t) + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right) \tag{B.18}$$

付録 C

線形代数

第 5 章で扱った行列によるベクトルの変換を体系的に調べる分野は，**線形代数（linear algebra）**と呼ばれます。この章では，本文の内容をより深く理解するための，線形代数の基礎を説明します。

C.1 行列の基本

本文で触れた部分のおさらいも含め，行列に関する基本的な概念を説明します。

C.1.1 対角行列と単位行列

推移行列の話をした§ 5.1 では

$$\begin{aligned} y_1 &= ax_1 + bx_2 \\ y_2 &= cx_1 + dx_2 \end{aligned} \tag{C.1}$$

の形の式を，行列

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \tag{C.2}$$

によるベクトル

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} \tag{C.3}$$

の間の変換式

$$y = Ax \tag{C.4}$$

によって表しました。ここでもし，$b=c=0$ であったら，(C.1) は

$$\begin{aligned} y_1 &= ax_1 \\ y_2 &= dx_2 \end{aligned} \tag{C.5}$$

となって，ベクトルの一つ目と二つ目の要素は交じり合うことなくそれぞれ独立になります。この時の対応する変換行列は

$$A = \begin{pmatrix} a & 0 \\ 0 & d \end{pmatrix} \tag{C.6}$$

で，対角の成分以外はゼロになります。このような行列のことを，**対角行列（diagonal matrix）**と呼びます。さらにそのゼロでない要素がすべて 1 ならば

$$\begin{aligned} y_1 &= x_1 \\ y_2 &= x_2 \end{aligned} \tag{C.7}$$

となり，ベクトルは何も変換を受けないことになります。この場合の，対角成分がすべて 1 でそれ以外はすべて 0 の正方行列

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \tag{C.8}$$

は，**単位行列（unit matrix）**と呼ばれます。

今，2×2 行列の場合のみを考えていますが，対角行列や単位行列の定義は，任意の次元の正方行列にあてはまります。

C.1.2 行列の可換性

行列の積については本文で説明しましたが，これに関する基本的な性質で非常に重要なものがあります。それは，二つの行列 A と B の積で，AB と BA では，一般に結果が異なるということです。対角行列

$$A = \begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix}, \quad B = \begin{pmatrix} c & 0 \\ 0 & d \end{pmatrix} \tag{C.9}$$

同士の積であれば

$$\begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix}\begin{pmatrix} c & 0 \\ 0 & d \end{pmatrix} = \begin{pmatrix} c & 0 \\ 0 & d \end{pmatrix}\begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix} = \begin{pmatrix} ac & 0 \\ 0 & bd \end{pmatrix} \tag{C.10}$$

となるので

$$AB - BA = 0 \tag{C.11}$$

が成り立ちます。ここで，右辺の 0 は**ゼロ行列（zero matrix）**と呼ばれる行列

$$0 = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \tag{C.12}$$

であり，数 0 とは異なります（混同しないように O などの別の記号を用いることもありますが，本書では 0 を使います）。関係 (C.11) が成り立つ行列同士は，お互いに**可換（commutative）**であるといいます。しかし，一般にはそうではなく，可換でない行列同士は，**非可換（non-commutative）**であるといいます。

これまで行列の説明において，ベクトルを変換させるものだという側面を強調してきましたが，その観点からみると，非可換であることは，異なる仕方で状態を変換させるとき，変換の順序によって結果が変わるということに対応しています。例えばルービックキューブの一つの列を前後に回してから左右に回転するのと，左右に回転してから前後に回すのでは，結果が違うことがわかるでしょう。個体群のモデルの例も考えてみましょう。年齢ステージが二つの分布に，推移行列

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix} \tag{C.13}$$

をはじめに作用した後に，後から，二つ目のステージの個体をゼロにする行列

$$B = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \tag{C.14}$$

を作用すると，合わせて

$$BA = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 2 \\ 0 & 0 \end{pmatrix} \tag{C.15}$$

という変換になります。対して，これらの順序を入れ替えたものは

$$AB = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \tag{C.16}$$

となり，すべてのステージで個体数をゼロにする変換になります。したがってこの場合

$$AB - BA \neq 0 \tag{C.17}$$

であり，A と B は非可換ということになります。

C.1.3 逆行列

第 5 章の中で，四則演算のうち，割り算を除く演算に対応する行列の演算（和と積）については説明しました。ここでは残りの割り算に対応する演算を説明します。

§ 5.1 では，$N(t+1) = AN(t)$ を通して過去あるいは現在の状態 $N(t)$ から未来の状態 $N(t+1)$ を調べる方法を考えました。しかしもし仮に行列の割り算ができたなら，$N(t+1)/A = N(t)$ のようにして，未来あるいは現在の状態 $N(t+1)$ から過去の状態 $N(t)$ を調べる式に変形できるようになります。

実際にはこのように行列の割り算を定義できないのですが，$y = Ax$ という式に対し，何らかの行列 B を左から演算して

$$By = BAx = x \tag{C.18}$$

となれば，つまり $BA = I$ となれば，求めるものと同じ結果が得られます。どんな正方行列に対してもこのような行列が存在すわけではないのですが，正方行列 A に対し

$$AB = BA = I \tag{C.19}$$

となる行列 B が存在する場合には，A は**正則（regular）**であるといいます。またこのとき，B を A の**逆行列（inverse matrix）**といい，A^{-1} のように表します。正則であることを，**非特異（non-singular）**であるとか，**可逆（invertible）**であるともいいます。

2 次の正方行列の逆行列は比較的簡単に求められ，行列 (C.2) の逆行列は

$$A^{-1} = \frac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \tag{C.20}$$

という公式を使って求められます。実際に $AA^{-1} = A^{-1}A = I$ が成り立つことは練習問題として計算して確かめてみてください。

公式 (C.20) に含まれる因子

$$\det A = ad - bc \tag{C.21}$$

は，行列 A の**行列式（determinant）**と呼ばれます。同じものを $|A|$ のように表現することも一般的です。公式 (C.20) からわかるように，$ad - bc = 0$ だとゼ

ロでの割り算が含まれることになるため，逆行列は定義できません。反対に，$ad-bc\neq 0$ であれば逆行列が適切に定義できることがわかります。

次元が 2 次元よりも大きい場合，行列式や逆行列を求める式はもっと複雑になります。しかし，行列式がゼロでないことが，逆行列が存在する条件であることは同じです。

C.2　固有値問題

本文の § 5.2 で説明したように，正方行列 A と，0 でないベクトル x，そしてある数 λ に対し

$$Ax=\lambda x \tag{C.22}$$

が成り立つとき，λ を A の固有値，$\boldsymbol{x}$ を固有値 λ に属する固有ベクトルというのでした。また，与えられた正方行列 A の固有値を求める問題を，固有値問題といいました。ここでは，2 次の正方行列を具体例として，この固有値問題を解析的に解く流れを説明します。

式 (C.22) は，単位行列 I を用いて

$$(A-\lambda I)x=0 \tag{C.23}$$

と書き換えられます。この $(A-\lambda I)$ もまた正方行列です。もし (C.23) の左辺の行列 $(A-\lambda I)$ に逆行列が存在すれば，それを左からかけることで $Ix=x=0$ となってしまうので，これが $x=0$ でない（非自明な）解を持つためには，$(A-\lambda I)$ に逆行列が存在しないことが必要です。逆行列が存在しない条件は，$\det(A-\lambda I)=0$ でした。反対に，$\det(A-\lambda I)=0$ であれば，(C.23) が自明でない解を持つことが言えます。よって，n 次正方行列 A に対し

$$\det(A-\lambda I)=0 \tag{C.24}$$

と λ が A の固有値であることは同じ（同値）です。つまり，式 (C.24) を解けば，固有値が求まるということです。この式 (C.24) を，**固有値方程式（eigenvalue equation）**と呼びます。

2 次の正方行列については，固有値方程式は

$$\det(A-\lambda I)=\det\begin{pmatrix}a-\lambda & b\\ c & d-\lambda\end{pmatrix} \tag{C.25}$$

であり，公式 (C.21) を利用して

$$\begin{aligned}\det(A-\lambda I) =&(a-\lambda)(d-\lambda)-bc\\ =&\lambda^2-(a+d)\lambda+(ad-bc)\\ =&0\end{aligned} \tag{C.26}$$

となります。つまり，この場合の固有値方程式は 2 次方程式なので，学校で習う解の公式を使って

$$\lambda=\frac{(a+d)\pm\sqrt{(a+d)^2-4(ad-bc)}}{2} \tag{C.27}$$

から固有値が求められます。

References

長谷川, 浩. (2015). **線型代数 *[改訂版]***. 日本評論社.

付録 D

Python による出力と描画

D.1 グラフ作成

D.1.1 線のスタイルとマーカー

複数の曲線を同じグラフに加える際には，それぞれの曲線のスタイルを変えることが推奨されます。色を変えるのが一番わかりやすい場合が多いですが，白黒で印刷した場合や，色覚の個人差に配慮して，線の形状自体を変えることが望ましいです。`matplotlib` では，線の形状は `linestyle` あるいはその省略として `ls` により指定できます。主な線の形状は，表 D.1 の通りです。

表 D.1　Matplotlib の主な linestyle

スタイル名	文字列指定	省略形	形状
実線	`'solid'`	`'-'`	————
破線	`'dashed'`	`'-'`	- - - -
一点鎖線	`'dashdot'`	`'-.'`	-.-.
点線	`'dotted'`	`':'`	·····
線なし	`'none'`	`' '`	表示なし

D.1.2 直線の描画

グラフに直線を加えるには，水平方向（horizontal）の直線の場合は axhline，垂直方向（vertical）の場合は axvline という関数を使って

```
plt.figure ( figsize =(8, 6))

hline = 0.5
vline = 0.2
plt.axhline(hline, linestyle ='-',  color='r', label=f'y={
  hline}')
plt.axvline(vline, linestyle ='--', color='b', label=f'x={
  vline}')
```

のようにします。

D.1.3 等高線

等高線を加えるには，contour という関数を使います。この関数は，入力されたデータの値を基に，等高線を描画します。

```
plt.figure (figsize =(8, 6))

x = np.linspace(0, 1, 100)
y = np.linspace(0, 1, 100)
X, Y = np.meshgrid(x, y)
Z = X**2 + Y**2
plt.contour(X, Y, Z, cmap='viridis', levels=10)
```

D.2 文字列のフォーマット

D.2.1 数値のフォーマット

計算途中の変数やモデルのパラメータを読みやすく表示するには，Python 3.6 以降で標準的に用いられている **f 文字列（f-string）**が便利です。文字列の前に f を置き，変数を波括弧{ }で囲むだけで値が埋め込まれます。さらに波括弧の中でコロン（:）に続けて書式を記述することで，桁数や小数点以下の表示を指定できます。主な書式は，表 D.2 にまとめてあります。

表 D.2　Python 書式指定（f 文字列）の主な記述例

目的	書き方（例）	出力結果
基本	`f"{123:d}"`	123
ゼロ埋め（指定桁数）	`f"{7:03d}"`	007
小数点以下の桁数指定	`f"{3.1415:.2f}"`	3.14
3 桁ごとのカンマ区切り	`f"{1000000:,}"`	1,000,000
パーセント表記	`f"{0.123:.1%}"`	12.3%

D.2.2　TeX 形式

Matplotlib では，グラフのタイトルや軸ラベルに TEX 形式の数式を用いれます。そのためには，文字列の先頭に接頭辞 r を付け，r"..."の形で記述し，該当箇所をドル記号（$）で挟みます。

```
# 例
plt.xlabel(r'$\tau = 0.5$')
plt.title(r'$\frac{dx}{dt} = f(x)$')
```

こうすることで，「$\tau = 0.5$」や「$\frac{dx}{dt} = f(x)$」のような TEX 形式の表現がそのまま出力されるようになります。

さらに，シミュレーション中の変数の値を TEX 数式の中に埋め込みたい場合は，f 文字列と組み合わせて fr"..."（または rf"..."）と記述します。

```
alpha = 0.125
label_text = fr'$\alpha = {alpha:.2f}$'
```

ただし，注意点があります。TEX のコマンドで多用される波括弧などは，f 文字列の「変数置換フィールド」と構文が衝突してしまいます。そのため，TEX 本来の波括弧を文字として出力したい箇所では，波括弧を二重（{{および}}）にする（エスケープする）必要があります。

```
# 例
plt.xlabel(rf'$N_{i+1}={N[i+1]:.2f}$') # 間違い
plt.xlabel(rf'$N_{{i+1}}={N[i+1]:.2f}$') # 正しい書き方
```

References

Althoff, C. (2018). **独学プログラマー** *Python* **言語の基本から仕事のやり方まで** (清水川貴之, Trans.). 日経 BP.

国本大悟 & 須藤秋良. (2023). **スッキリわかる** *Python* **入門** (second). インプレス.

付録 E

一部ソフトウェアの利用法

E.1 VS Code

E.1.1 実行方法

ファイルにプログラムを記述しても，それをファイルとして保存してから実行しないと，編集結果が反映されないため注意しましょう。

E.1.2 Tips

- 短い範囲をテスト実行したいなら、範囲選択して shift+enter でファイルを保存せずとも実行できる
- file>auto save　をオンにしておくと便利
- 右下の OVR アイコンをクリックすれば overtype モードを終了できる

付録 F

練習問題の答え

5.1：16，4
5.2：9，2

索引（記号）

∫, 44
∑, 28
log, 52

索引（英語）

ABM, 164
agent, 163
agent-based model, 164
algorithm, 5
alien species, 117
Allee effect, 61
analytical solution, 185
append, 17
arange, 18
argument, 14
array, 18
asymptotically stable, 109
axhline, 200
axvline, 200

birth rate, 30

carrying capacity, 58
class, 164
coexistence equilibrium point, 97
column, 70
commutative, 195
complex conjugate, 187
complex number, 77, 187
component, 70
continuity equation, 147
contour, 200
contraceptive vaccine, 92

def, 14
density distribution, 144
derivative, 40
determinant, 196
deterministic, 129
diagonal matrix, 194
dictionary, 19
differential equation, 45
diffusion, 147
diffusion coefficient, 131, 148
diffusion equation, 148
dimension, 32
discrete, 28
dominant eigenvalue, 76
dot, 75

ecosystem, 95
eigenvalue, 76, 197
eigenvalue equation, 197

eigenvalue problem, 77, 197
eigenvector, 76, 197
elasticity, 79
element, 16, 70
entropy, 159
equilibrium point, 59
Euler method, 47, 191
Euler's number, 38
extend, 17

f-string, 200
Fisher-KPP equation, 154
float, 11
function, 13
functional response, 113

Gaussian distribution, 137
growth rate, 30

IBM, 163
imaginary unit, 77, 187
index, 17
individual-based model, 163
inner product, 73
instance, 164
int, 11
integer, 11
integral, 44
integral curve, 102
integral symbol, 44
intrinsic rate of increase, 38
invasion, 62, 153
invertible, 196
isocline, 97

Jacobian matrix, 111

key, 20

len, 17
limit cycle, 116
linear algebra, 193
linearization, 110
linspace, 18
list, 16
logarithm, 51
logistic equation, 58
Lotka-Volterra equations, 96

matrix, 69
mesopredator, 116, 117
model, 1
Monte Carlo method, 130
mortality rate, 30

Nagumo equation, 154
Napier's constant, 38
ndarray, 18
neutrally stable, 112
non-commutative, 195
non-singular, 196
NORA, 164
norm, 100
normal distribution, 137
normalization, 34
nullcline, 97
numerical simulation, 2

numerical solution, 185

object, 164
orbit, 102

paradox of enrichment, 116
partial derivative, 79, 145
perturbation, 109
population dynamics, iii
population ecology, iii
predator, 96
prey, 96
primitive function, 40
probability density function, 136
projection matrix, 70
pseudorandom numbers, 131

quiver, 100

random walk, 130
reaction diffusion equation, 153
real number, 11
regular, 196
remove, 17
return value, 14
Rosenzweig-MacArthur model, 114
row, 70
Runge-Kutta method, 192

sensitivity, 79
sensitivity analysis, 79
SIT, 123, 156
size, 17
source term, 153
square matrix, 70
stable age distribution, 77
sterile insect release technique, 123
stochasticity, 130
string, 11
summation symbol, 28

tangent line, 42
transition matrix, 70
traveling wave solution, 154

unit, 32
unit matrix, 194

variance, 131
vector field, 100

zero matrix, 195
zeros, 33

索引（日本語）

Allee 効果, 61

Euler の数, 38
Euler 法, 47, 191

Fisher-KPP 方程式, 154
f 文字列, 200

Gauss 分布, 137

Jacobi 行列, 111

Lotka-Volterra 方程式, 96

Malthus 係数, 46
Malthus モデル, 34
Monte Carlo 法, 130

Nagumo 方程式, 154
Napier の定数, 38

Rosenzweig-MacArthur モデル, 114
Runge-Kutta 法, 192

Taylor 展開, 191

アイソクライン, 97
アルゴリズム, 5
安定齢分布, 77
インスタンス, 164
インデックス, 17
エントロピー, 159
エージェント, 163
エージェントベースモデル, 164
オブジェクト, 164
解析解, 185
可換, 195
可逆, 196
拡散, 147
拡散係数, 131, 148
拡散方程式, 148
確率性, 130
確率密度分布, 136
環境収容力, 58
関数, 13
感度, 79
感度分析, 79
外来種, 117
規格化, 34
軌道, 102

機能的応答, 113
共存平衡点, 97
虚数単位, 77, 187
キー, 20
擬似乱数, 131
行, 70
行列, 69
行列式, 196
クラス, 164
決定論的, 129
原始関数, 40
個体群生態学, iii
個体群動態論, iii
個体ベースモデル, 163
固有値, 76, 197
固有値方程式, 197
固有値問題, 77, 197
固有ベクトル, 76, 197
サイズ, 17
最大固有値, 76
死亡率, 30
射影行列, 70
出生率, 30
進行波解, 154
侵入, 62, 153
次元, 32
辞書, 19
実数, 11
推移行列, 70
数値解, 185
数値シミュレーション, 2
正規分布, 137
整数, 11
正則, 196
生態系, 95
成長率, 30
成分, 70
正方行列, 70
積分, 44
積分記号, 44
積分曲線, 102
接線, 42
摂動, 109
線形化, 110
線形代数, 193
ゼロ行列, 195
漸近安定, 109
総和記号, 28
ソース項, 153
対角行列, 194
対数, 51
単位, 32
単位行列, 194
弾力性, 79
中位捕食者, 116, 117
中立安定, 112
導関数, 40
内積, 73
内的増加率, 38
ヌルクライン, 97
ノルム, 100
配列, 18

反応拡散方程式, 153
非可換, 195
引数, 14
被食者, 96
非特異, 196
微分方程式, 45
富栄養化のパラドクス, 116
複素共役, 187
複素数, 77, 187
不妊化ワクチン, 92
不妊虫放飼法, 123, 156
分散, 131
平衡点, 59
偏微分, 79, 145
ベクトル場, 100
捕食者, 96
密度分布, 144
文字列, 11
モデル, 1
戻り値, 14
要素, 16, 70
ランダムウォーク, 130
離散的, 28
リスト, 16
リミットサイクル, 116
列, 70
連続の式, 147
ロジスティック方程式, 58